Machine Learning · Tree Ensembles

Decision Trees and Ensembles

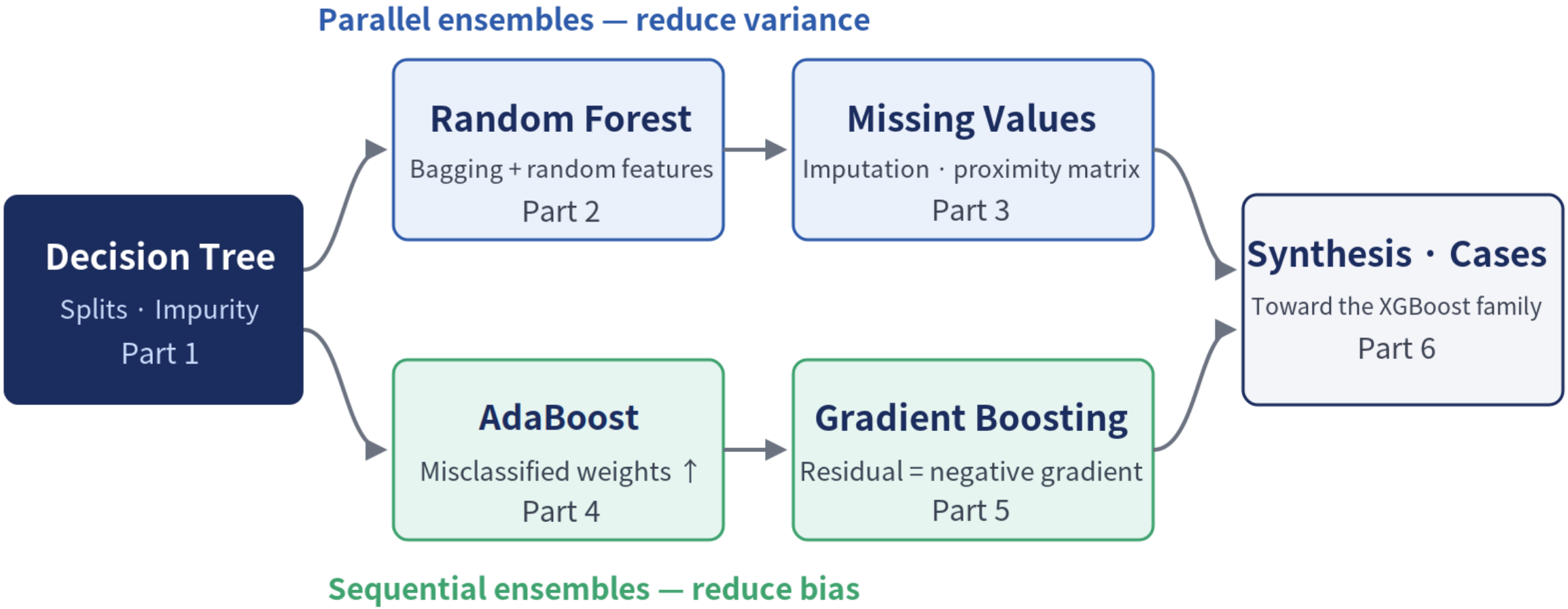
Decision Tree → Random Forest → AdaBoost → Gradient Boosting

Splits and Impurity · Bagging and Missing Values · Boosting and Residual Learning

Five lecture sets (decision trees · RF missing values · AdaBoost · gradient boosting ×2) reordered into one flow, with added algorithm formulas, current scikit-learn behavior, and finance cases.

Today's Map

Starting from a single tree, the method branches into parallel (bagging) and sequential (boosting)



Three Takeaways for Today

1

A tree greedily picks "the question that reduces impurity the most"

Splits are scored by Gini or entropy. Grown deep, a tree memorizes the training data and overfits.

2

Bagging (RF) reduces variance; boosting reduces bias

RF averages diverse trees to cut instability; boosting keeps fixing the previous model's mistakes (weights, residuals).

3

Gradient boosting residual = negative gradient of the loss

This one line links AdaBoost and GBM, and bridges to XGBoost · LightGBM · CatBoost.

The original examples (heart disease · weight) are kept; the order is tree → RF → missing values → boosting

Background ① Supervised Learning · Classification and Regression · Loss Functions

Supervised learning

- Learn f(X) ≈ y from pairs of inputs X (features) and answers y (target, label)
- Data used for learning = training set; data for checking performance = test set

Classification vs regression

- Classification: y is a category — heart disease Yes/No, default or not
- Regression: y is continuous — body weight, next month's return

Loss function — the ruler for "how wrong" a model is

- Regression mostly uses squared error $(y - \hat{y})^2$; classification uses misclassification rate or log loss
- Trees split on impurity (Gini · entropy · MSE); boosting uses the gradient of the loss → the core of Part 5

Background ② Overfitting and Bias–Variance

Model error splits into bias², variance, and irreducible noise

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\text{Bias}^2}_{\text{too simple}} + \underbrace{\text{Var}}_{\text{too sensitive}} + \underbrace{\sigma_\varepsilon^2}_{\text{noise}}$$

High-bias model

- Too simple to capture the pattern (underfitting)
- e.g. a depth-1 tree (stump)
- The error boosting targets

High-variance model

- Predictions swing widely when the training data change slightly (overfitting)
- e.g. a deep tree grown to the end
- The error bagging · RF target

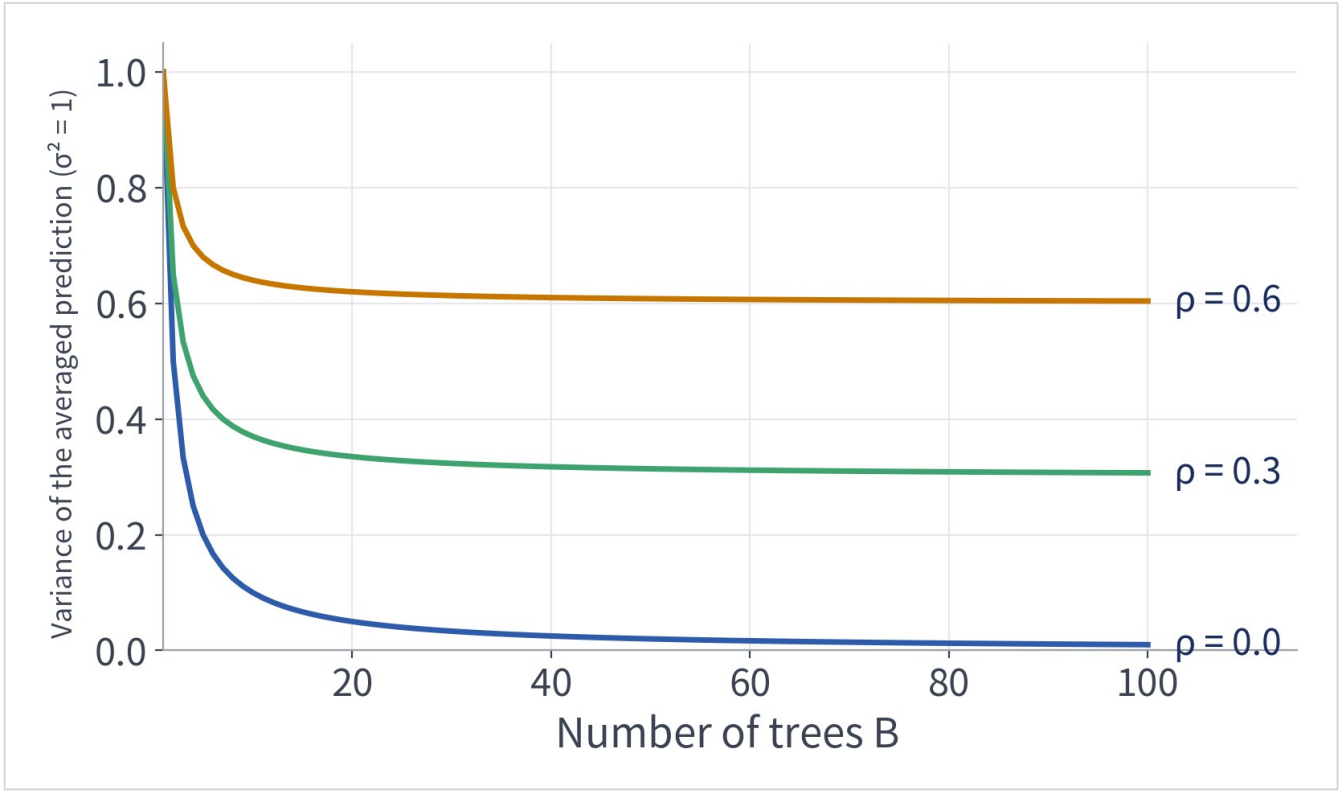
Background ③ Why Do Ensembles Work?

Averaging B models reduces variance, but the part due to inter-model correlation ρ remains

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

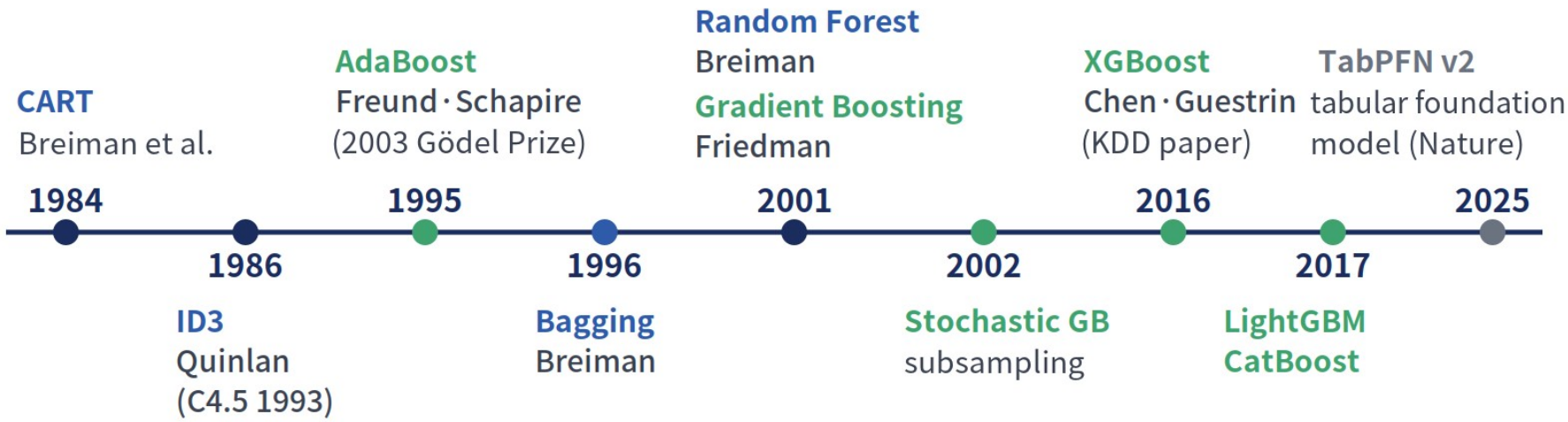
How to read it

- If ρ = 0, variance → 0 as B grows
- If ρ is large, the first term remains no matter how much you average
- So RF also samples features at random to lower the correlation between trees (Part 2)



Episode — 40 Years of Tree Ensembles

From CART (1984) to XGBoost (2016): the order in which today's algorithms appeared



Breiman created bagging and RF; Friedman created GBM. Both co-authored CART. The AdaBoost paper won the 2003 Gödel Prize in theoretical computer science.

01

Part 1 · Decision Trees

Decision Trees — Splitting Data by Twenty Questions

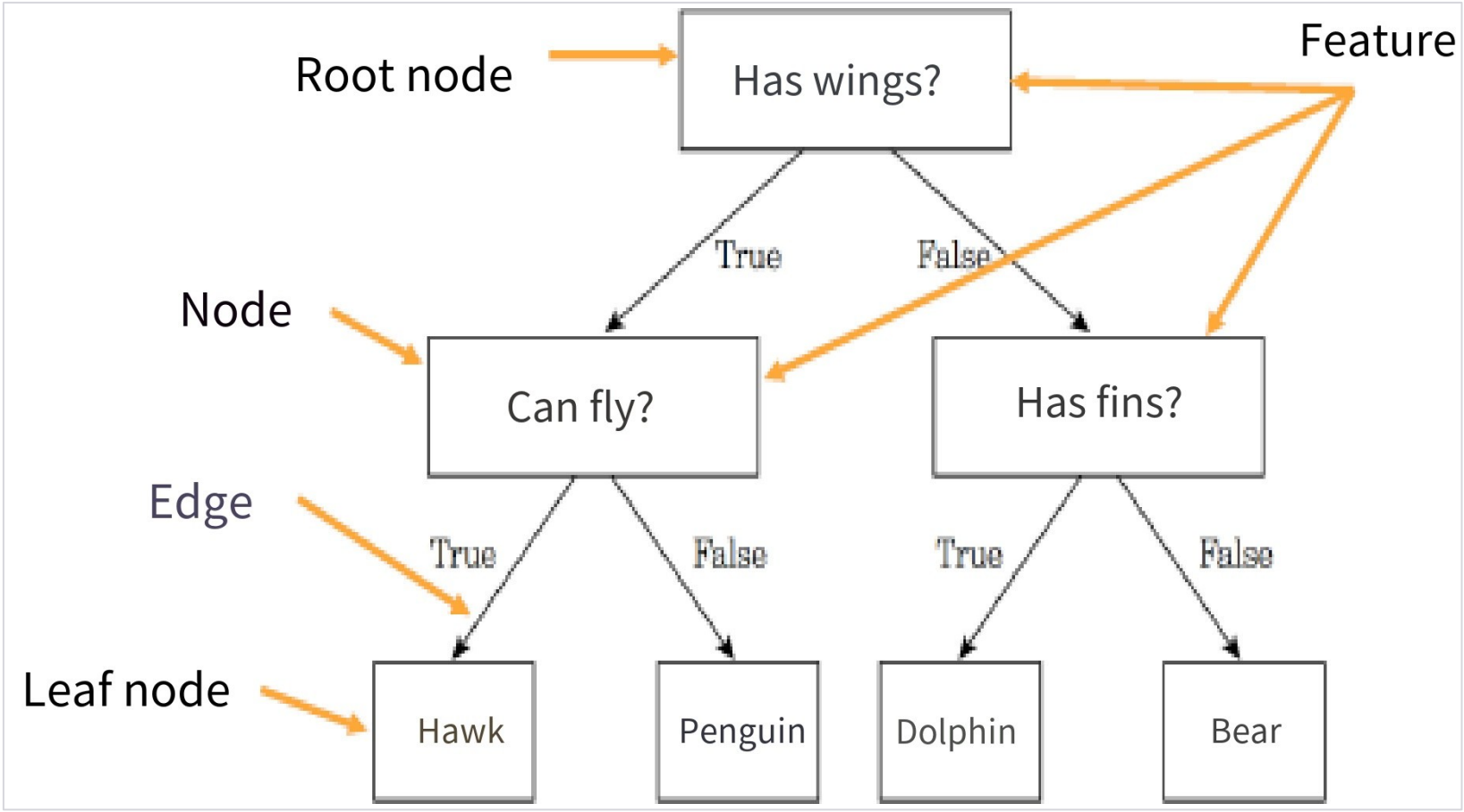
Structure · impurity · information gain · heart disease example · numeric/ordinal/categorical features · controlling overfitting

A Decision Tree = Twenty Questions

Repeatedly split the data in two with one question (feature)

Terms

- Root node: the first question asked of all the data
- Node: an intermediate question
- Edge: a branch for the True/False answer
- Leaf node: an end that is not split further — the prediction comes from here

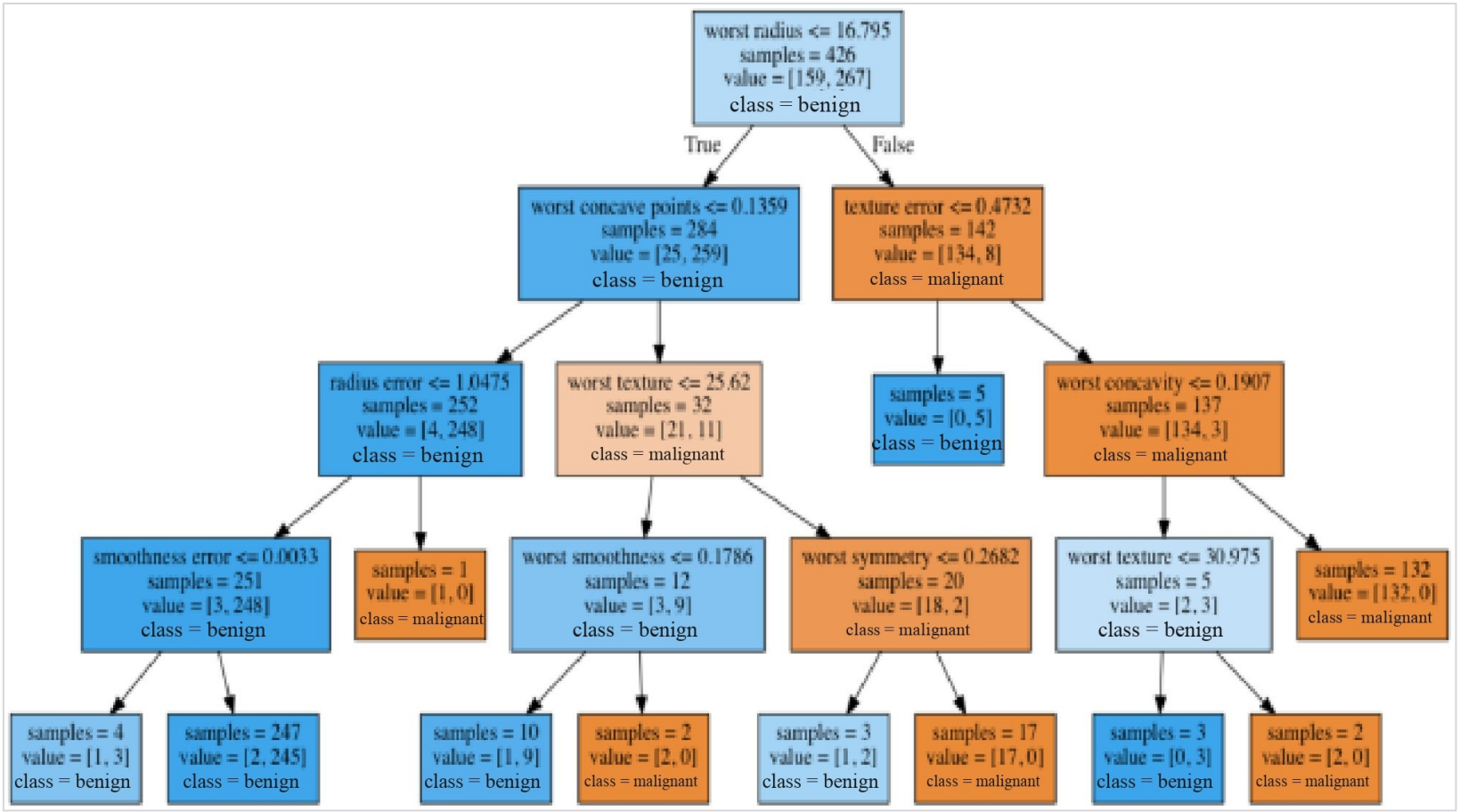


Decision Tree Analysis — Breast Cancer Data

scikit-learn load_breast_cancer: 569 samples · 30 features · malignant/benign

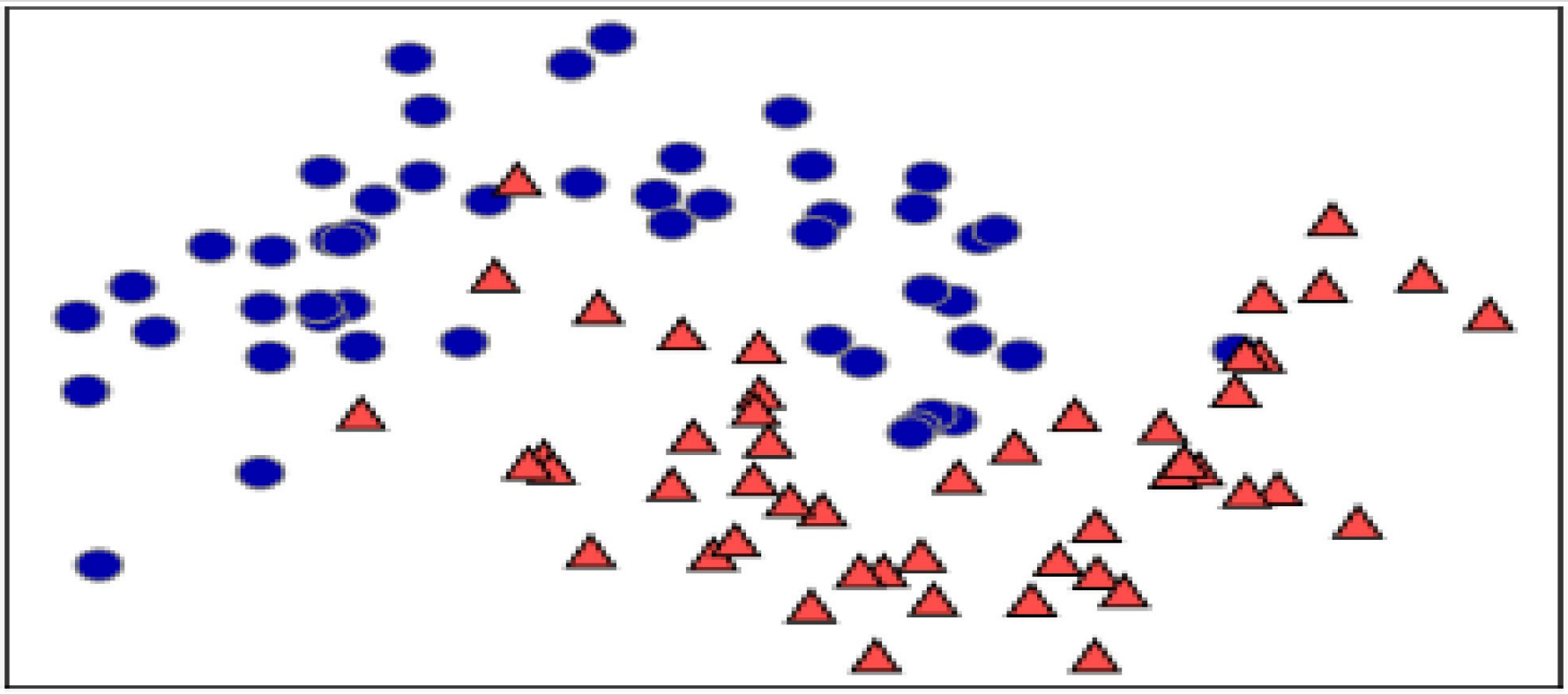
Reading a node

- samples: number of samples reaching this node (root = 426 training samples)
- value: counts of [malignant, benign]
- class: majority class = this node's prediction
- The darker the color, the purer the node



Building a Decision Tree — two_moons Data

Each node learns a yes/no test such as "Is feature i greater than a?"



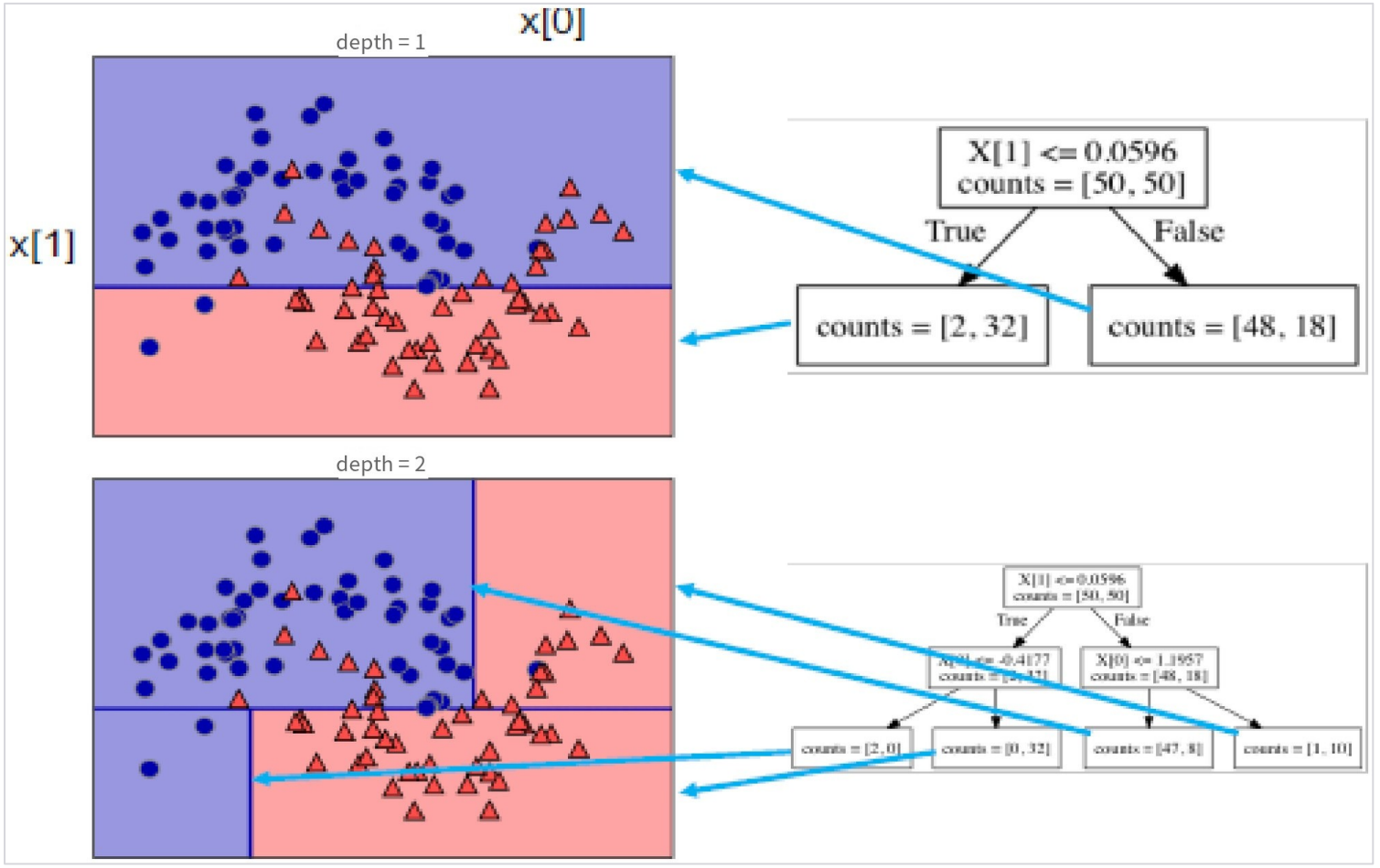
For a continuous feature, all candidate thresholds a are compared and the one that best separates the two groups is chosen.

Training a Decision Tree — Depth 1 → Depth 2

Each split cuts a region once more

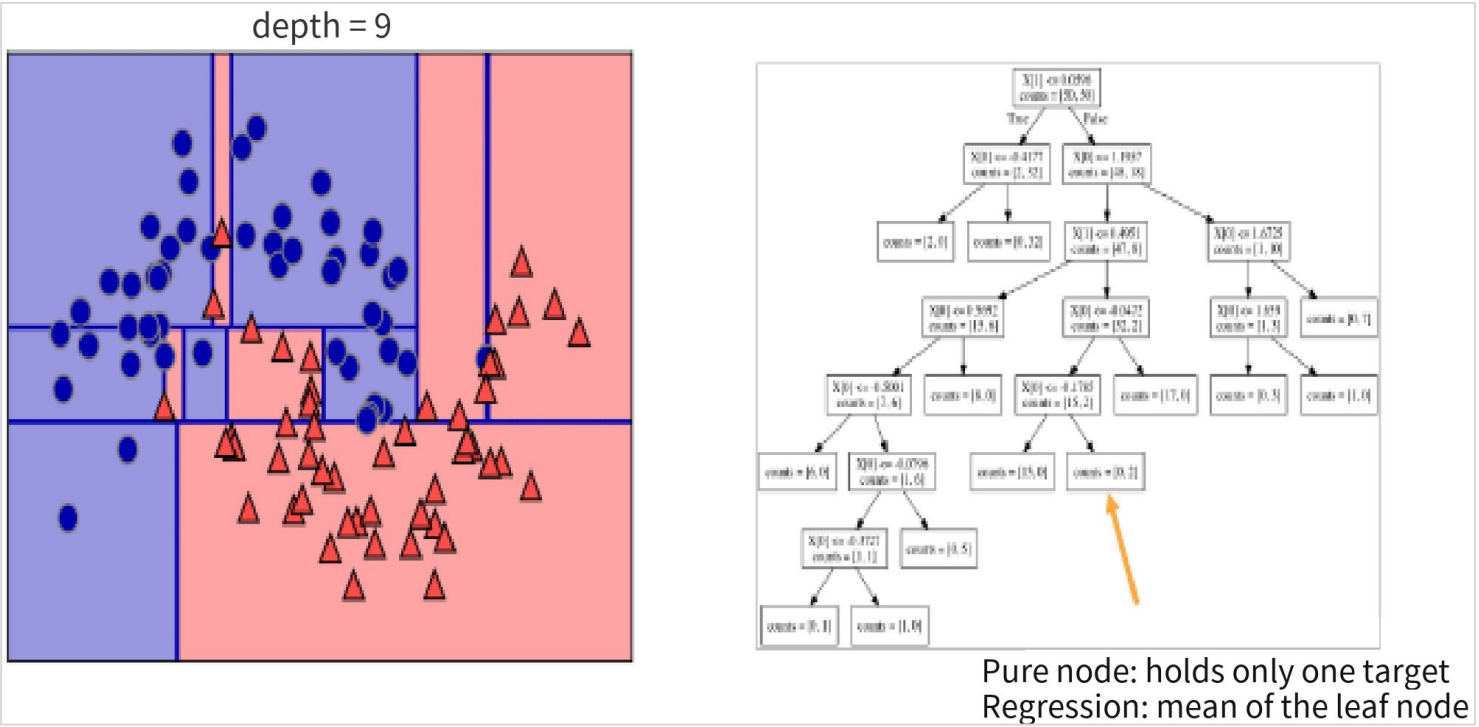
What happens

- Depth 1: the single test X[1] ≤ 0.0596 turns [50, 50] → [2, 32] / [48, 18]
- Depth 2: each of the two regions is split again
- Each time, pick the (feature, threshold) that reduces impurity the most — greedy search



Decision Tree Prediction

Each test uses one feature at a time, so the decision boundary is an axis-parallel staircase



Pure node: holds only one target
Regression: mean of the leaf node

A new sample follows the questions from the root down, and its prediction is the majority class (classification) or mean (regression) of the leaf it reaches.

Uncertainty Measures — Entropy and Gini Impurity

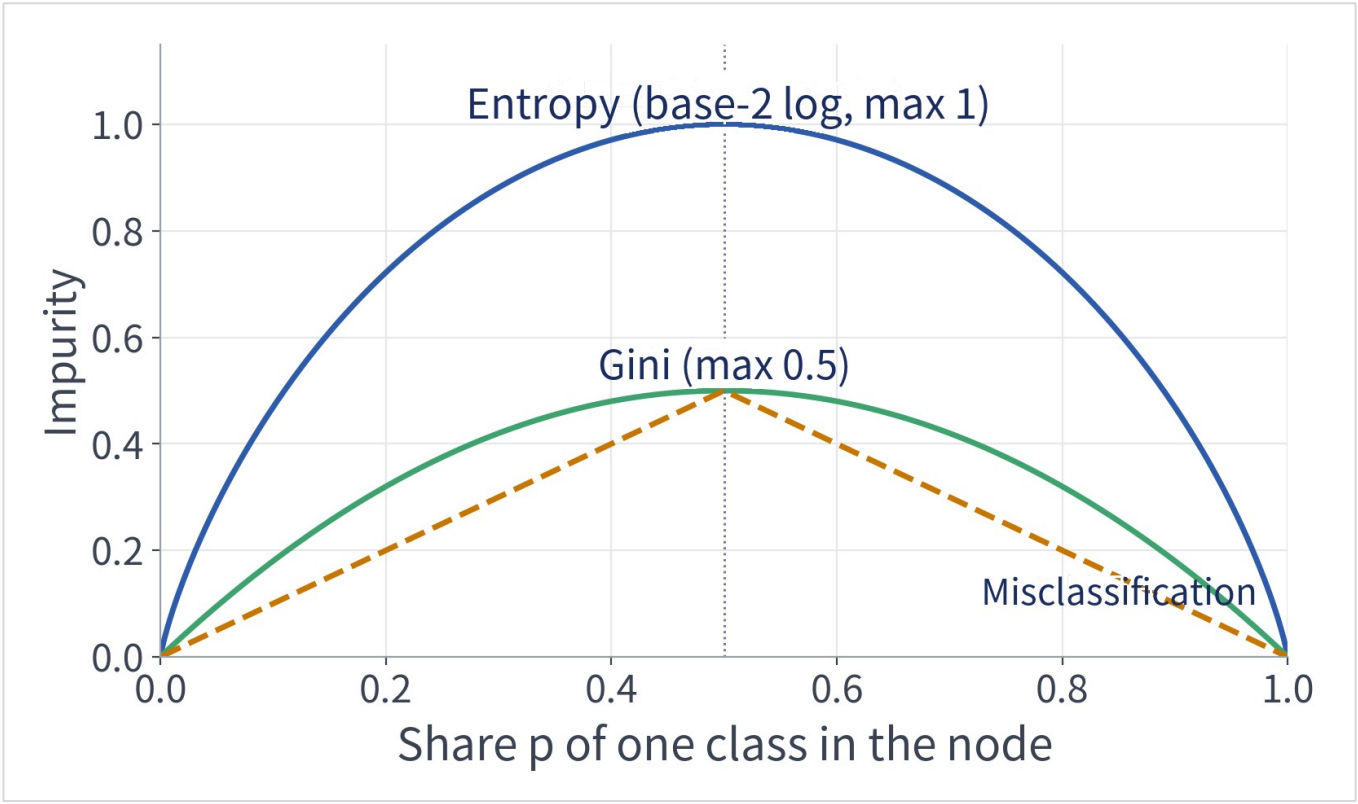
Measure the "degree of mixing" by the class proportions $p_1, \ldots, p_n$ in a node (sum = 1)

$$\text{Entropy}(t) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

$$\text{Gini}(t) = 1 - \sum_{i=1}^{n} p_i^2$$

Points to note

- Both are 0 for a pure node
- The log base (2 or e) does not affect the ranking of splits
- scikit-learn default: criterion="gini"



Information Gain

Information gain = expected reduction in uncertainty from a split

$$\mathrm{IG} = H(\text{parent}) - \sum_k \frac{N_k}{N} H(\text{child}_k)$$

Original lecture example

- Probability of getting a job offer: 20%
- Probability of holding a relevant degree: 50%
- It becomes 30% with the degree and 10% without
- Question: what is the "holds a degree" information worth?

$$H_0 = -\left[\,0.2\log_2 0.2 + 0.8\log_2 0.8\,\right] = 0.7219$$

$$\bar{H} = 0.5\,H(0.1) + 0.5\,H(0.3) = 0.5(0.4690) + 0.5(0.8813) = 0.6751$$

$$\mathrm{IG} = 0.7219 - 0.6751 = 0.0468$$

Recomputed from the original figures ($\log_2$). The degree information reduces uncertainty by about 0.047 bits.

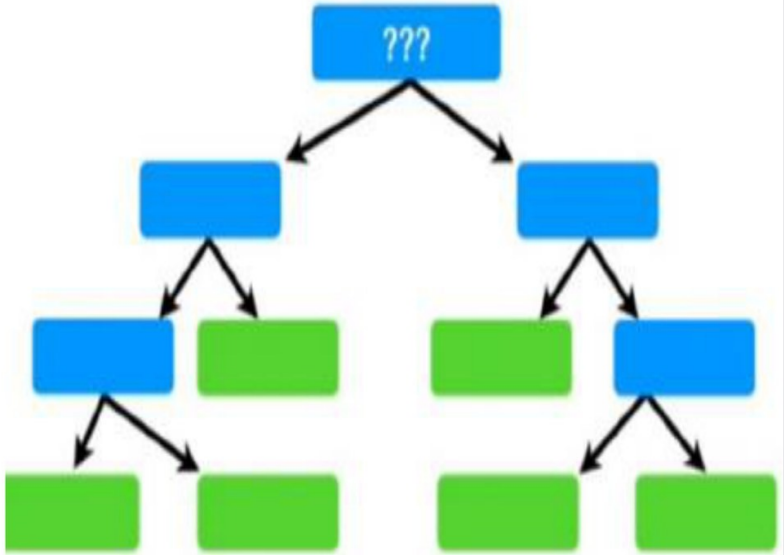
Heart Disease Example — Which Question Goes at the Root?

Three features: Chest Pain · Good Blood Circulation · Blocked Arteries

Task

- Target: Heart Disease Yes/No
- The root is one of CP · BC · BA
- "???" is a missing value — covered in Part 3
- Figures: from StatQuest (Josh Starmer) lecture slides

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



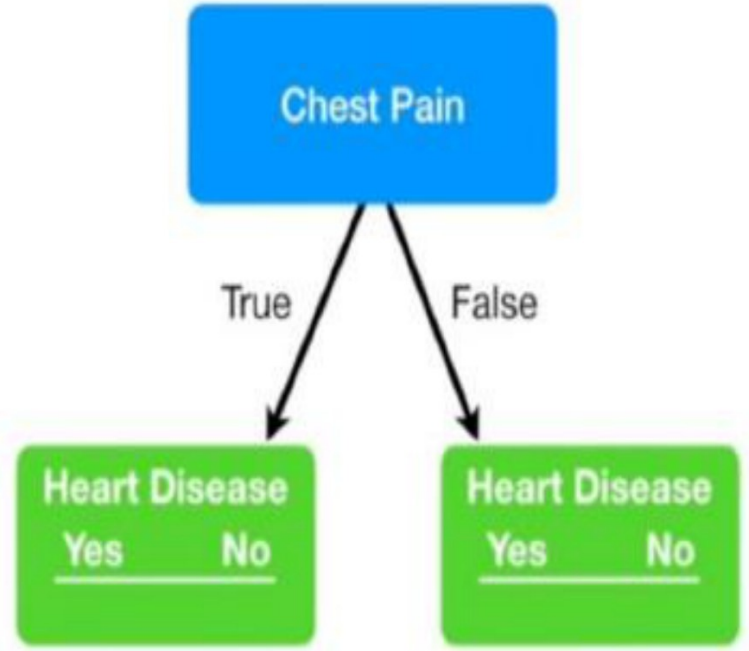
Root Candidate ① Splitting by Chest Pain (CP)

Record each patient's heart disease status in a leaf according to CP True/False

Procedure

- Look only at the CP column and the target column
- CP = Yes → left leaf, No → right leaf
- Count heart disease Yes/No in each leaf

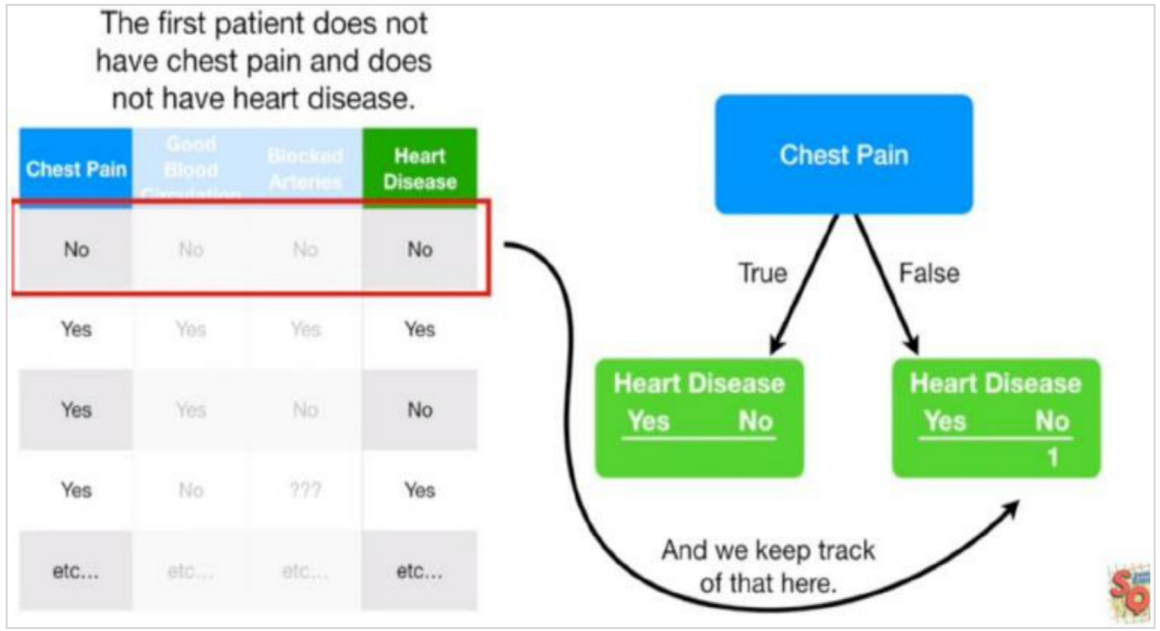
Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



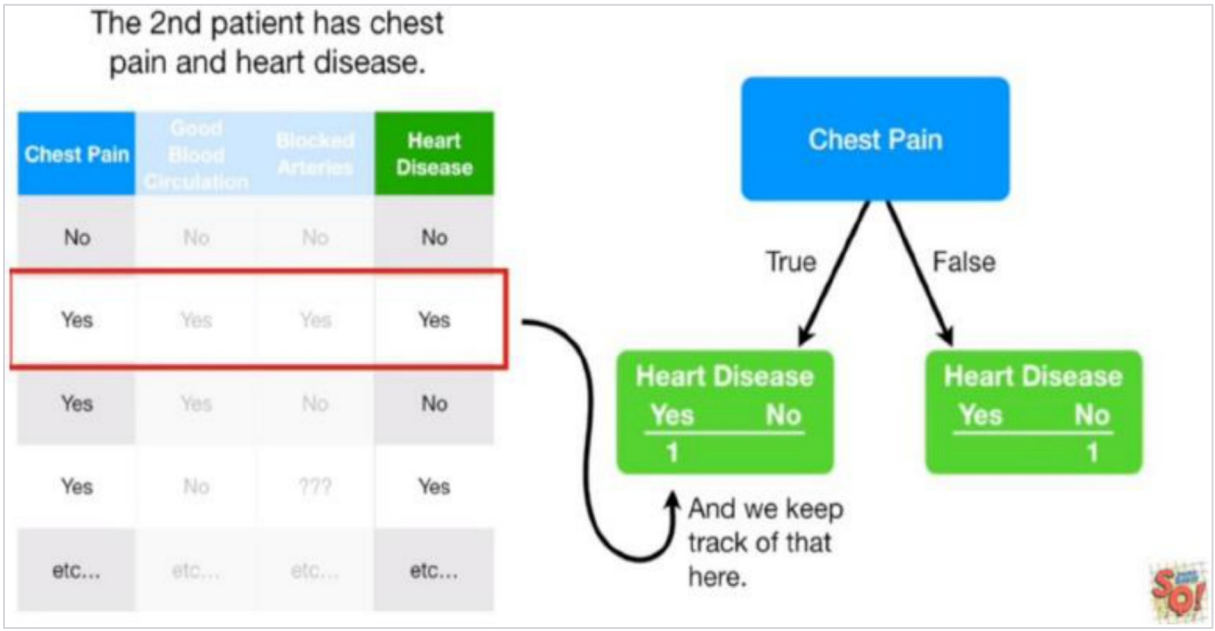
Record Patients in the Leaves One by One (1)

#1: no CP · no heart disease → right leaf No / #2: CP · heart disease → left leaf Yes

Patient 1



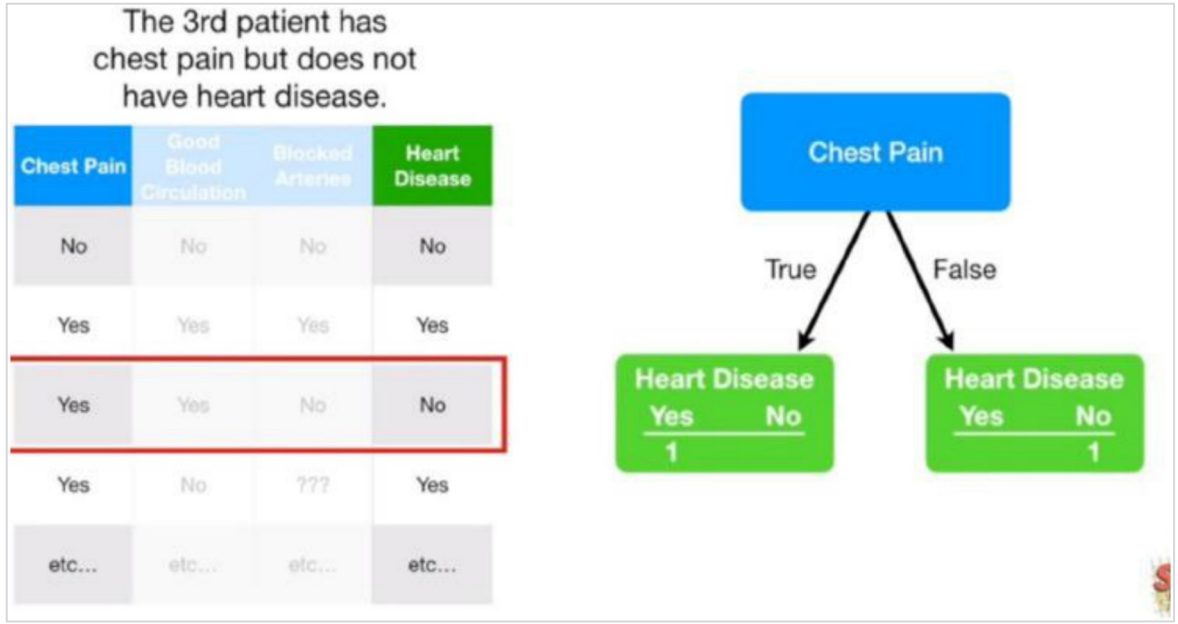
Patient 2



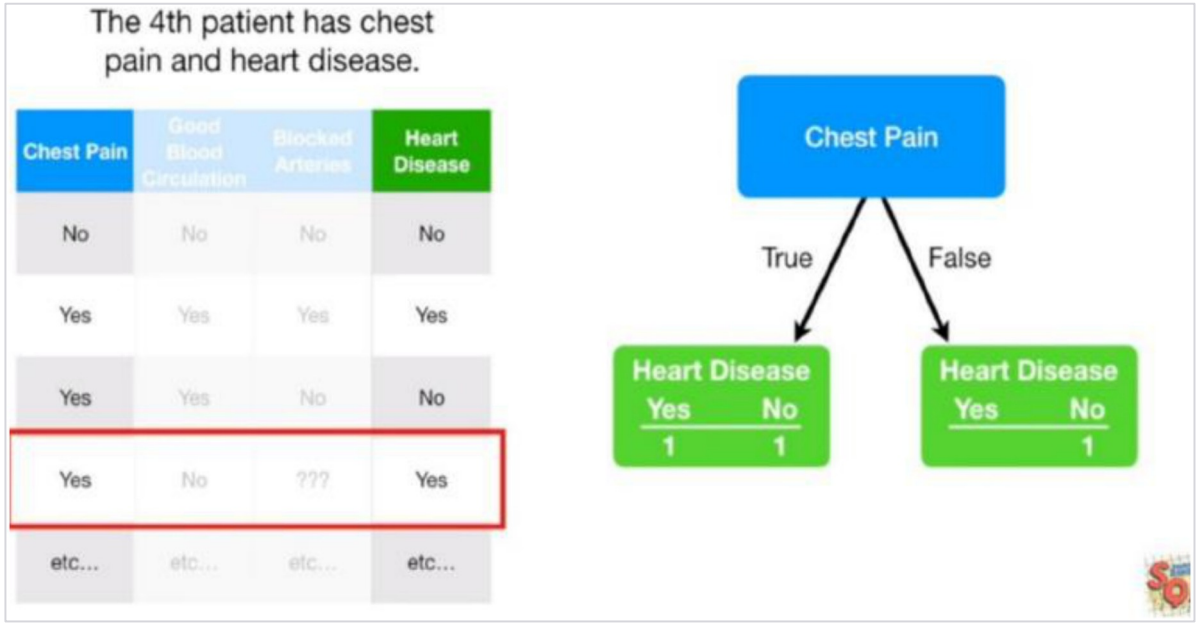
Record Patients in the Leaves One by One (2)

#3: CP · no heart disease → left leaf No / #4: CP · heart disease → left leaf Yes

Patient 3



Patient 4



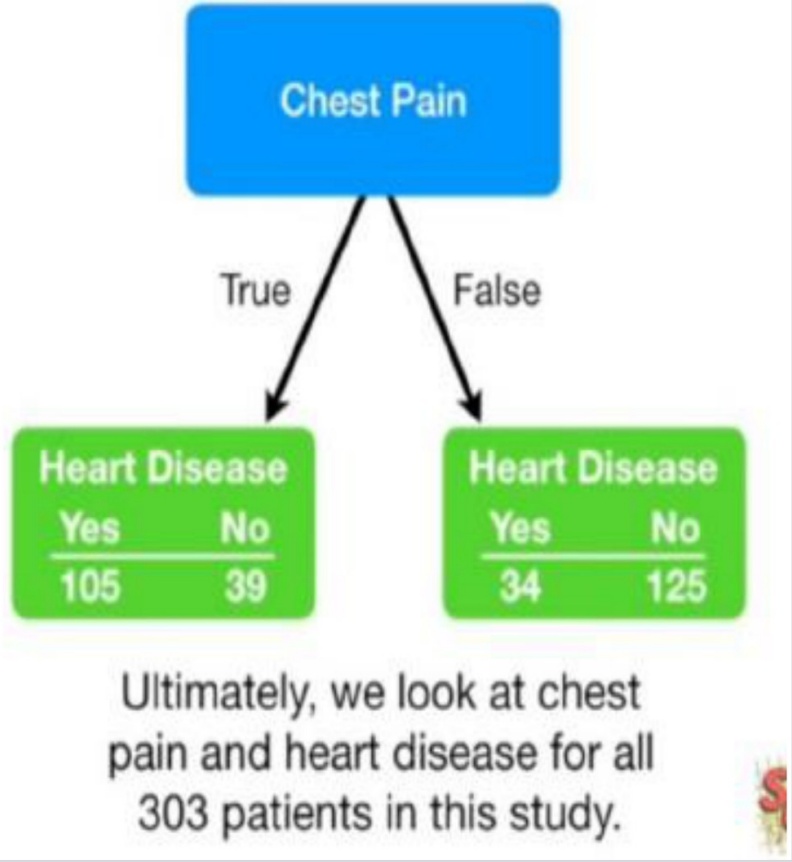
Tallying All 303 Patients

By CP: left leaf 105/39, right leaf 34/125

Observation

- Neither leaf is 100% Yes or 100% No
- → both leaves are "impure"
- The other candidates are tallied the same way and compared

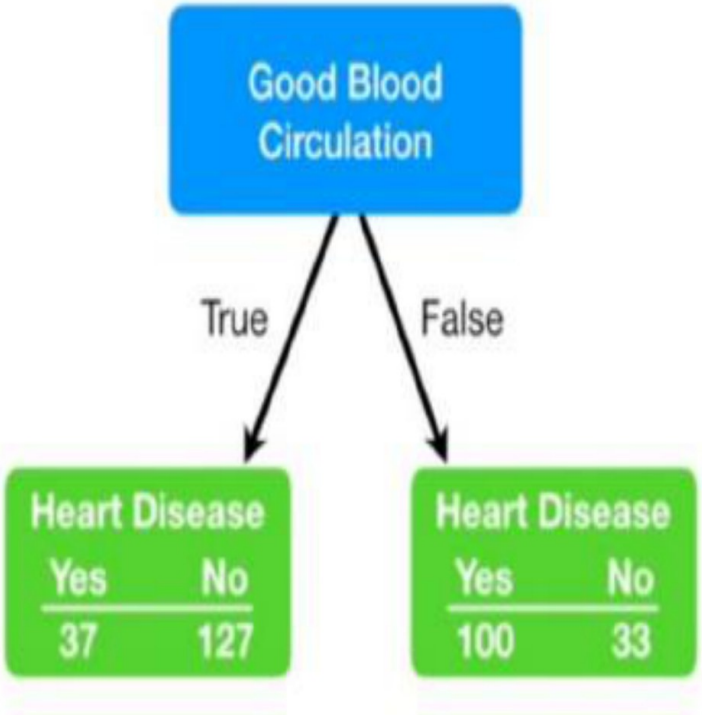
Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



Root Candidate ② Tally by Blood Circulation (BC)

Fill the BC True/False leaves the same way: 37/127 · 100/33

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...

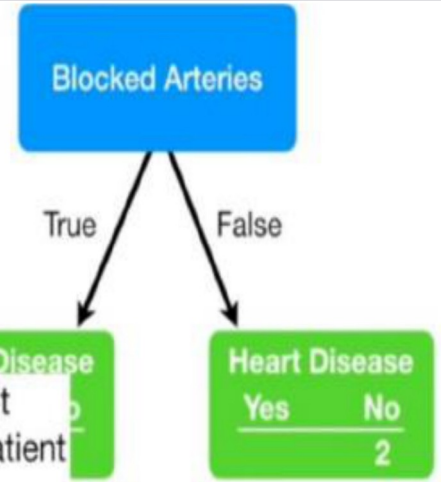


Root Candidate ③ Blocked Arteries (BA) — Skip Missing Values

Patients whose BA value is "???" are skipped for now

Skip patients whose BA is ???

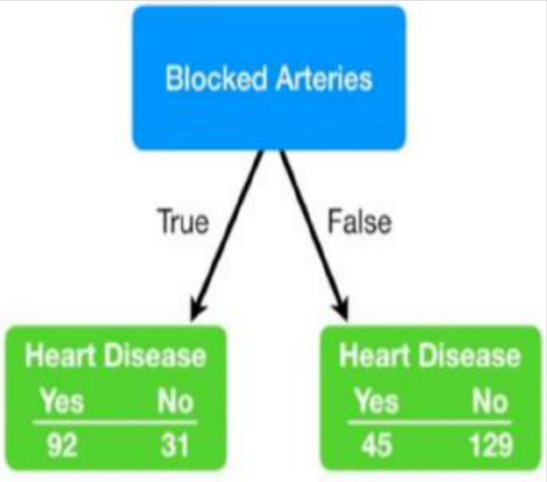
Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



Since we don't know if this patient had blocked arteries or not, we'll skip it.

Full tally by BA: 92/31 · 45/129

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...

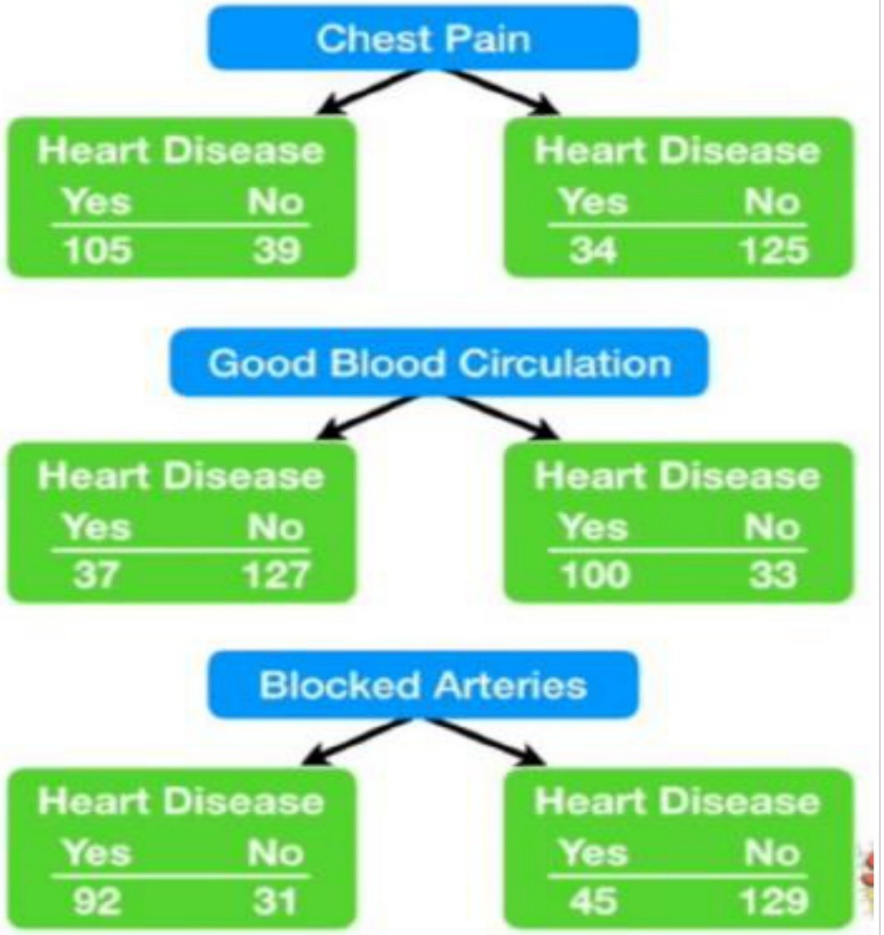


All three candidates have mixed leaves → the measure of which is "less mixed" = impurity

Impurity — Comparing the Leaves of the Three Candidates

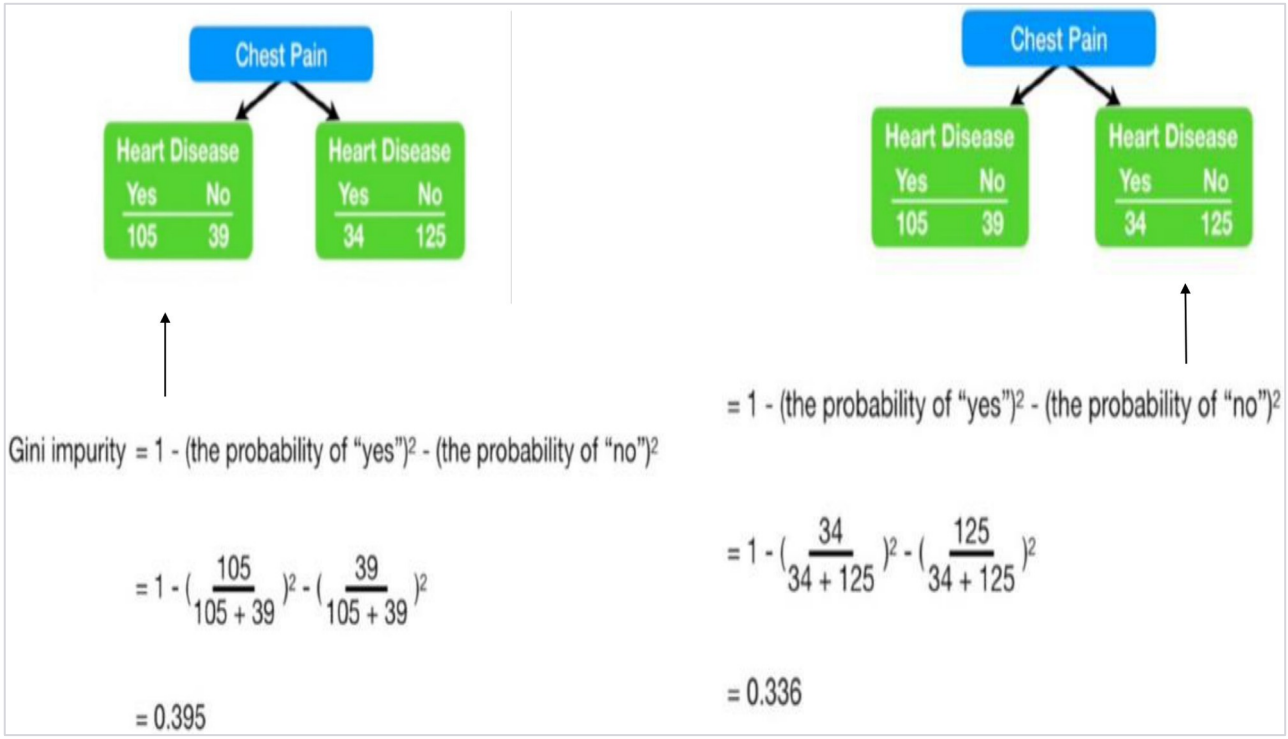
No leaf is 100% Yes or No, so all are impure. One measure is Gini

No leaf node is 100% YES or NO, so all of them are impure. We will compare this impurity, and one measure for it is "Gini".



Computing Gini Impurity — Per Leaf

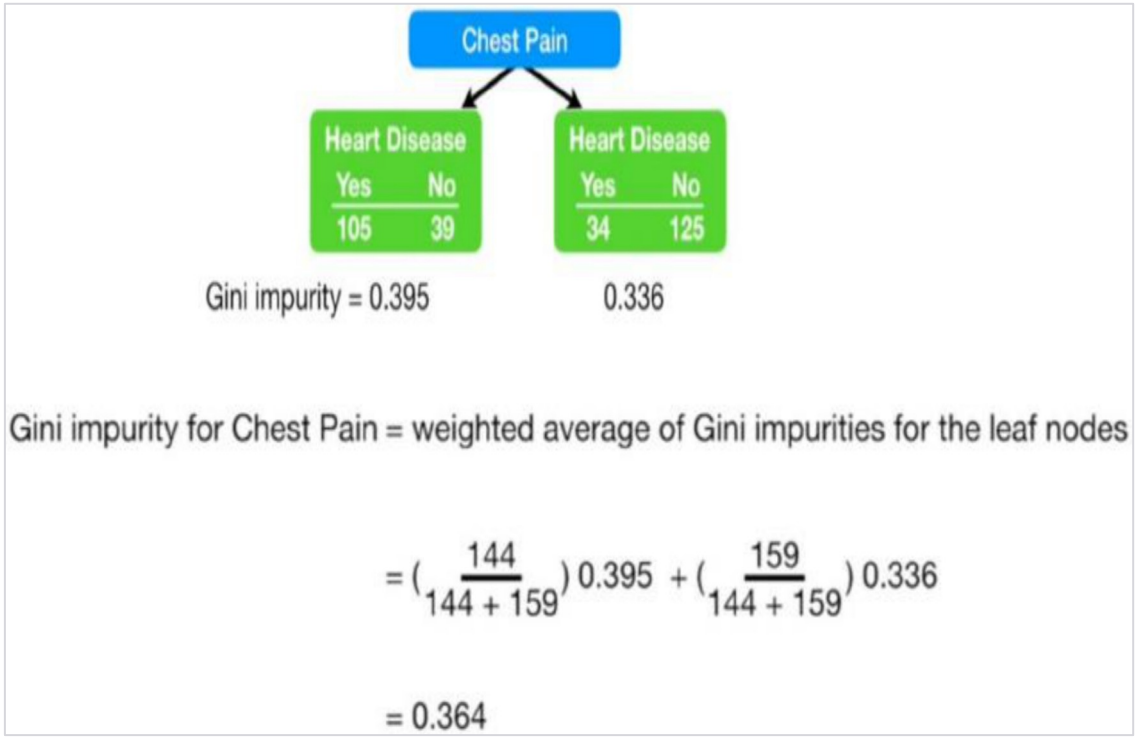
Gini = 1 − (P(Yes))² − (P(No))² · the two Chest Pain leaves



$$G_L = 1 - \left(\frac{105}{144}\right)^2 - \left(\frac{39}{144}\right)^2 = 0.395 \qquad G_R = 1 - \left(\frac{34}{159}\right)^2 - \left(\frac{125}{159}\right)^2 = 0.336$$

Gini of the Whole Split = Weighted Average of Leaf Gini

Weighted by the number of patients in each leaf — Gini for Chest Pain (CP) = 0.364



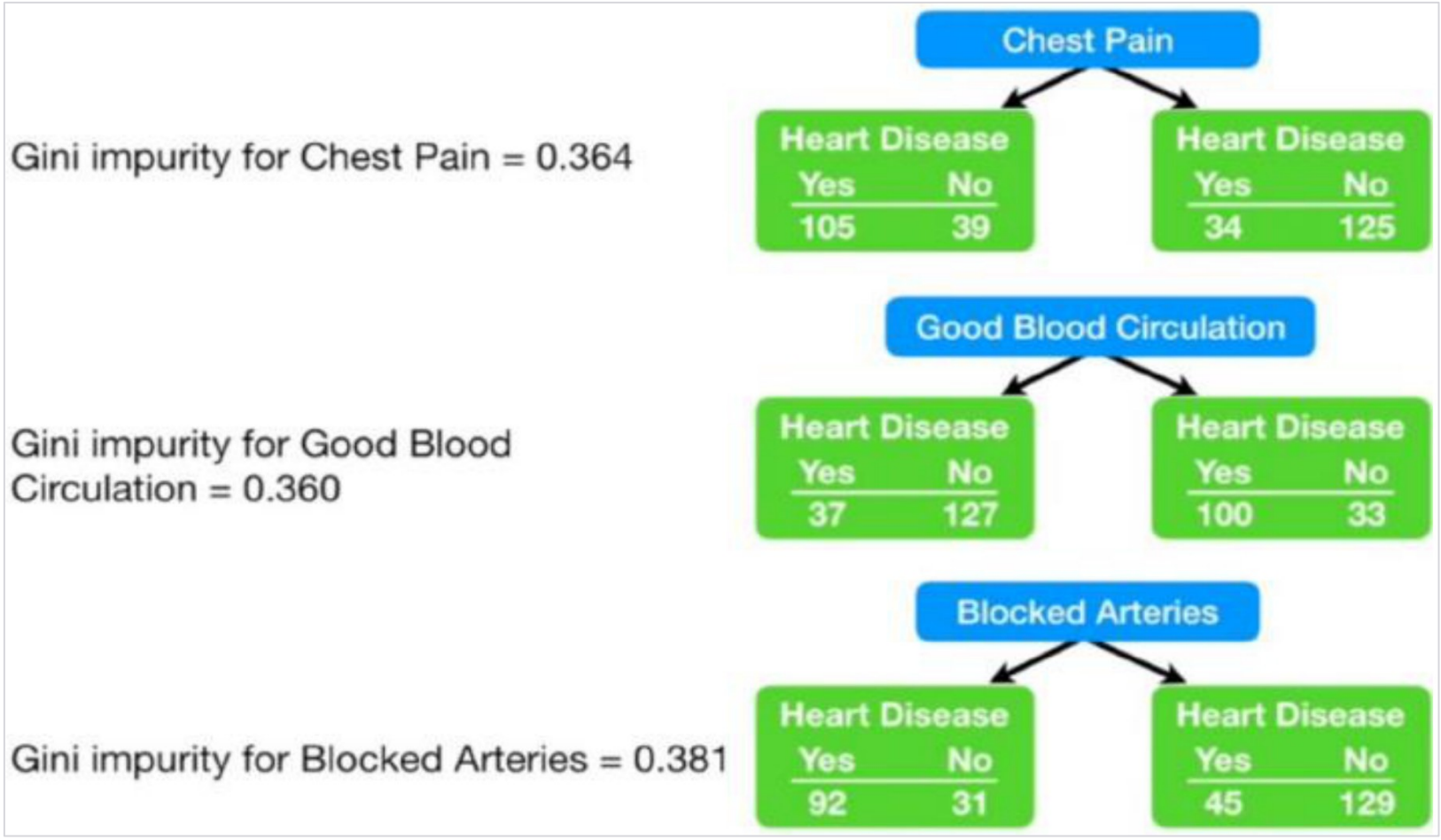
$$G_{\text{CP}} = \frac{144}{303}(0.395) + \frac{159}{303}(0.336) = 0.364$$

Root Decision — Blood Circulation (BC), Lowest Gini

CP 0.364 · BC 0.360 · BA 0.381

Decision

- Put BC, with the lowest impurity, at the root node
- Compare the leaf-size-weighted average Gini (formula on the previous slide)
- When the differences are small, the root changes if the sample changes slightly → high variance of trees

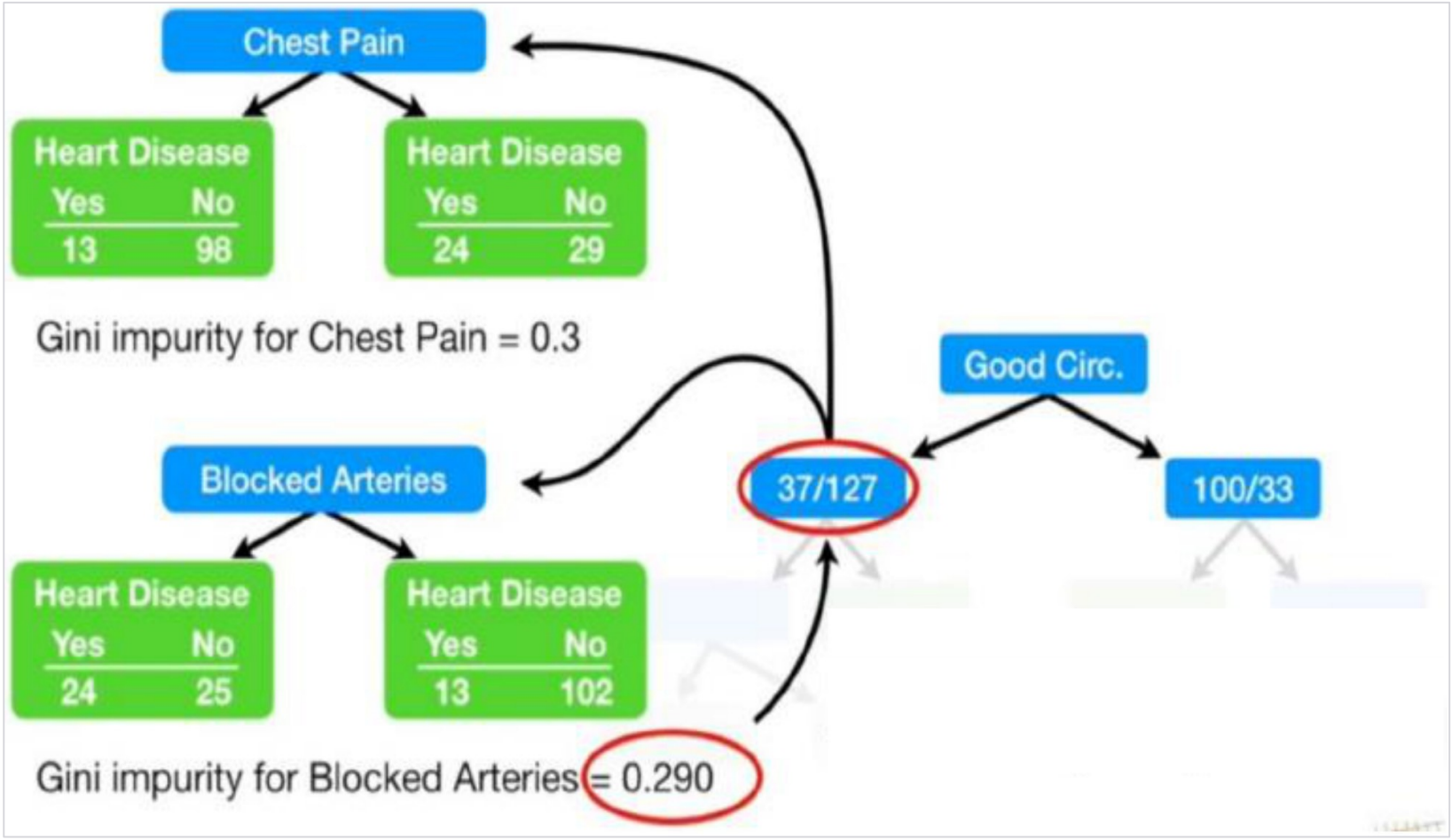


Second-Level Split — BC = True Side (37/127)

Compare the remaining candidates CP · BA the same way

Comparison

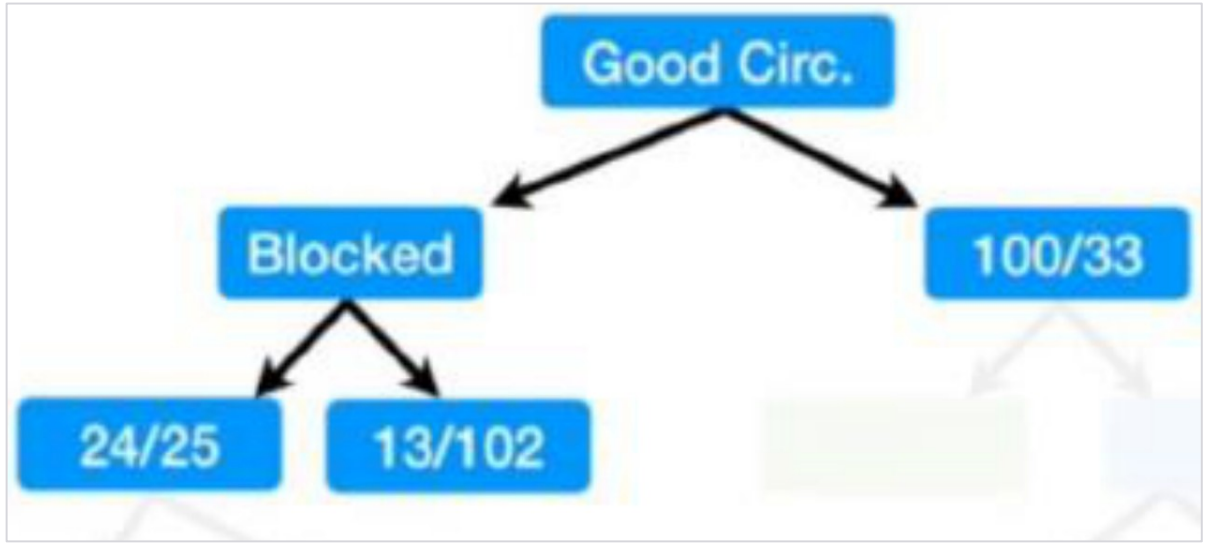
- Split by CP: Gini 0.3
- Split by BA: Gini 0.290 → chosen
- Same rule at lower nodes: lowest impurity



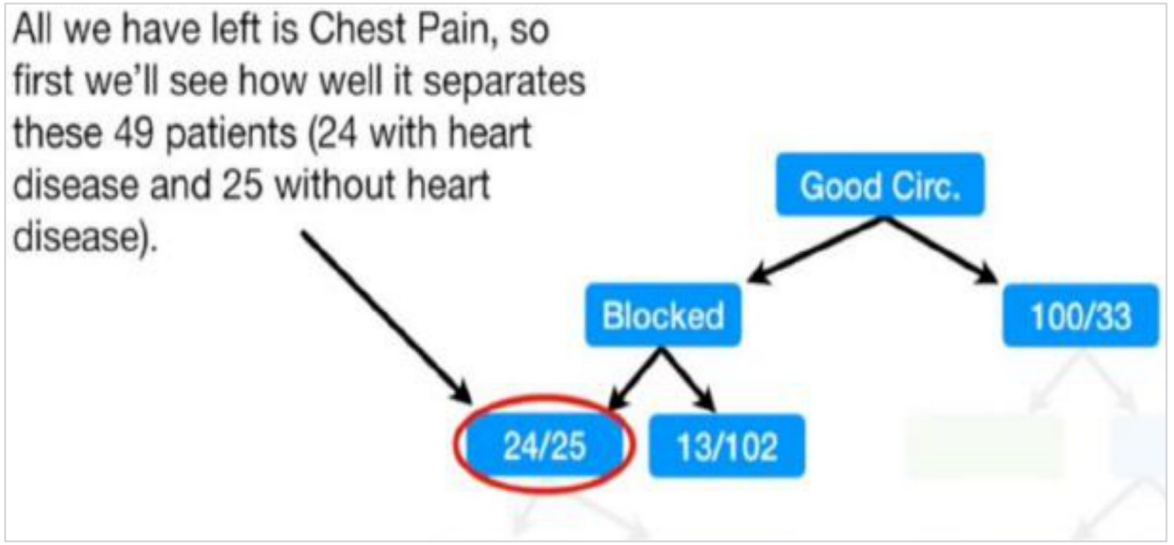
Level 2 Done → Preparing the Level-3 Split

Only Chest Pain (CP) is left for the left 24/25 node

Tree after level 2



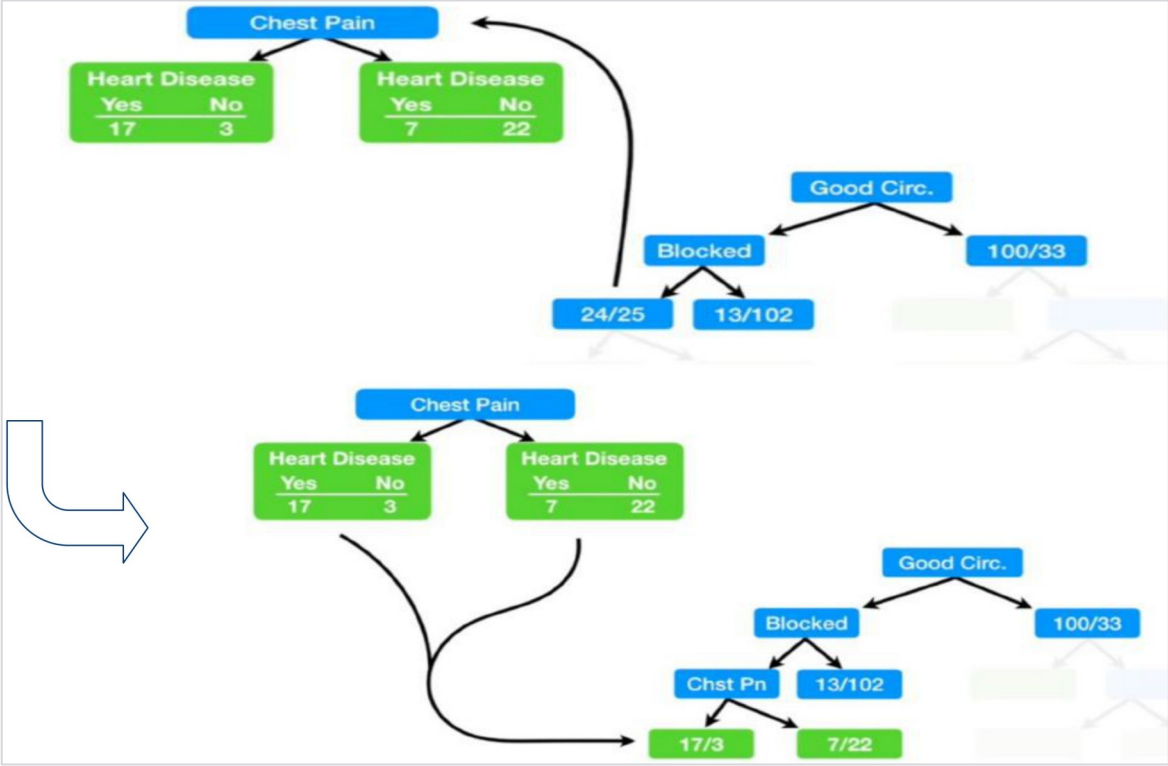
24/25 node: 49 patients (24 heart disease · 25 healthy)



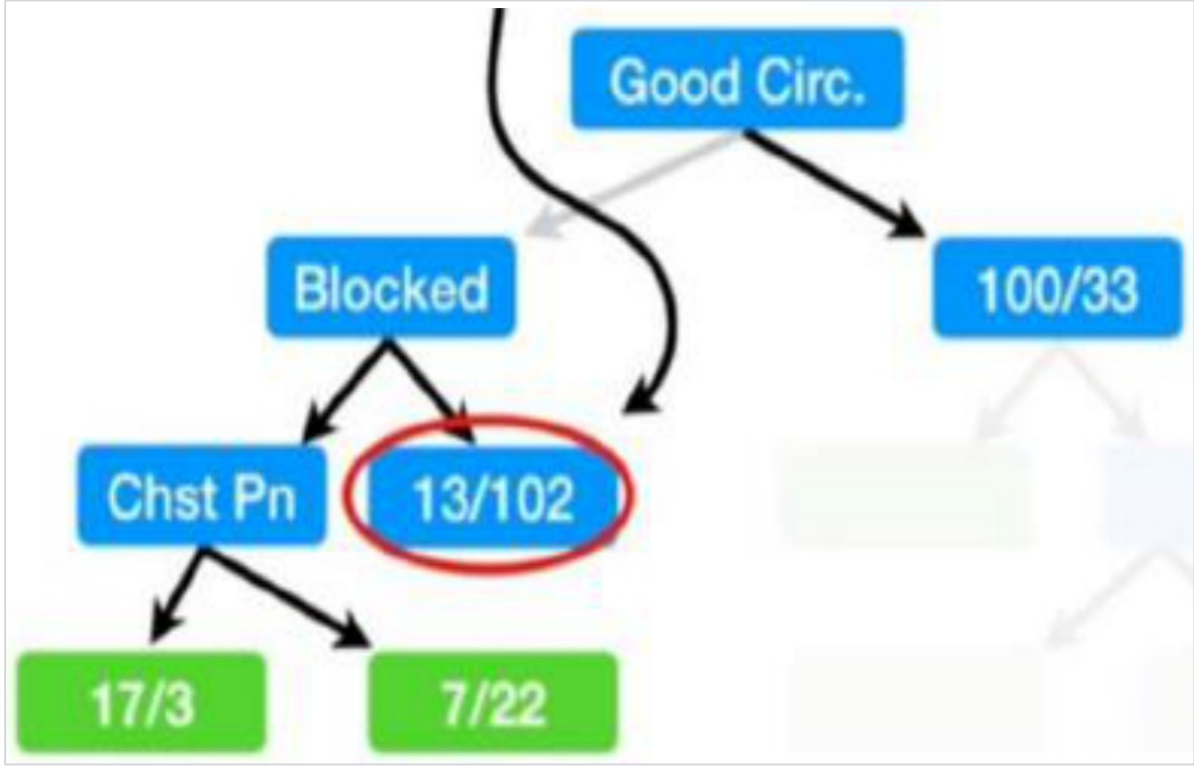
Level-3 Split — Keep Splitting with the Remaining Features

The 24/25 node splits by CP into 17/3 · 7/22. Next is the 13/102 node

Split the 24/25 node by CP



13/102 node: 13 of 115 patients have heart disease



Split or Stop — Gini Before vs After the Split

13/102 node: Gini 0.2 before the split vs 0.29 after splitting by CP

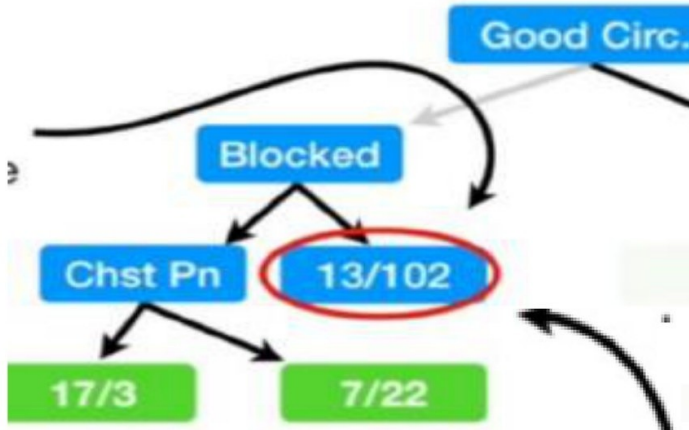
Stopping rule

- Splitting actually increases impurity (0.2 → 0.29)
- → do not split; keep it as a leaf node
- "Can be split further" is not the same as "worth splitting"

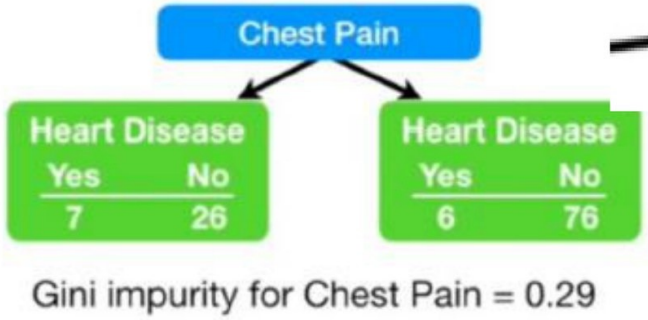
$$= 1 - (\text{the probability of "yes"})^2 - (\text{the probability of "no"})^2$$

$$= 1 - \left(\frac{13}{13+102}\right)^2 - \left(\frac{102}{13+102}\right)^2$$

$$= 0.2$$



- Gini after splitting on chest pain = 0.29



Gini impurity for Chest Pain = 0.29

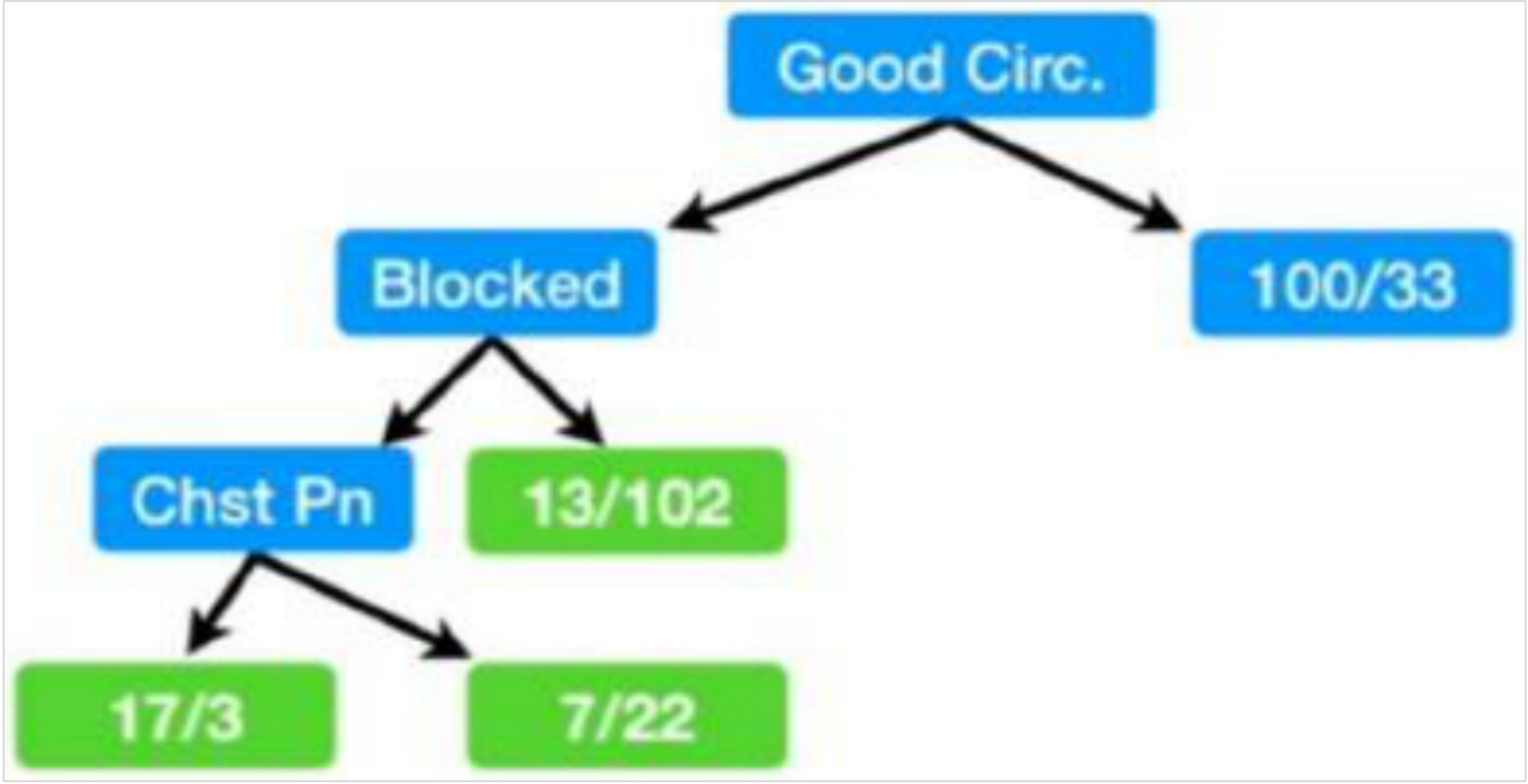
The split actually increases impurity, so we do not split.

Finish the Right Branch

Apply the same three rules to every node

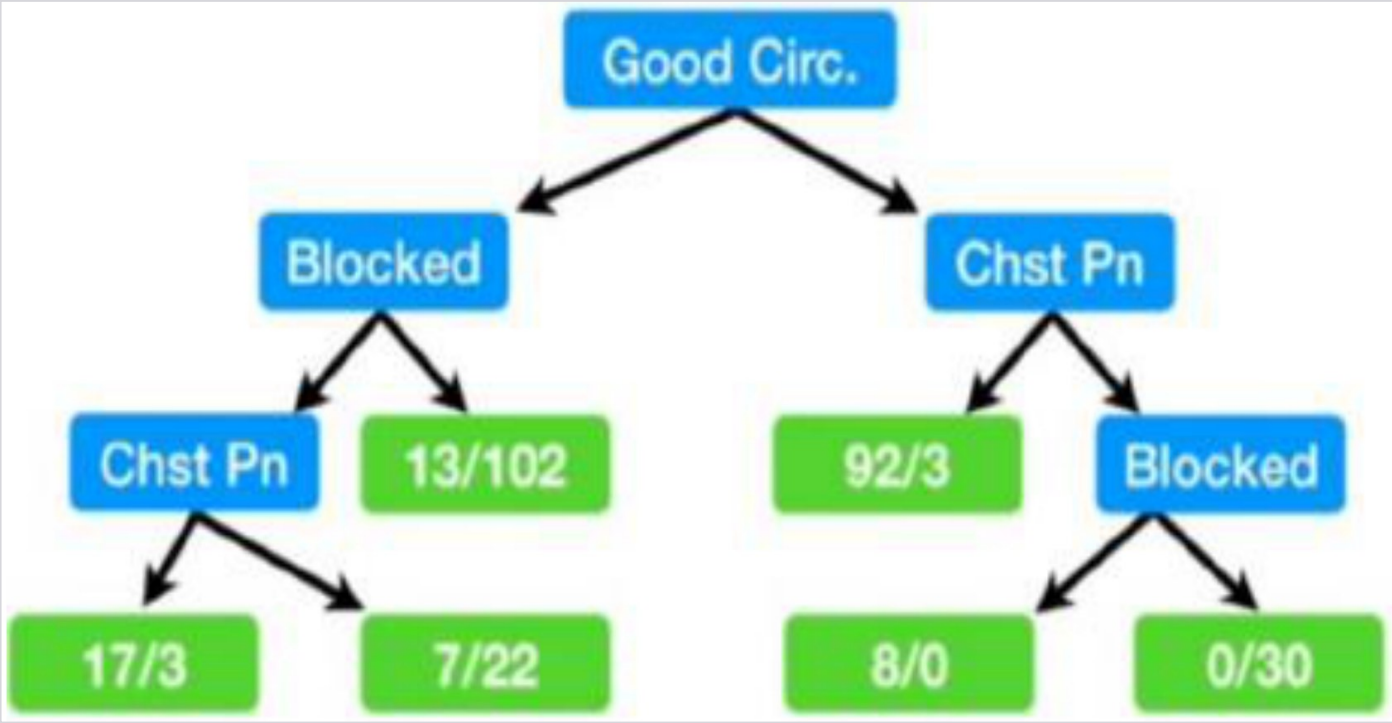
Three rules

- 1. Compute Gini for every possible split
- 2. If the node's own Gini is lowest → leaf
- 3. If a split reduces impurity → choose the lowest split



The Finished Tree

Root BC → left: BA / right: CP. Leaf numbers are heart disease Yes/No patient counts



Fully pure leaves such as 8/0 and 0/30 also appear. A new patient follows the questions down and is assigned the leaf's majority class.

Numeric Features — Splitting by Weight

① Sort → ② averages of adjacent values as candidate thresholds → ③ Gini for each candidate

Raw data

Weight	Heart Disease
220	Yes
180	Yes
225	Yes
190	No
155	No

① Sort by weight

	Weight	Heart Disease
Lowest	155	No
	180	Yes
	190	No
	220	Yes
Highest	225	Yes

② Adjacent averages

Weight	Heart Disease
155	No
167.5	
180	Yes
185	
190	No
205	
220	Yes
222.5	
225	Yes

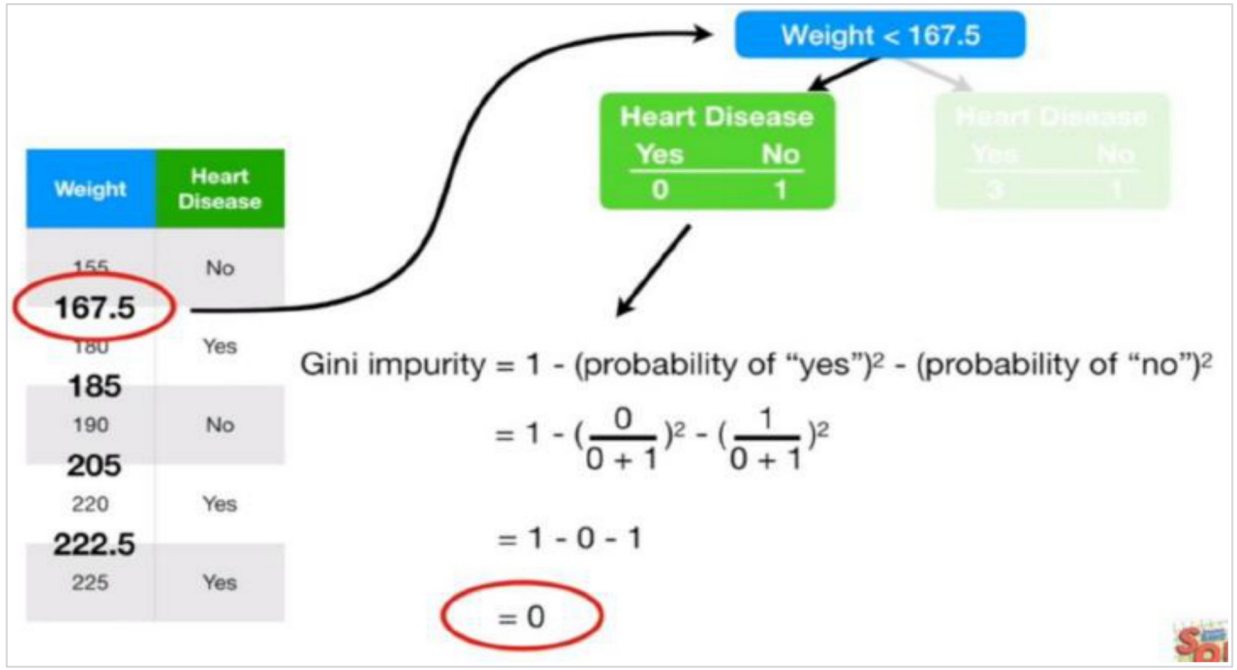
③ Gini per candidate

Weight	Heart Disease	
155	No	
167.5		Gini impurity = ?
180	Yes	
185		Gini impurity = ?
190	No	
205		Gini impurity = ?
220	Yes	
222.5		Gini impurity = ?
225	Yes	

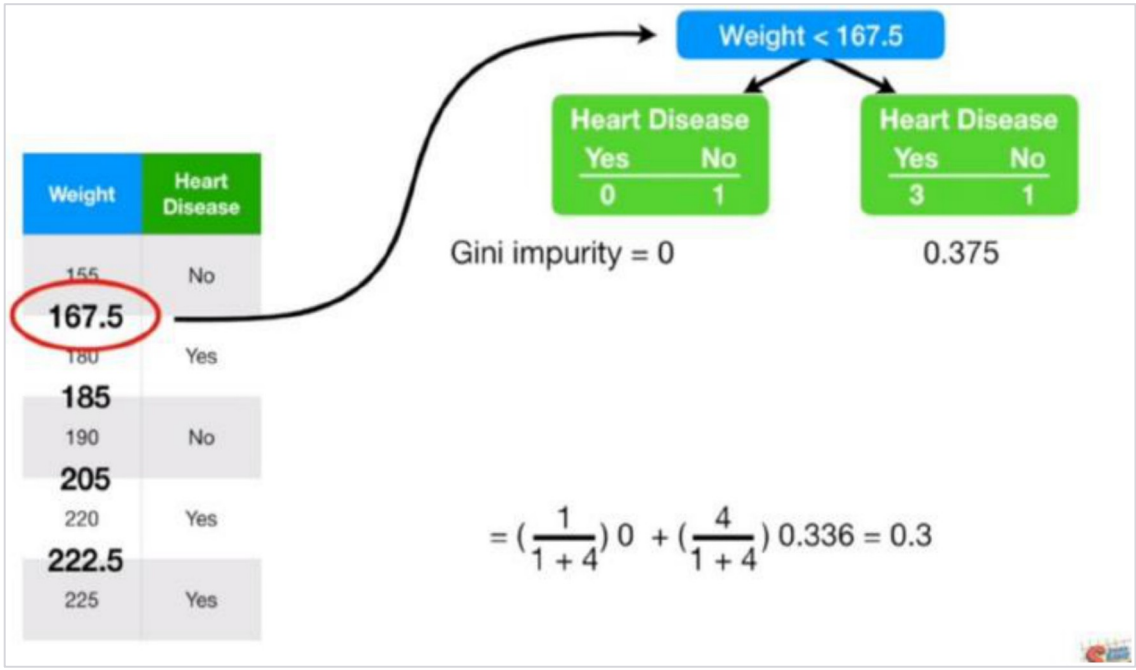
Gini of the Candidate Threshold "Weight < 167.5"

Left leaf (0/1) has Gini 0, right leaf (3/1) 0.375 → weighted average 0.3

Left leaf: pure (Gini 0)



Weighted average: (1/5)·0 + (4/5)·0.375 = 0.3

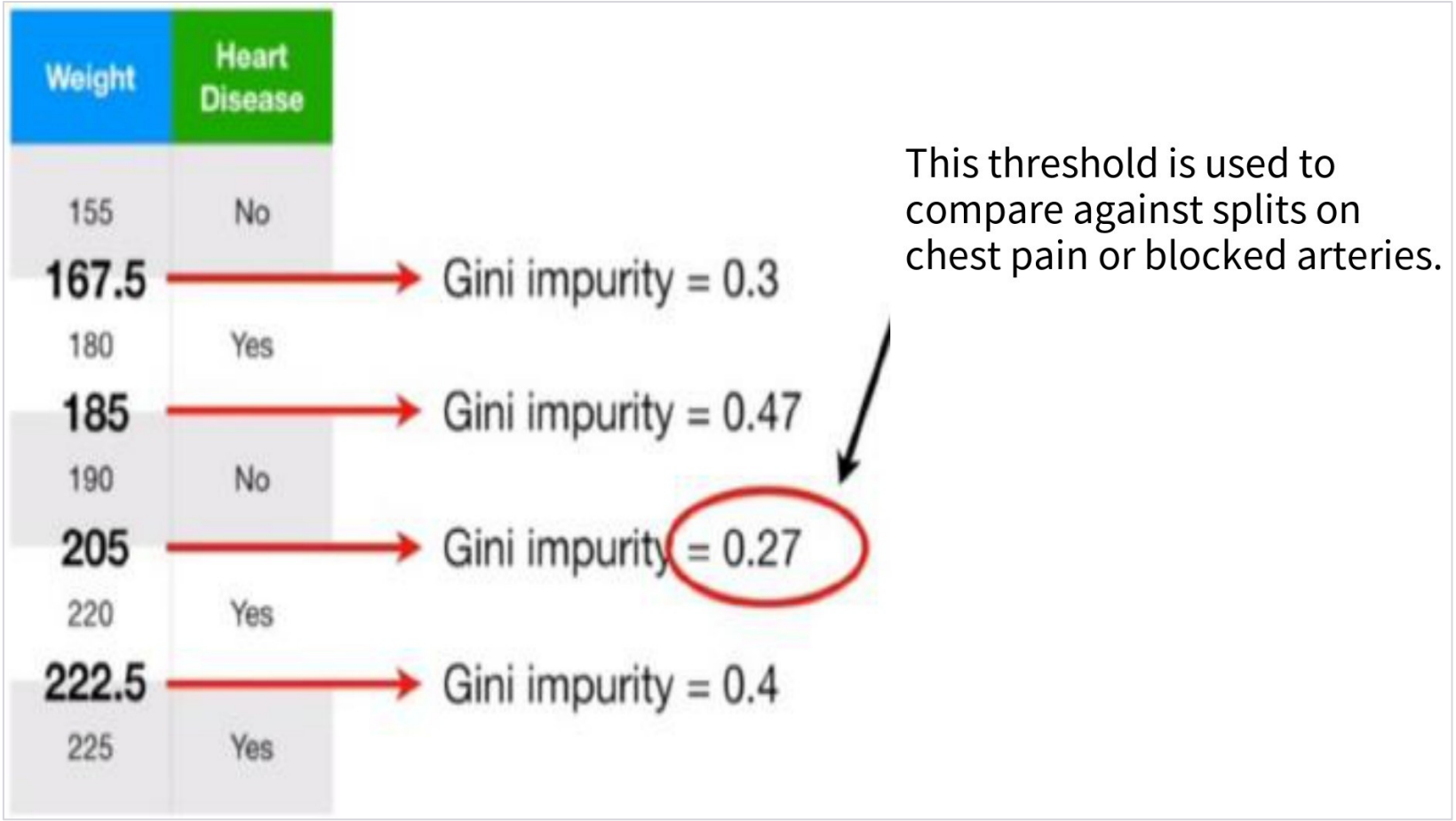


The Lowest-Impurity Split Is "Weight < 205"

Four candidates: 0.3 · 0.47 · 0.27 · 0.4

Next step

- Representative threshold for weight = 205 (Gini 0.27)
- Compare this with the Gini of other features such as CP · BA to choose the node's question
- Because of the sorting cost, large data use histogram binning (HistGB · LightGBM)



Ordinal Features — Cut by Rank

e.g. "Rank my jokes (1–4)" and whether the person likes StatQuest

Method

- Same as numeric, but compute Gini at each possible rank boundary
- Only Rank ≤ 1, ≤ 2, ≤ 3 are checked
- Rank ≤ 4 includes every sample, so it is not a split

Rank my jokes...	Likes StatQuest
1	Yes
1	No
3	Yes
1	Yes
etc...	etc...



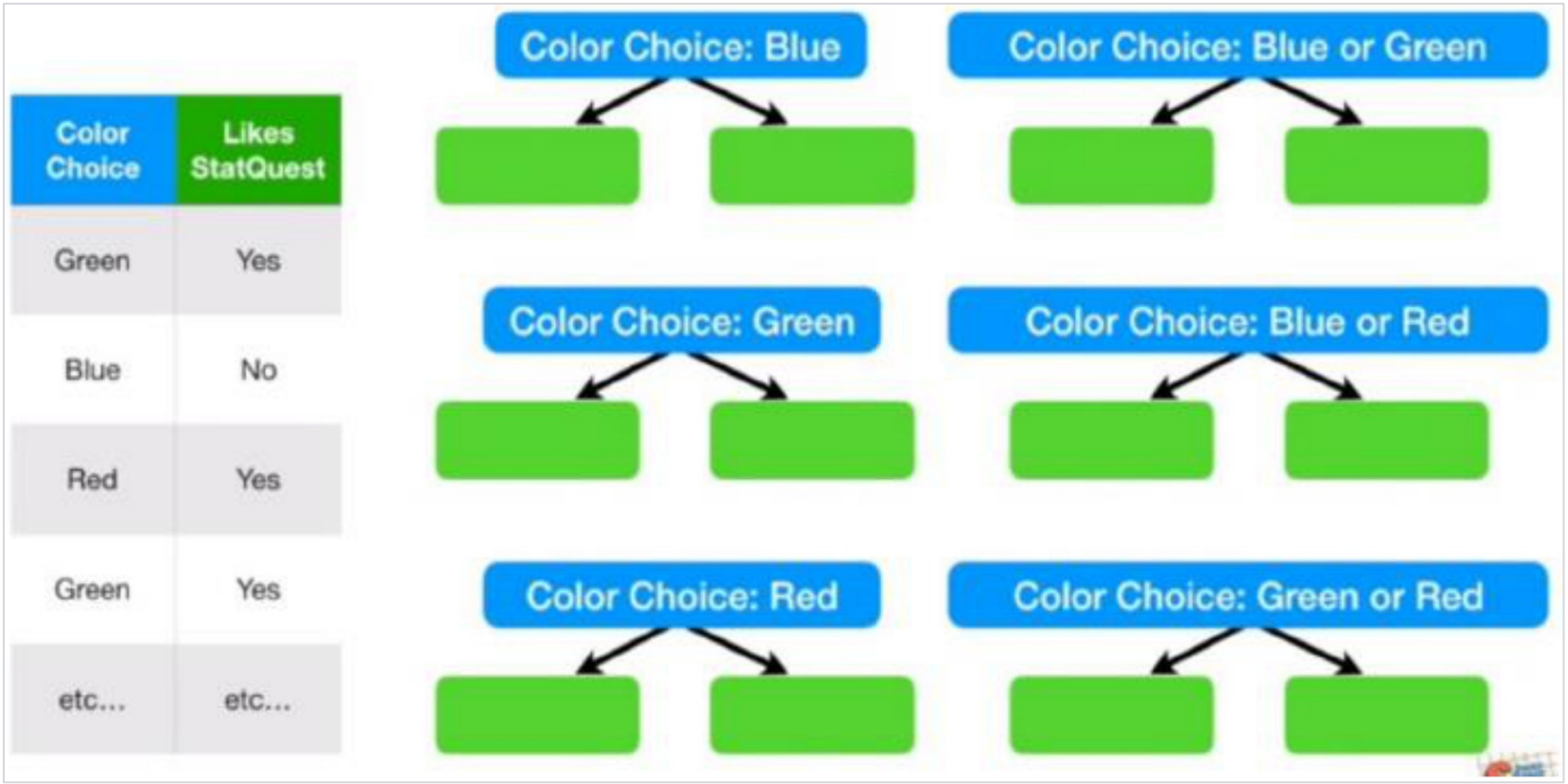
Categorical Features — Compute for Every Combination

e.g. favorite color is one of blue · green · red

Method

- Compute Gini for each category and every possible combination (Blue or Green, etc.)
- The number of combinations grows exponentially with the number of categories
- In practice: one-hot encoding, or the dedicated categorical handling in CatBoost · HistGB

Color Choice	Likes StatQuest
Green	Yes
Blue	No
Red	Yes
Green	Yes
etc...	etc...



Overfitting and Complexity Control

Grown until every leaf is pure, a tree fits the training data 100% but generalizes poorly

Hyperparameter	Meaning	Effect
max_depth	Maximum depth of the tree	The most direct complexity limit
max_leaf_nodes	Maximum number of leaf nodes	Limits size by number of leaves
min_samples_leaf	Minimum samples to be a leaf	Prevents tiny leaves (memorizing)
min_samples_split	Minimum samples to split a node	Suppresses splits of small nodes
ccp_alpha	Strength of cost-complexity pruning	Grow fully, then cut back (post-pruning)

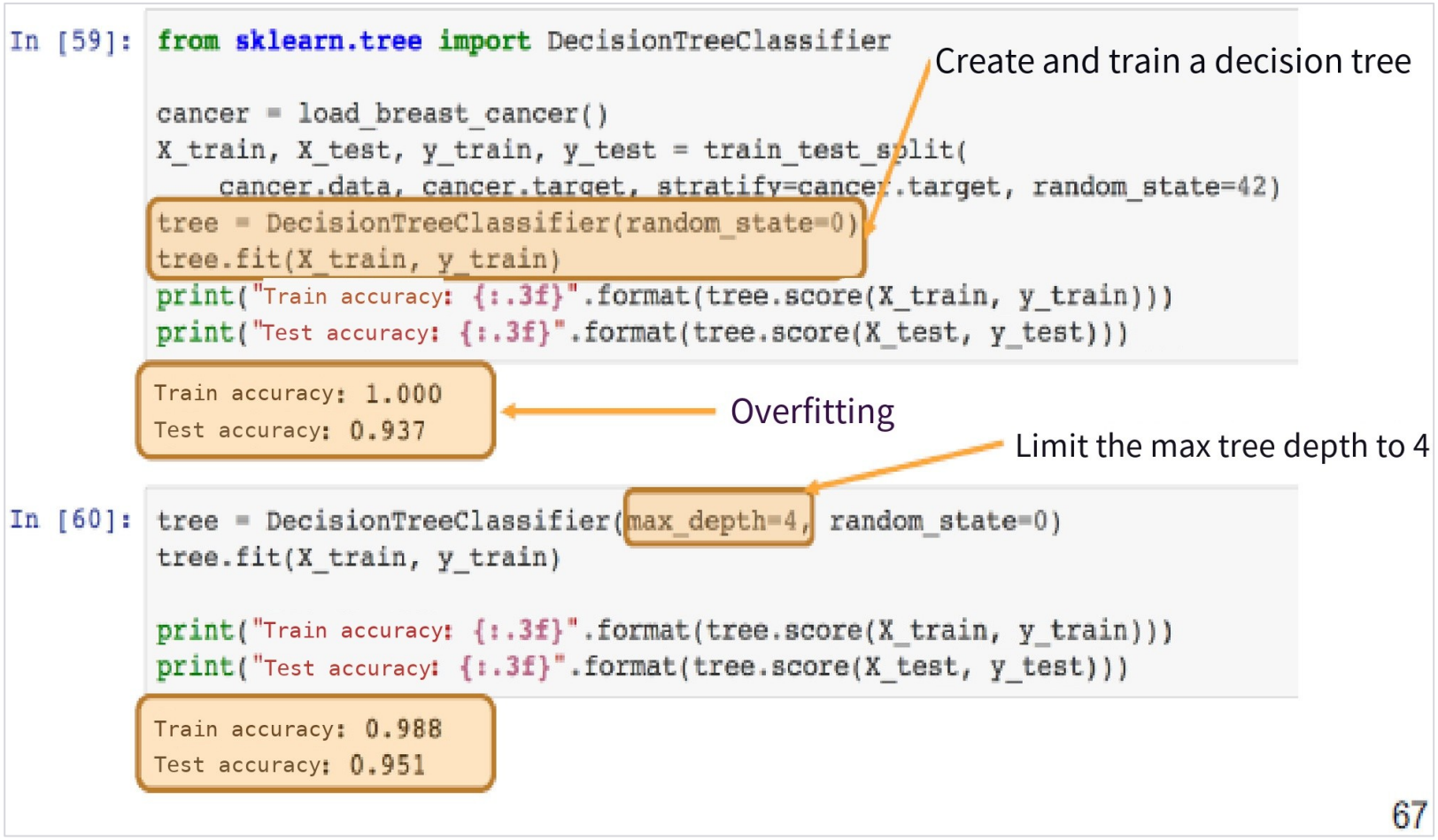
Pre-pruning: stop growth early with the four above · Post-pruning: cut back a fully grown tree with ccp_alpha

Implementation — Effect of Limiting Depth

Breast cancer data: unlimited tree vs max_depth = 4

Interpreting the results

- Unlimited: train 1.000 · test 0.937 → overfitting
- Depth 4: train 0.988 · test 0.951
- We gave up a little training accuracy and gained generalization
- Same values when rerun in scikit-learn 1.8

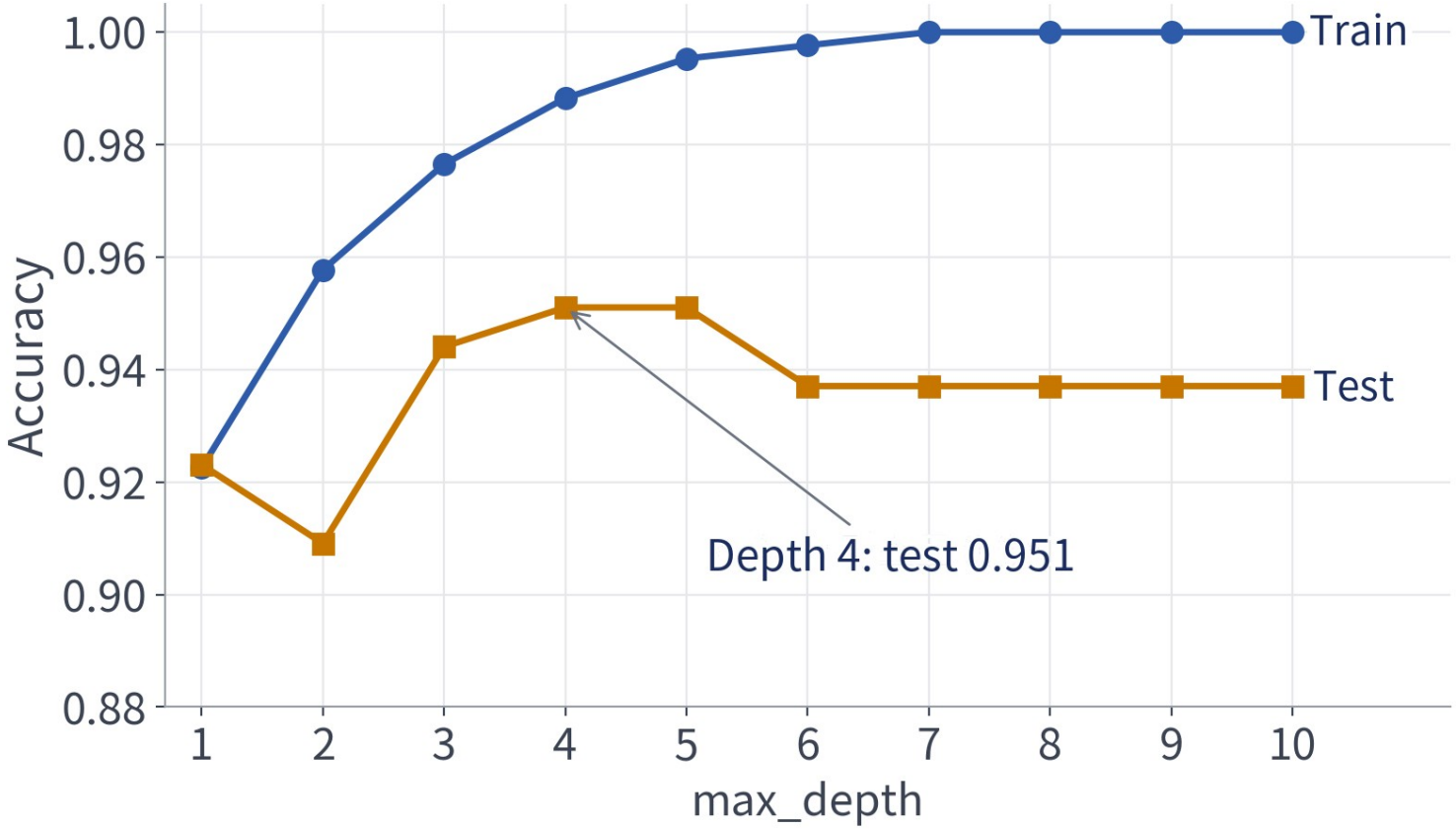


Supplement — Train/Test Accuracy by Depth

Varying max_depth from 1 to 10 on the same data and split (reproduced in scikit-learn 1.8)

How to read it

- Training accuracy rises to 1.0 as depth grows
- Test accuracy peaks at depth 4–5 and then falls
- In practice, choose depth by cross-validation



Supplement — Cost-Complexity Pruning (CART)

A single α balances the tree's error against its number of leaves

$$R_\alpha(T) = R(T) + \alpha\,|\tilde{T}|$$

Notation

- R(T): training error of tree T (sum of impurity)
- |T̃|: number of leaves
- α: penalty per leaf — larger means a smaller tree

In practice

- Get candidate α values with cost_complexity_pruning_path()
- Choose α by cross-validation
- Exactly the method of the CART book by Breiman et al. (1984)

Feature Importance

Between 0 (not used) and 1 (predicts perfectly alone); the total sums to 1

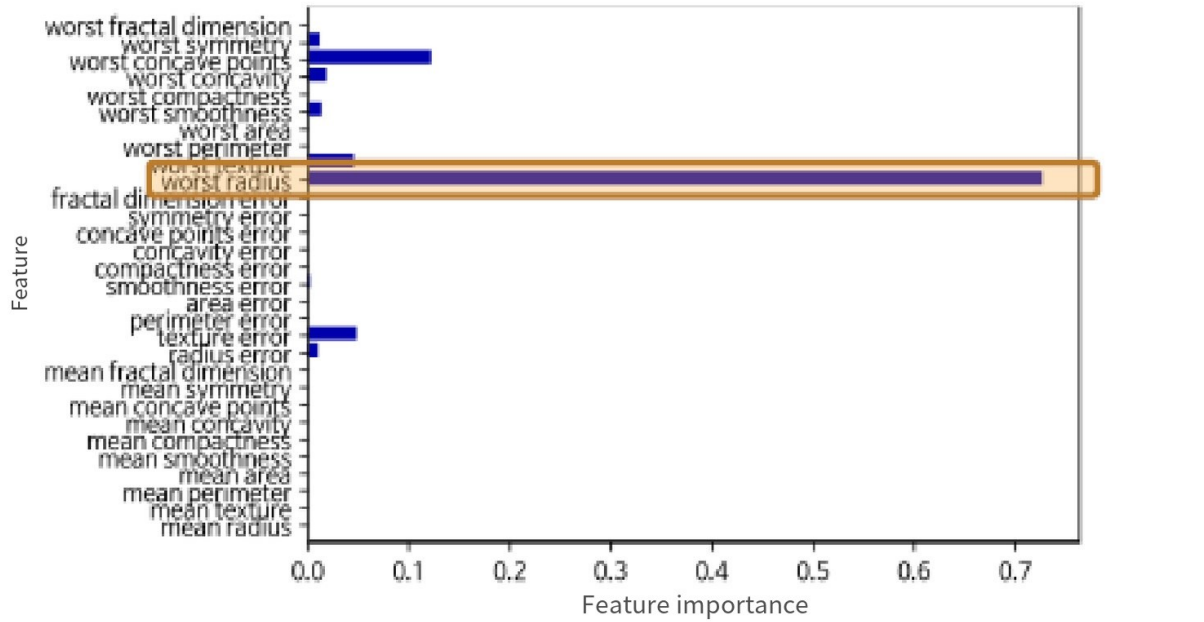
Caveats

- "worst radius" alone is 0.727 — splits near the root get large credit
- Impurity-decrease importance (MDI) tends to overrate continuous features with many values
- Correlated features share importance among themselves
- Cross-check: permutation importance · SHAP

Between 0 (not used for prediction) and 1 (predicts perfectly); the total sums to 1

```
In [63]: print("Feature importances:\n{}".format(tree.feature_importances_))

Feature importances:
[ 0.     0.     0.     0.     0.     0.     0.     0.     0.     0.01
  0.048  0.     0.     0.002  0.     0.     0.     0.     0.     0.727
  0.046  0.     0.     0.014  0.     0.018  0.122  0.012  0.     ]
```

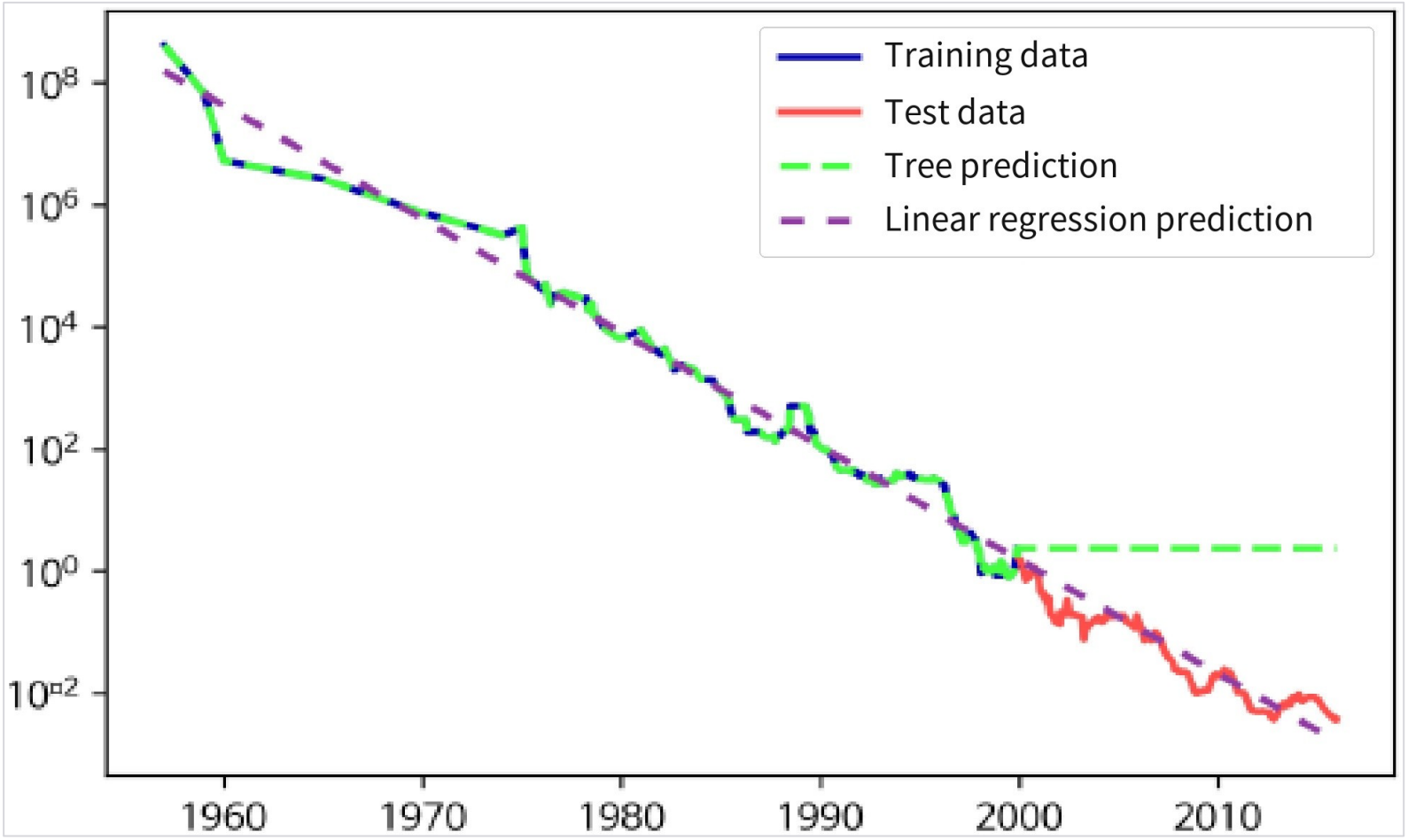


Trees Cannot Extrapolate

Outside the training range (after 2000), the tree prediction stays at the last leaf value

Implications for financial data

- Predicting trending targets such as prices or index levels directly with a tree is risky
- Convert to returns · rates of change · ratios to make them stationary
- A linear model continues the trend, but a tree goes flat (figure: log scale)



Pros and Cons of Decision Trees

Pros

- Small trees are easy to visualize and explain
- Features are handled one at a time, so scale does not matter → no normalization or standardization needed
- Mixing binary and continuous features is no problem

Cons

- Overfitting — the biggest weakness
- The structure changes greatly when the data change slightly (high variance)
- Axis-parallel boundaries · no extrapolation

Variance is reduced by averaging trees (Part 2 RF), bias by sequential learning (Parts 4–5 boosting)

02

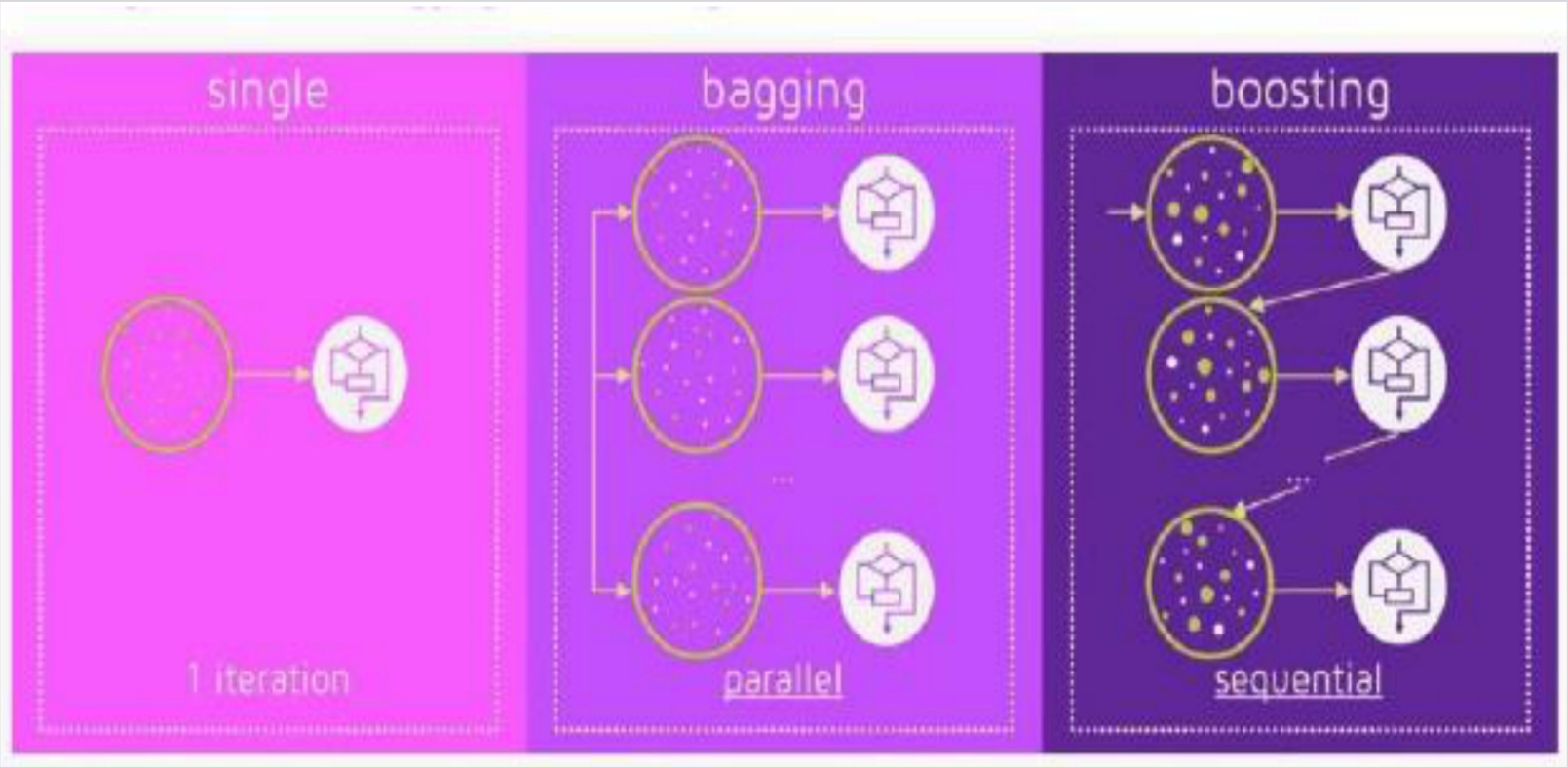
Part 2 · Random Forest

Random Forest — Averaging Many Different Trees

Ensembles · bagging · bootstrap · random feature selection · Out-of-Bag validation

Single Model vs Bagging vs Boosting

Bagging trains independently in parallel and then combines; boosting learns sequentially from the previous model's results



In the original lectures this figure opened the AdaBoost part. It was moved here to show both branches first.

Ensembles · Bagging · Boosting

Ensemble

Combine the outputs of several ML algorithms into one estimate. Weak learners can be bundled into a strong learner; how much it helps depends on the correlation between learners.

Bagging

Bootstrap AGGregatING

Draw several new datasets with replacement, train on each, and get the final estimate by voting (classification) or averaging (regression).

Boosting

Predict in sequence, correcting earlier errors. AdaBoost raises the weights of misclassified observations; GBM fits a new predictor to the previous predictor's residuals.

What Is a Random Forest?

Build many trees, with two layers of randomness so the trees differ from each other

Two layers of randomness

- Rows: draw a bootstrap sample from the data, so each tree sees different data
- Columns: at each node, only a random subset of features are candidates
- (Extra-Trees also randomizes the thresholds)

Final decision

- Classification: majority vote (or averaged probabilities)
- Regression: average of tree predictions
- Weighted voting is possible too — weights from holdout accuracy, etc.

$$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \qquad \hat{y} = \mathrm{mode}\left\{T_1(x), \ldots, T_B(x)\right\}$$

How to Build ① Bootstrap Data

Sample with replacement (duplicates allowed) to the same size as the original data

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Bootstrapped Dataset

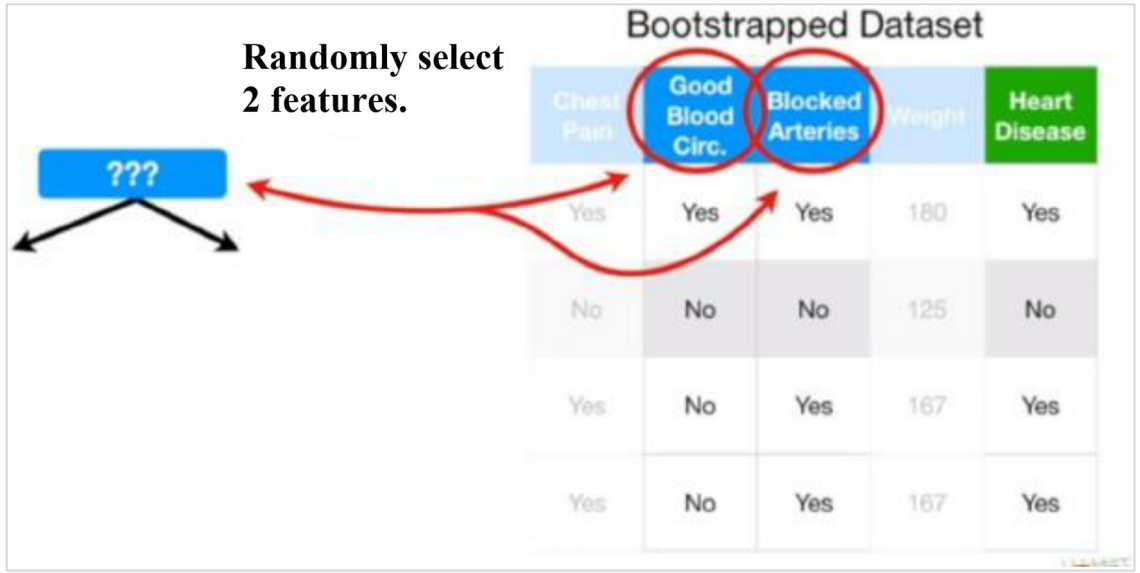
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

Original 4 patients → bootstrap: patient 3 was drawn twice and patient 4 was left out.

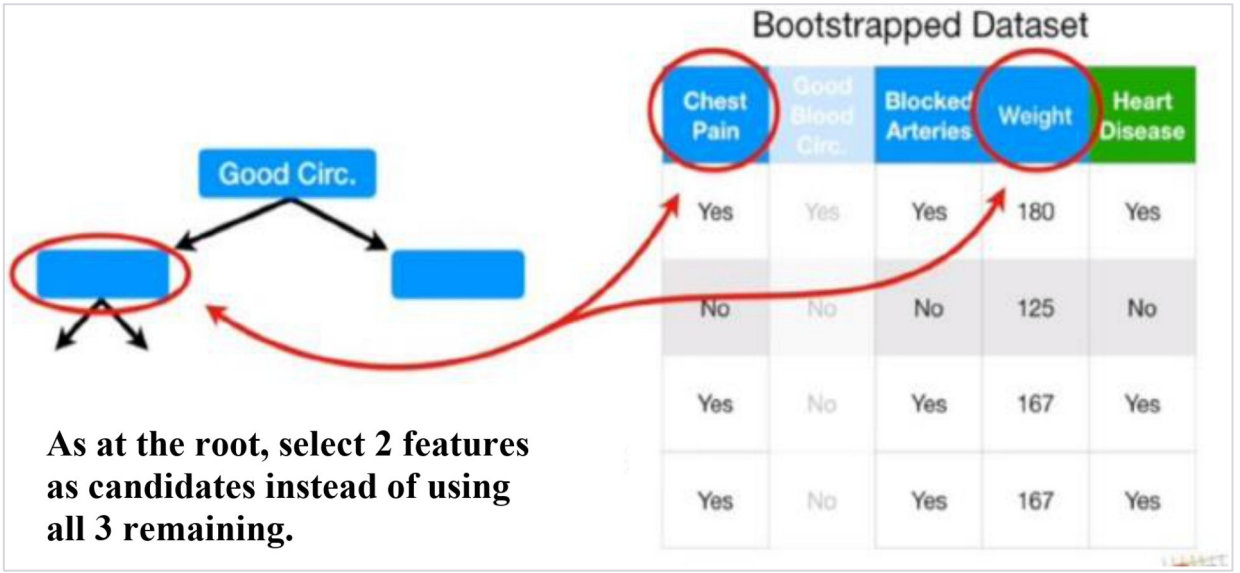
How to Build ② Only a Subset of Features per Node

At the root, pick just 2 at random (BC · BA) and take the best of them

Root: choose among 2 random features



Next node: again 2 of the remaining 3

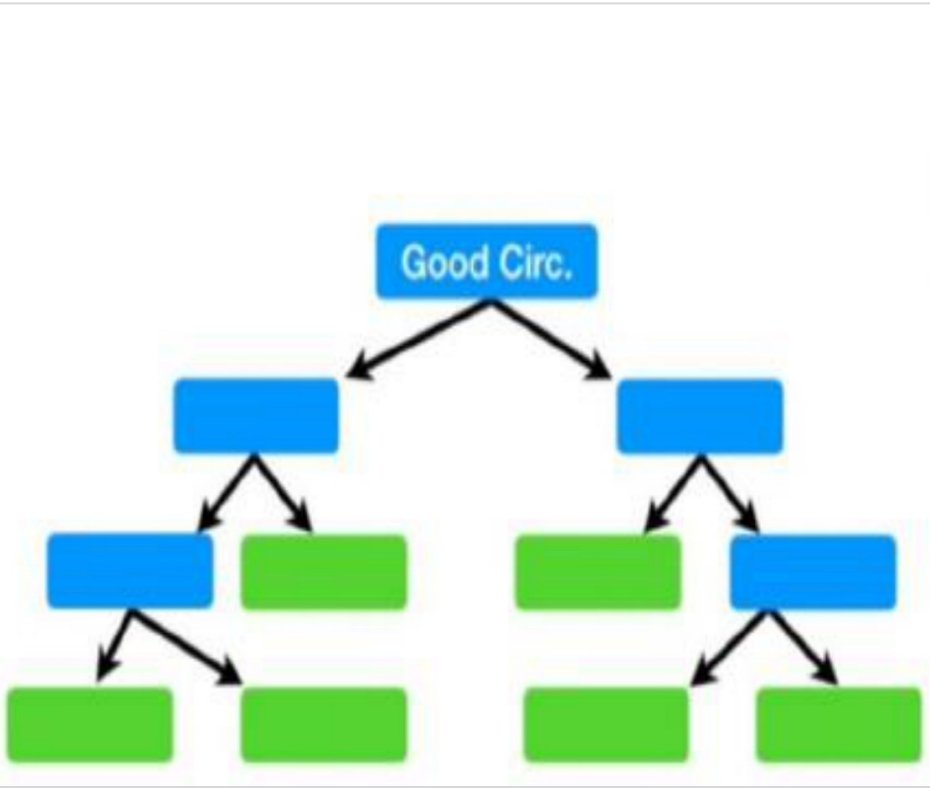


How to Build ② (cont.) — Completing the Tree

Grow the tree fully on bootstrap data, seeing only a random subset at every step

max_features

- Classification default: square root of the number of features (scikit-learn "sqrt")
- Recommended for regression: all features (1.0) — scikit-learn User Guide
- Smaller means lower correlation between trees, and lower accuracy per tree

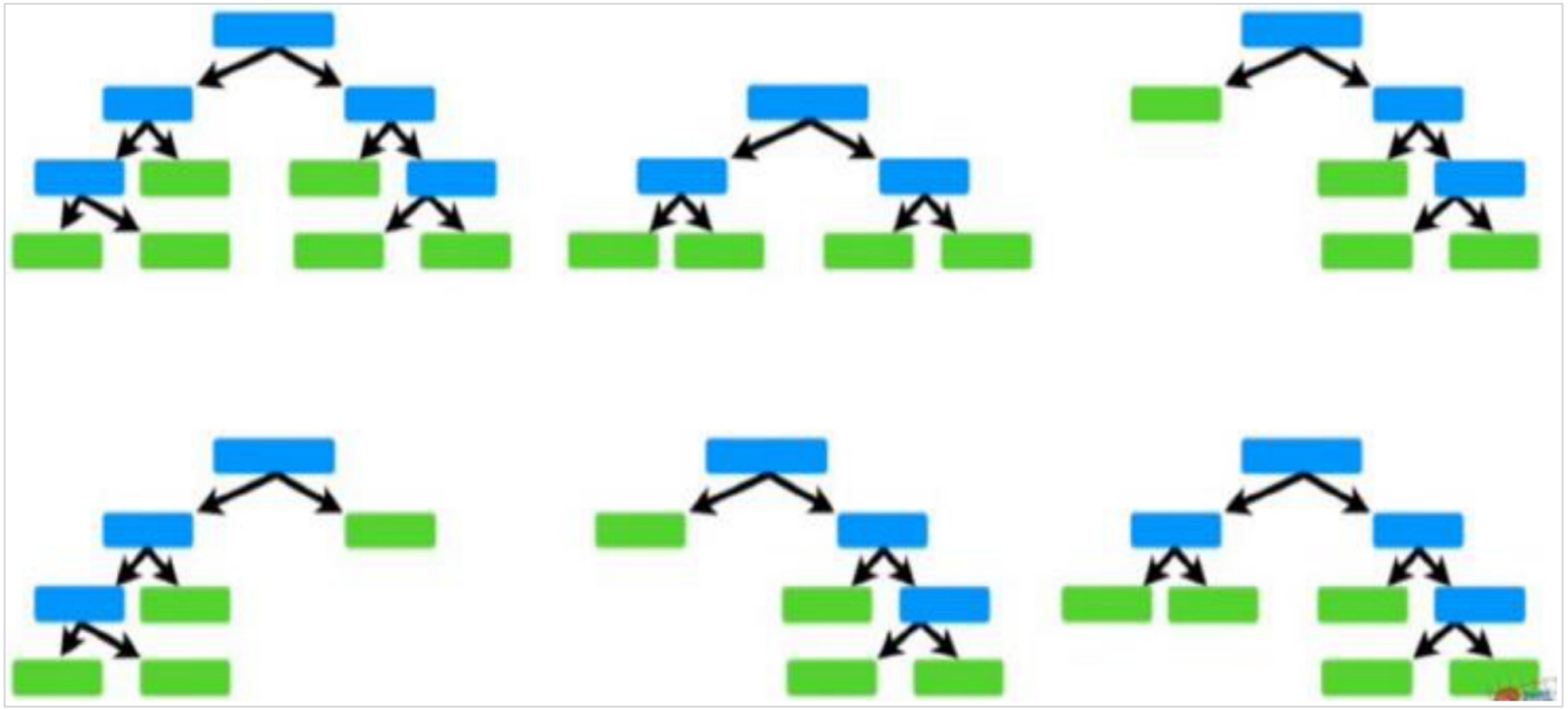


Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

How to Build ③ Repeat for the Desired Number of Trees

The bootstrap and feature choices differ each time, so trees of various shapes are built



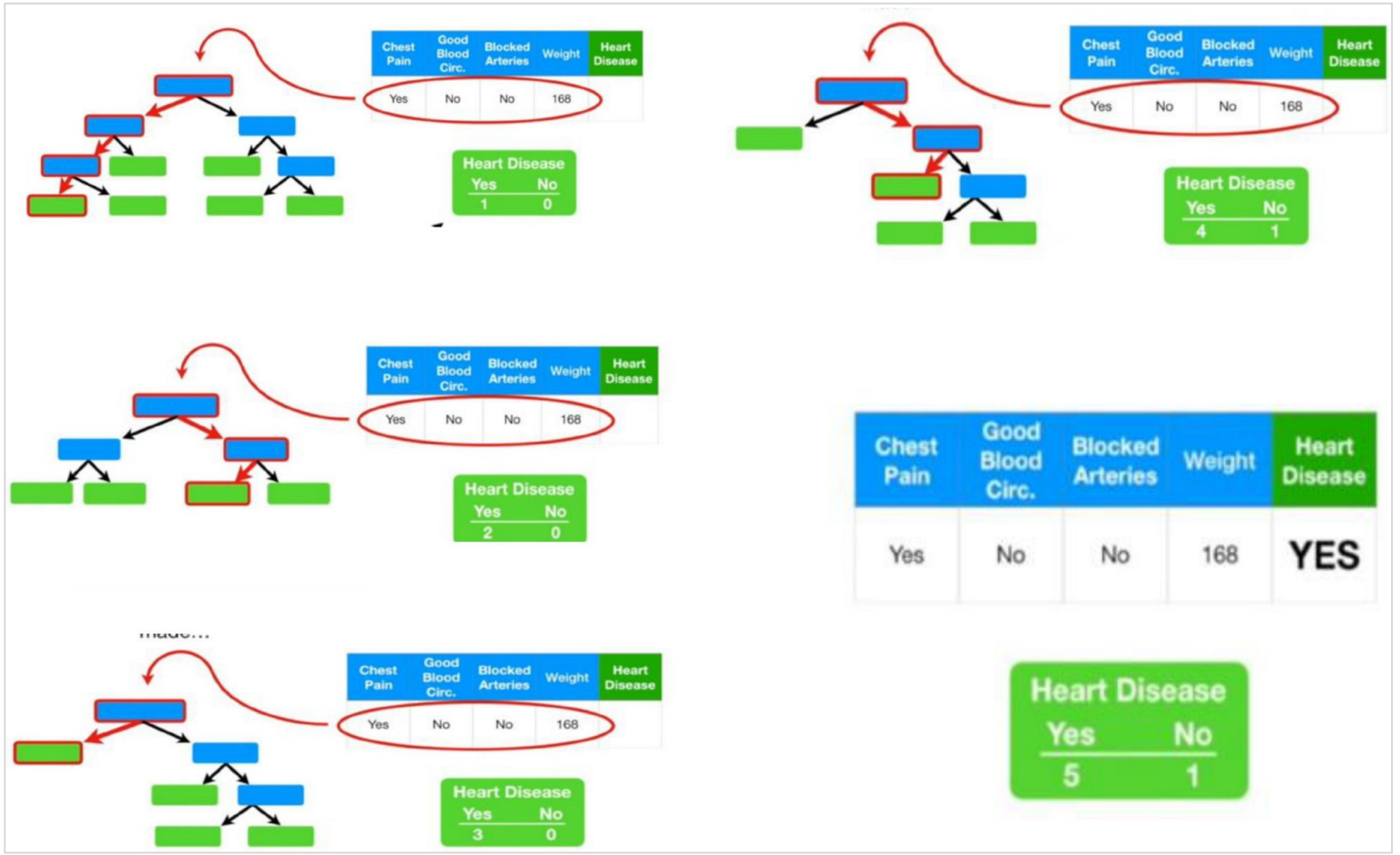
scikit-learn default n_estimators = 100. Adding trees does not worsen overfitting; performance converges.

Diagnosing a New Patient with Bagging

Run the new patient's data through all trees and vote

Result

- Vote: Yes 5 : No 1
- → majority vote: "Heart disease Yes"
- Bagging = Bootstrapping + Aggregate

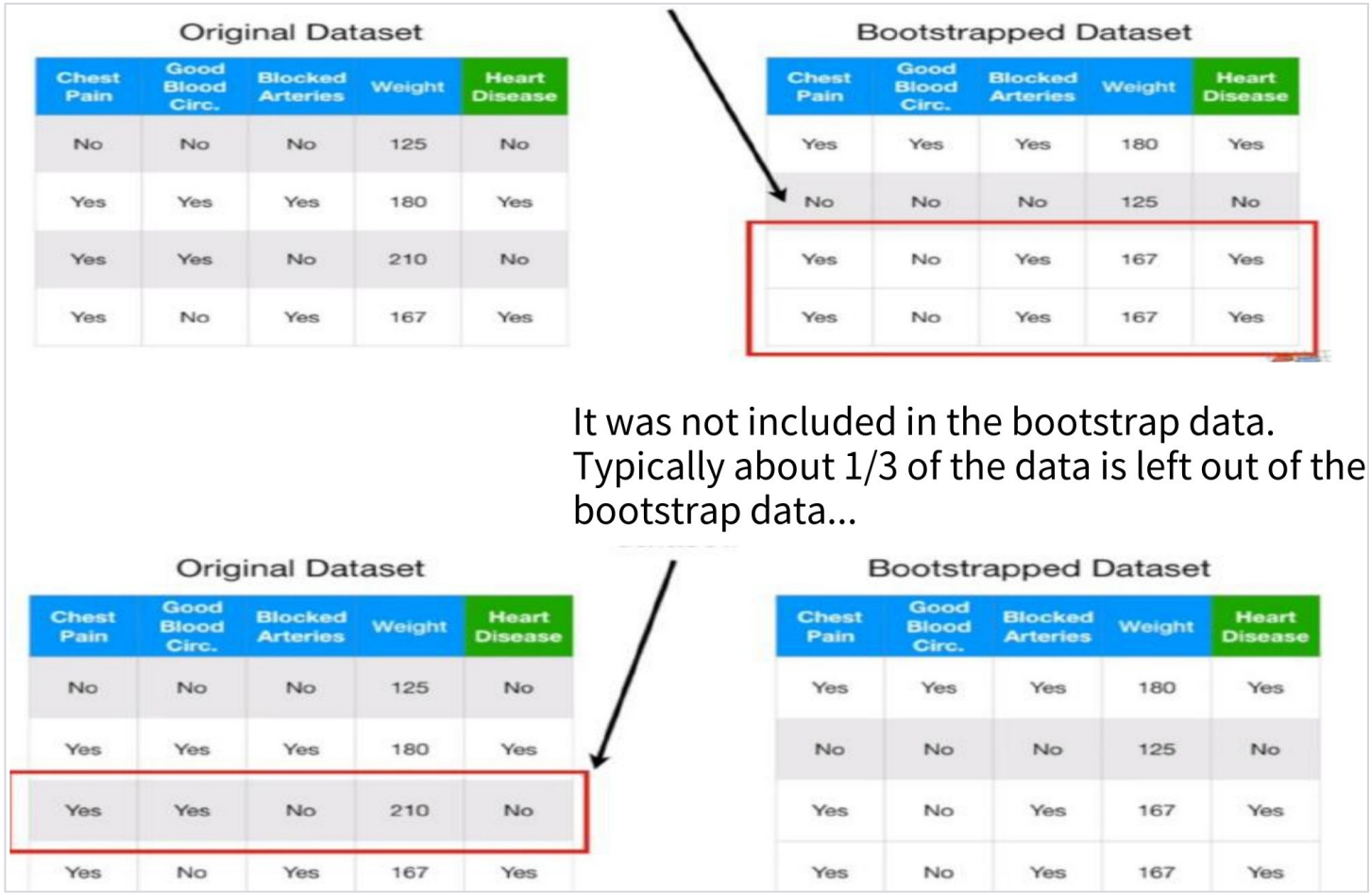


Out-of-Bag (OOB) Test — Data That Were Not Drawn

The bootstrap allows duplicates, so some samples are never drawn

Key point

- Typically about 1/3 are left out of the bootstrap
- Left-out samples = Out-of-Bag data
- They were not used to train this tree, so they can be used for validation



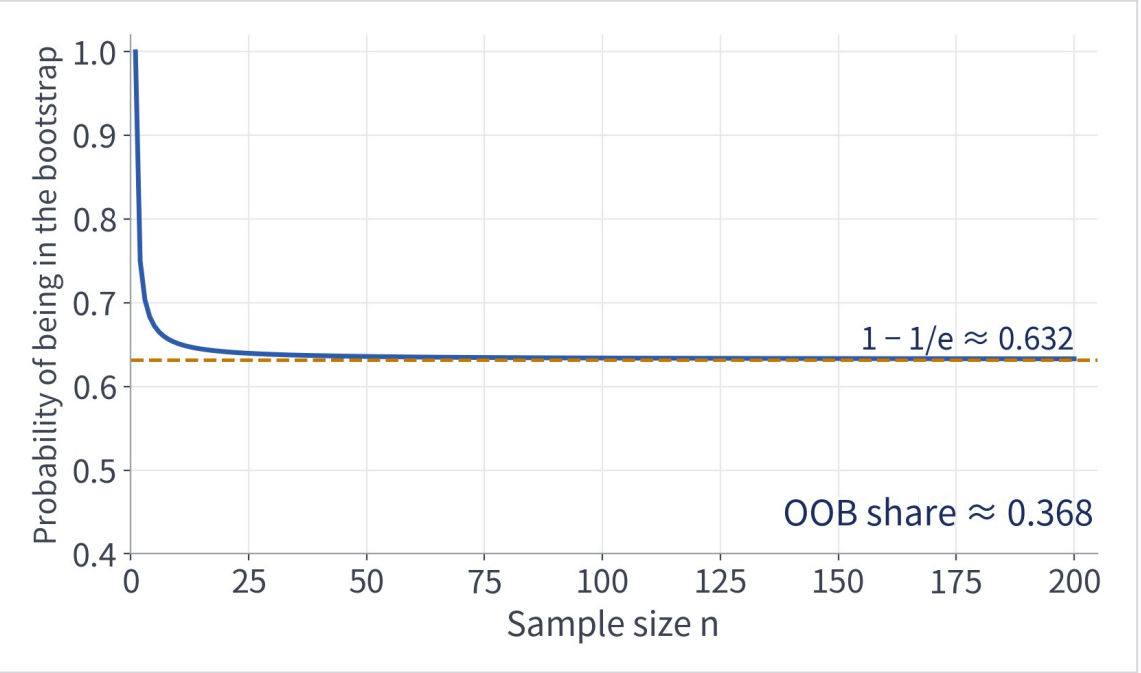
Supplement — Why "About 1/3"?

n draws with replacement from n: the probability a given sample is never drawn is $(1 - 1/n)^n \to 1/e$

$$\Pr(i \in \text{bootstrap}) = 1 - \left(1 - \frac{1}{n}\right)^n \longrightarrow 1 - e^{-1} \approx 0.632$$

Summary

- Share drawn ≈ 63.2%
- OOB share ≈ 36.8% — the "about 1/3" of the original lecture
- Beyond a few hundred thousand, it is almost exactly 1/e



Validating Trees with OOB Data

Data not included in the bootstrap are called the "Out-of-Bag dataset"

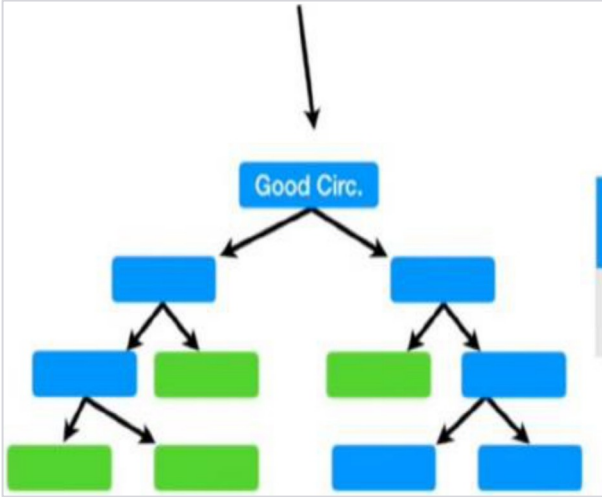
The 4th patient is OOB

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

Classify the OOB sample with this tree

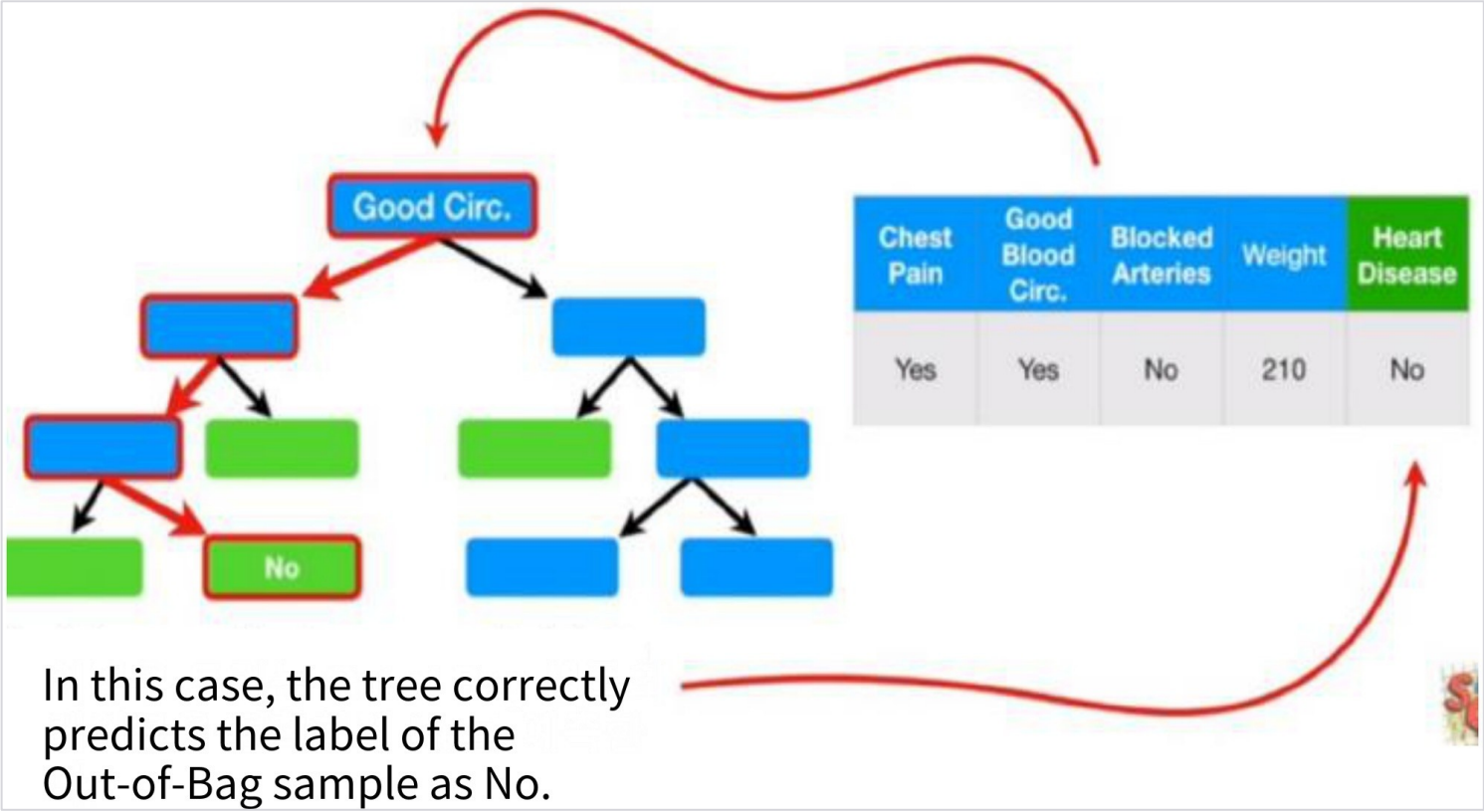


Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	No	210	No

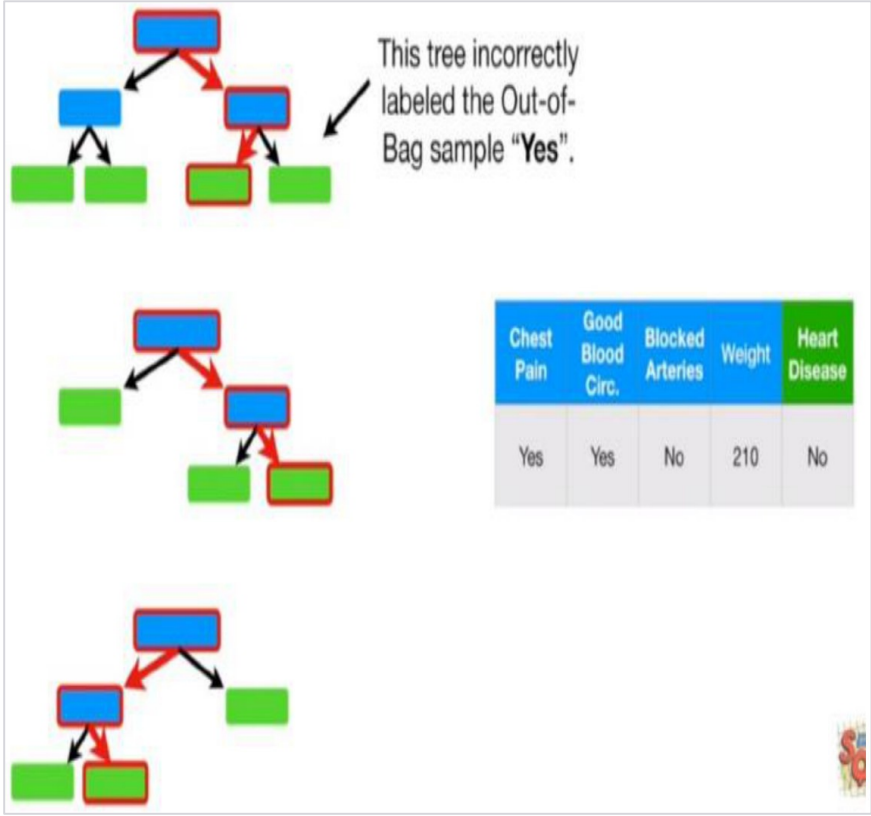
Classify the OOB Sample with Each Tree

Only the trees that did not train on this sample vote

Tree 1: predicts the correct "No"



Other trees: one tree misclassifies it as "Yes"

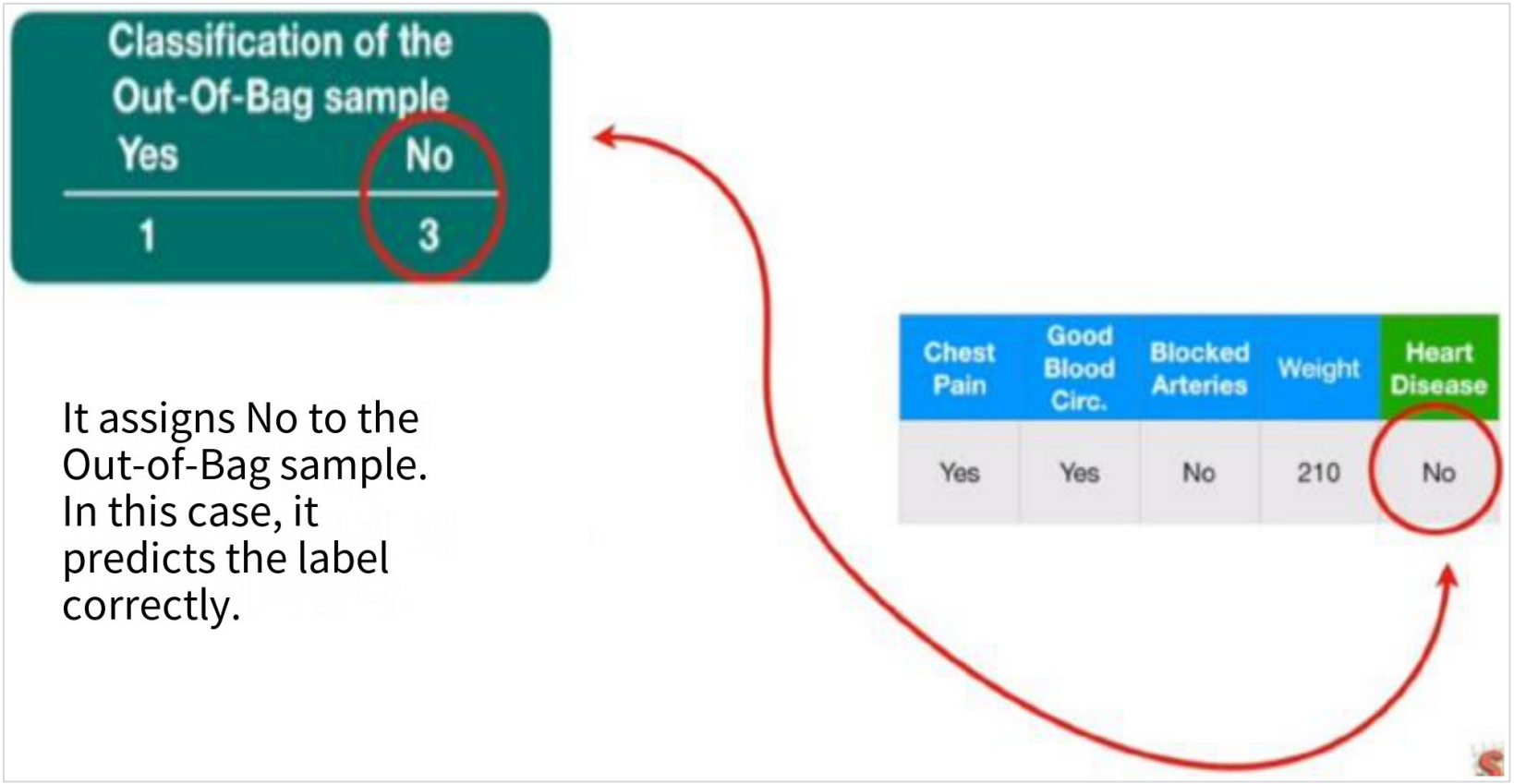


Final Classification of the OOB Sample

Vote: No 3 : Yes 1 → "No" — matches the truth

Result

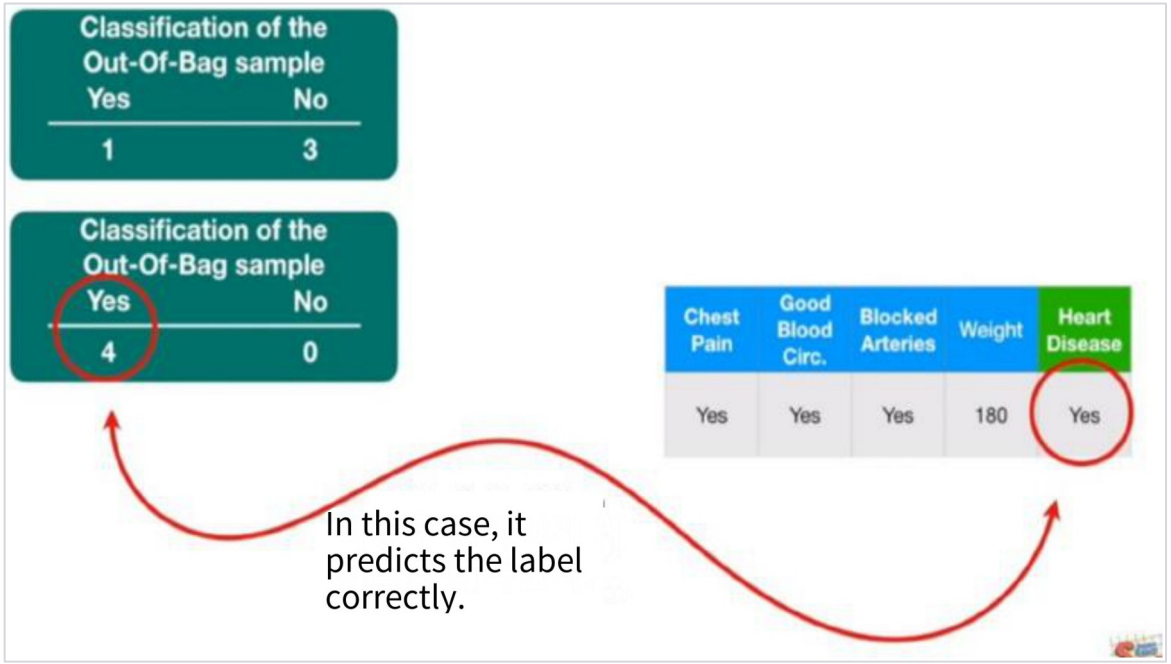
- Majority vote of the trees that left out the OOB sample
- In this case the label is predicted correctly
- Repeat for Every OOB Sample



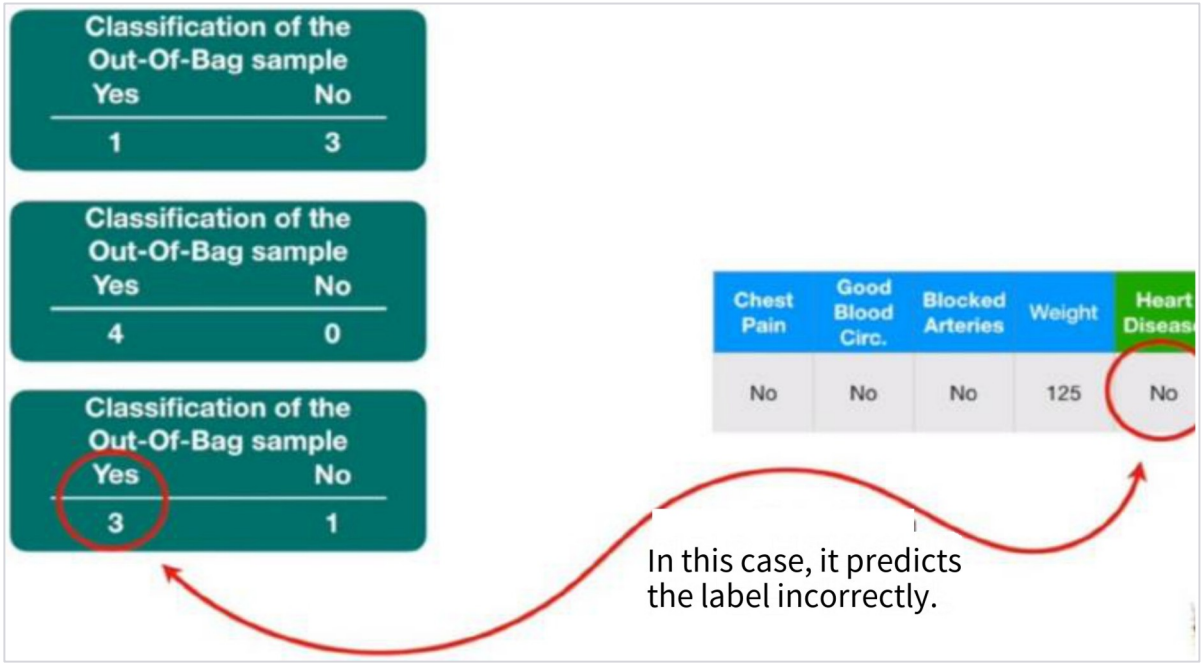
Repeat for all OOB samples

Sometimes right, sometimes wrong

Yes 4 : No 0 → truth Yes (correct)



Yes 3 : No 1 → truth No (incorrect)

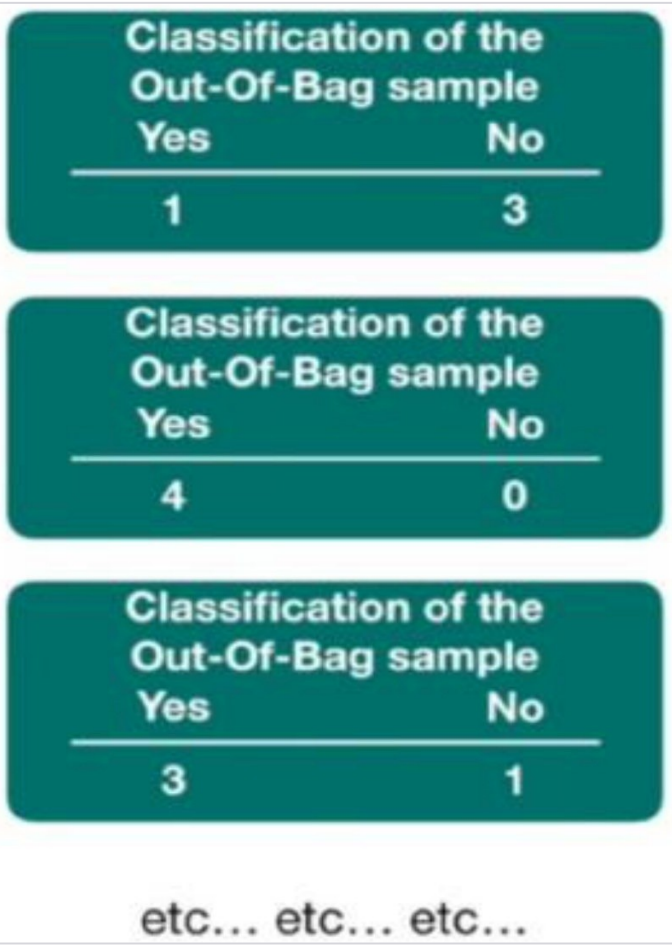


OOB Error = Share of Misclassified OOB Samples

Estimates RF accuracy without a separate validation set

Definition and use

- OOB accuracy = share of correctly classified OOB samples
- OOB error = 1 − OOB accuracy
- Used to choose the number of trees · max_features
- OOB tends to be pessimistic → also cross-validate when possible



The accuracy of the random forest can be measured by the share of Out-of-Bag samples it classifies correctly.

The share of misclassified Out-of-Bag samples is called the "Out-of-Bag error".

Supplement — RF in One Go with scikit-learn

Breast cancer data, same split as the earlier tree example (scikit-learn 1.8 output)

Python · scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,      # number of trees (default)
    max_features="sqrt",   # candidate features per node
(default)
    oob_score=True,        # compute OOB accuracy
    random_state=0, n_jobs=-1)
rf.fit(X_train, y_train)

rf.score(X_train, y_train)   # 1.000
rf.score(X_test, y_test)     # 0.958
rf.oob_score_                # 0.955
```

Compared with a single tree

- Single tree test 0.937 → RF 0.958
- OOB 0.955 is almost the same as the test accuracy
- Even with fully grown trees, averaging suppresses variance

03

Part 3 · Handling Missing Values

Missing Values — How Trees and Random Forests Handle Them

Simple imputation · using correlated columns · regression imputation · RF proximity-based iterative imputation · new samples

Background — Why and How Values Go Missing

Whether imputation is acceptable depends on the missingness mechanism

Type	Meaning	Example	Simple imputation
MCAR	Missing completely at random	Random gaps from input errors	Little bias
MAR	Missingness explained by other observed variables	Smaller stocks lack analyst estimates	Can be corrected using other columns
MNAR	Missingness itself relates to the value	Firms with poor results delay disclosure	Bias risk — missingness is informative

Finance case — Bryzgalova · Lerner · Lettau · Pelger, "Missing Financial Data" (RFS 2025)

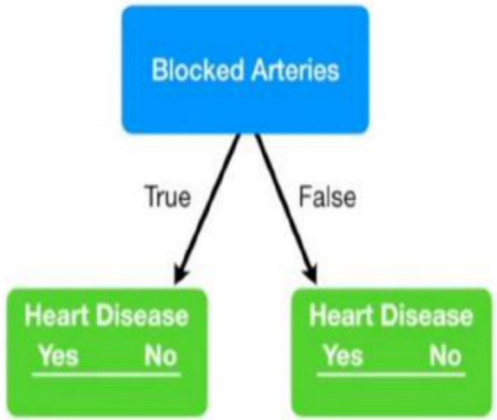
- Missing firm-characteristic data affect over 70% of firms, which make up about half of market cap
- The missingness pattern is systematic, so traditional methods such as mean imputation create bias

Decision Trees and Missing Values

A patient whose Blocked Arteries (BA) value is "???" — earlier we skipped them in the tally

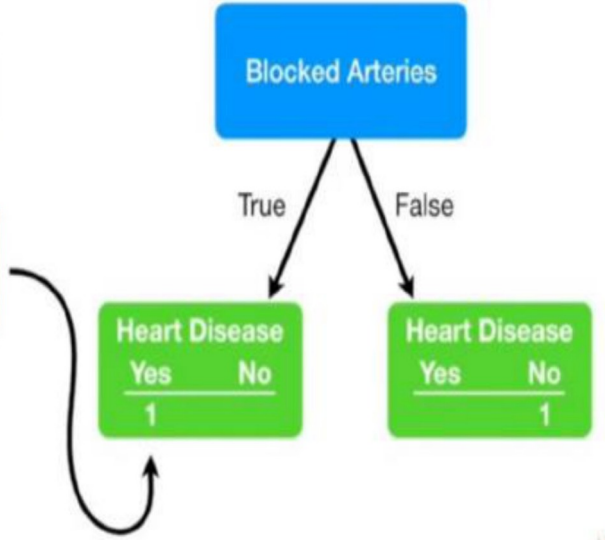
Tree on BA

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...



Patients with a value are tallied normally

Chest Pain	Good Blood Circulation	Blocked Arteries	Heart Disease
No	No	No	No
Yes	Yes	Yes	Yes
Yes	Yes	No	No
Yes	No	???	Yes
etc...	etc...	etc...	etc...

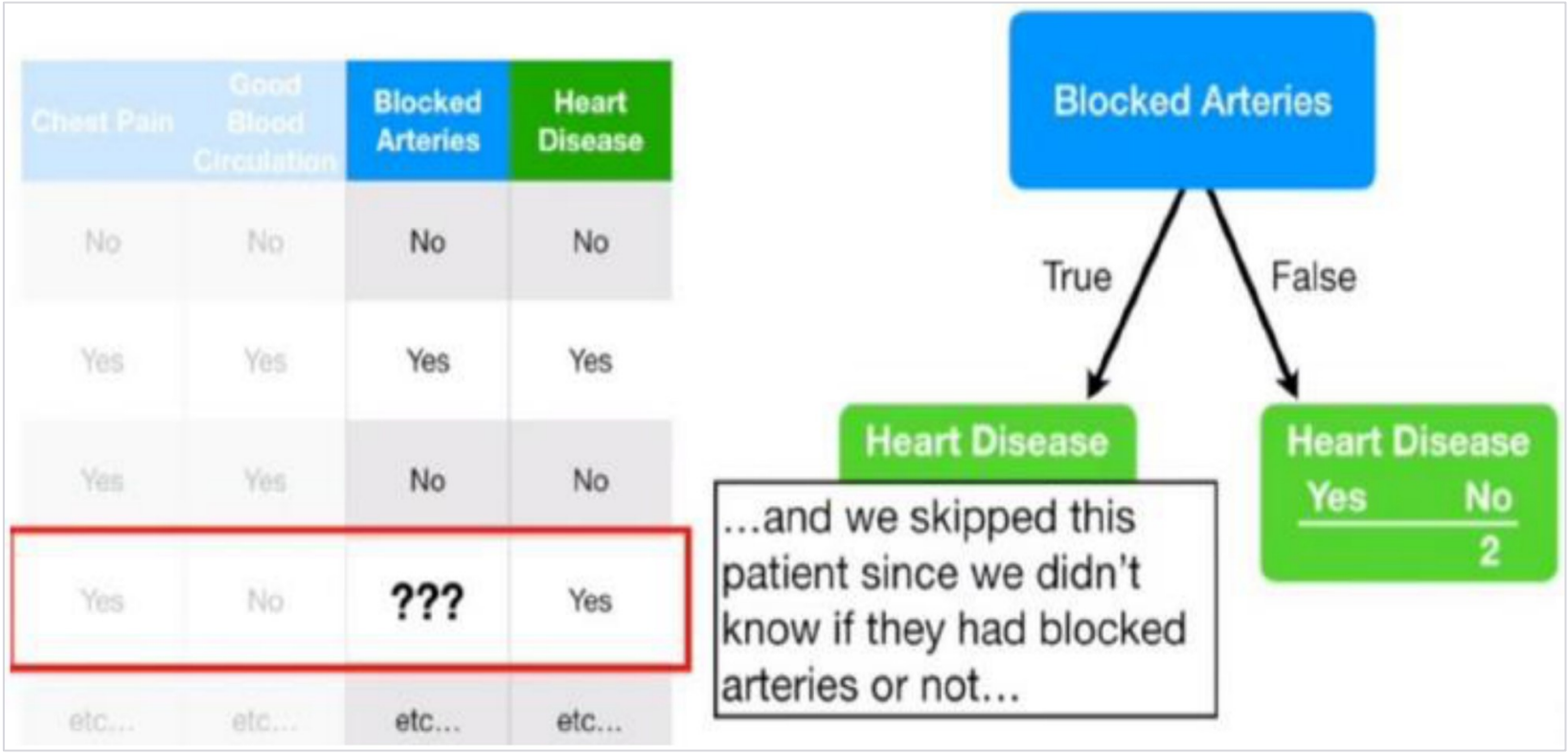


Is skipping the best option?

Dropping the "???" patient also discards their other information (CP, heart disease, etc.)

Alternatives

- Categorical: ① impute the most frequent value ② use another highly correlated column
- Numeric: ① impute the mean · median ② estimate by regression
- Figures: from StatQuest lecture slides

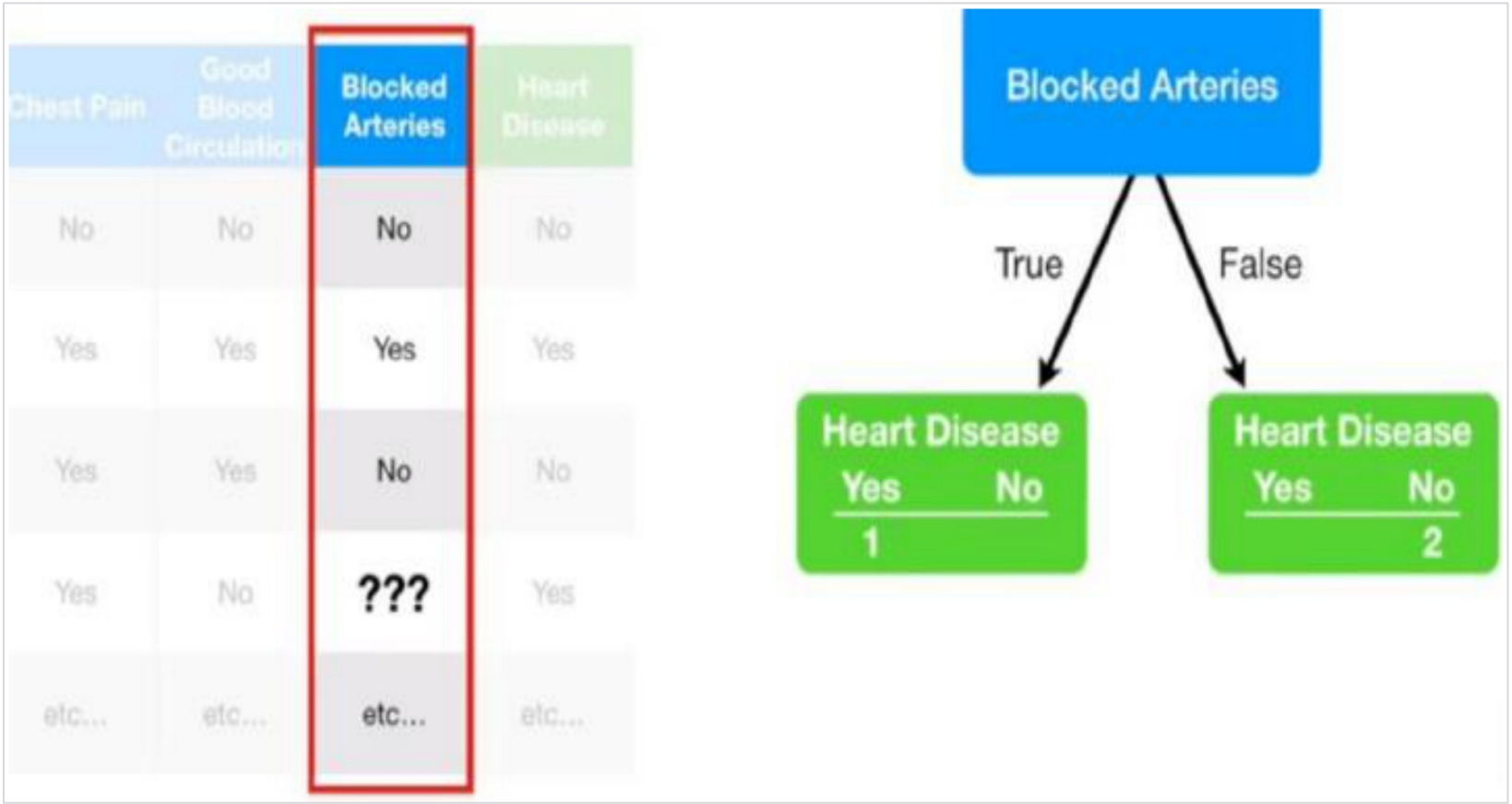


Categorical Missing ① Use the Most Frequent Value

Fill with the value that appears most often in the Blocked Arteries (BA) column

Features

- The simplest option
- Uses no information from other columns
- With many missing values, it piles into one category

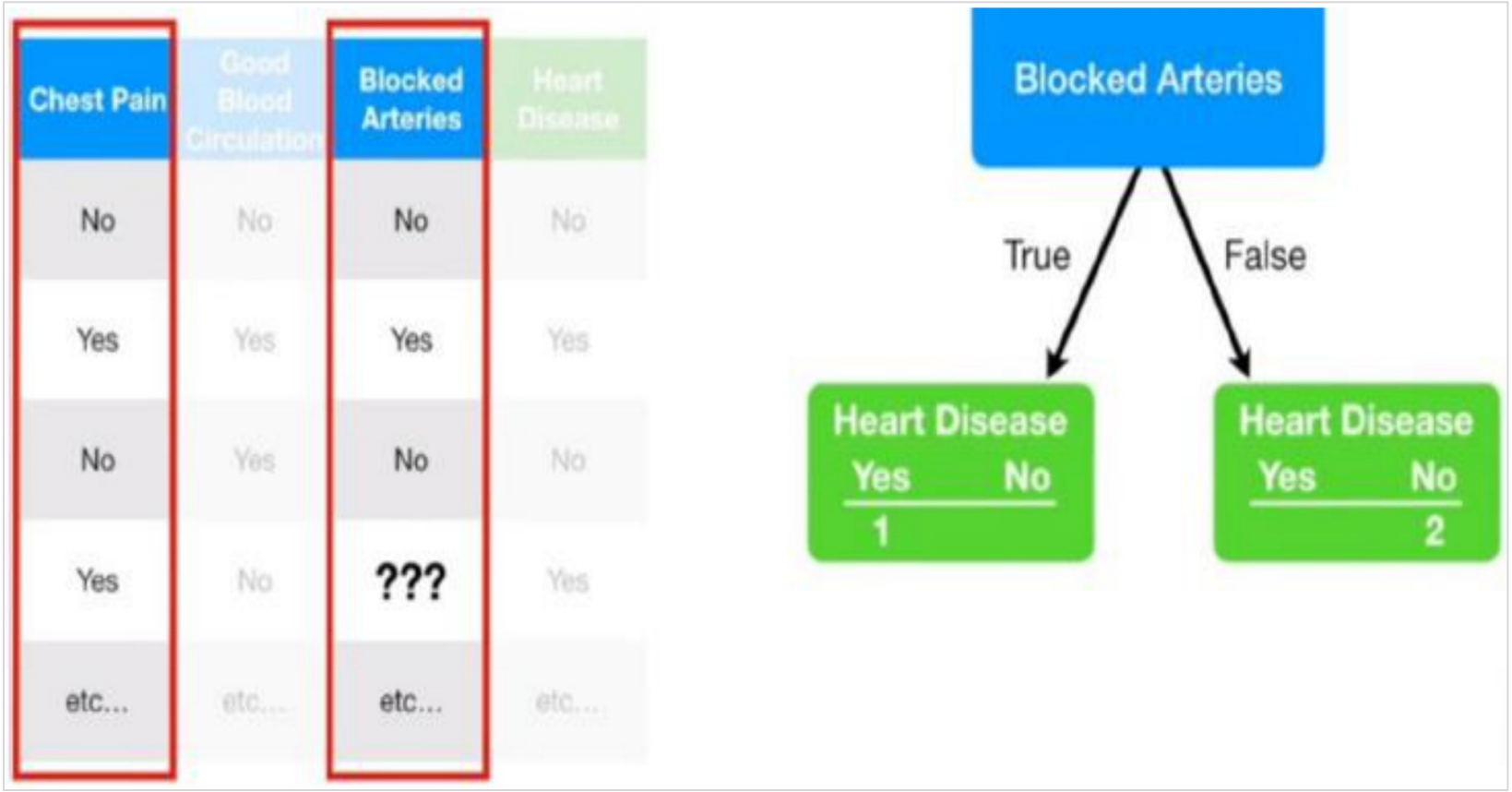


Categorical Missing ② Use a Highly Correlated Column as a Guide

Find Chest Pain (CP), the column most correlated with BA, and follow its values

Idea

- If CP and BA move together,
- a patient with CP = Yes likely has BA = Yes
- → uses information in other columns (MAR)



Applying Categorical ② — Fill by Following CP

Look at rows where CP and BA are both No or both Yes, set a rule, then apply it to "???"

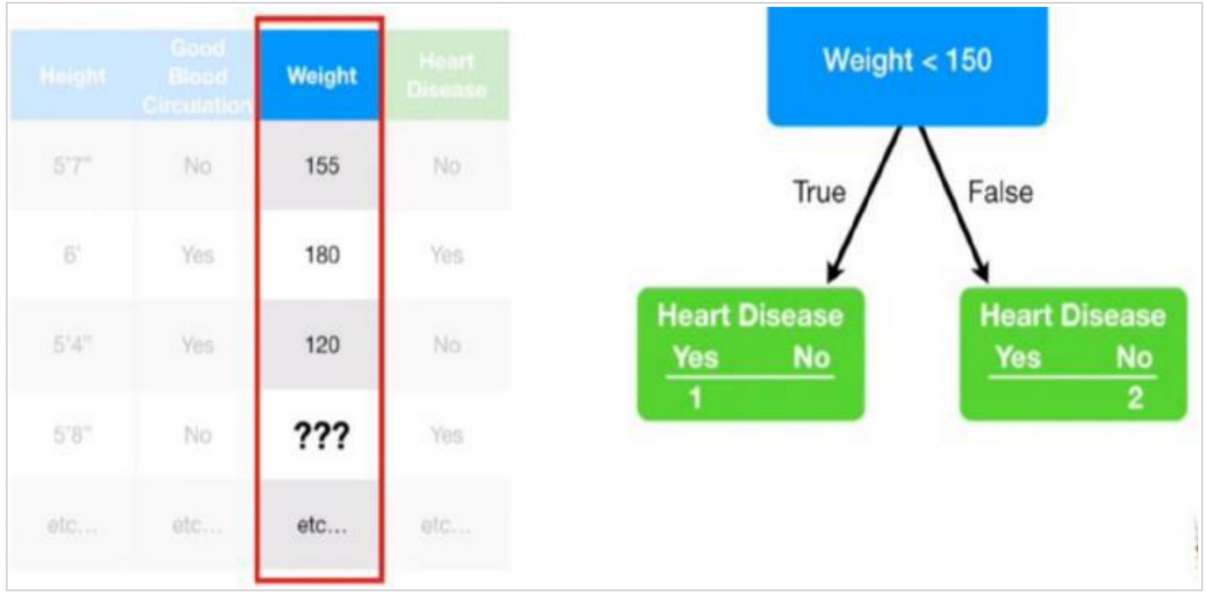


The missing patient has CP = Yes → fill BA with "Yes" too.

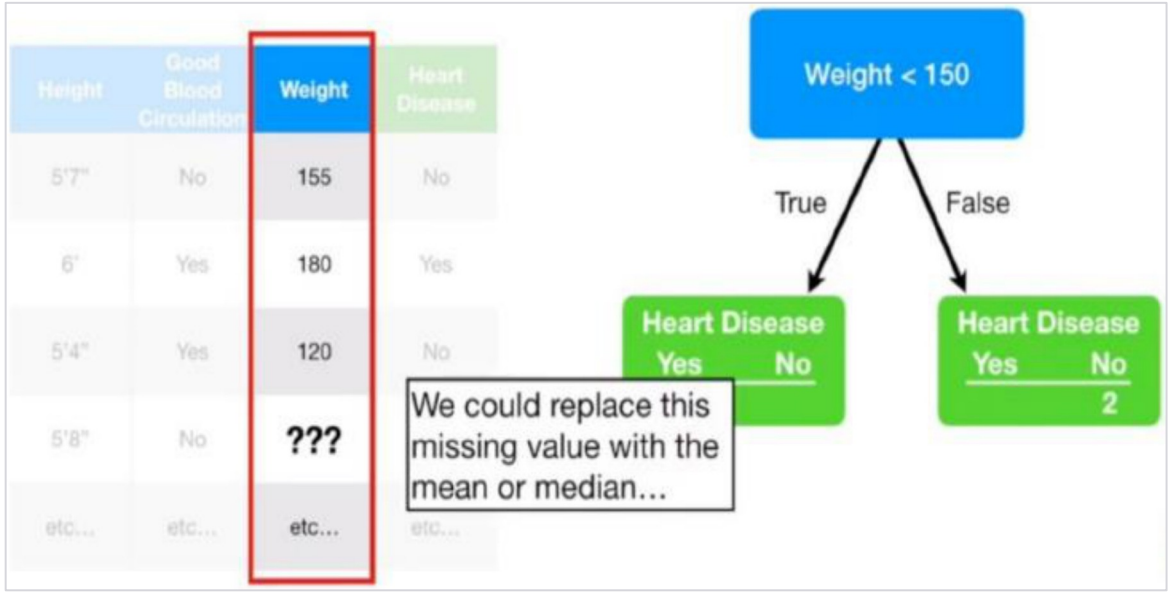
Numeric Missing ① Mean or Median

"???" in the Weight column

Missing weight



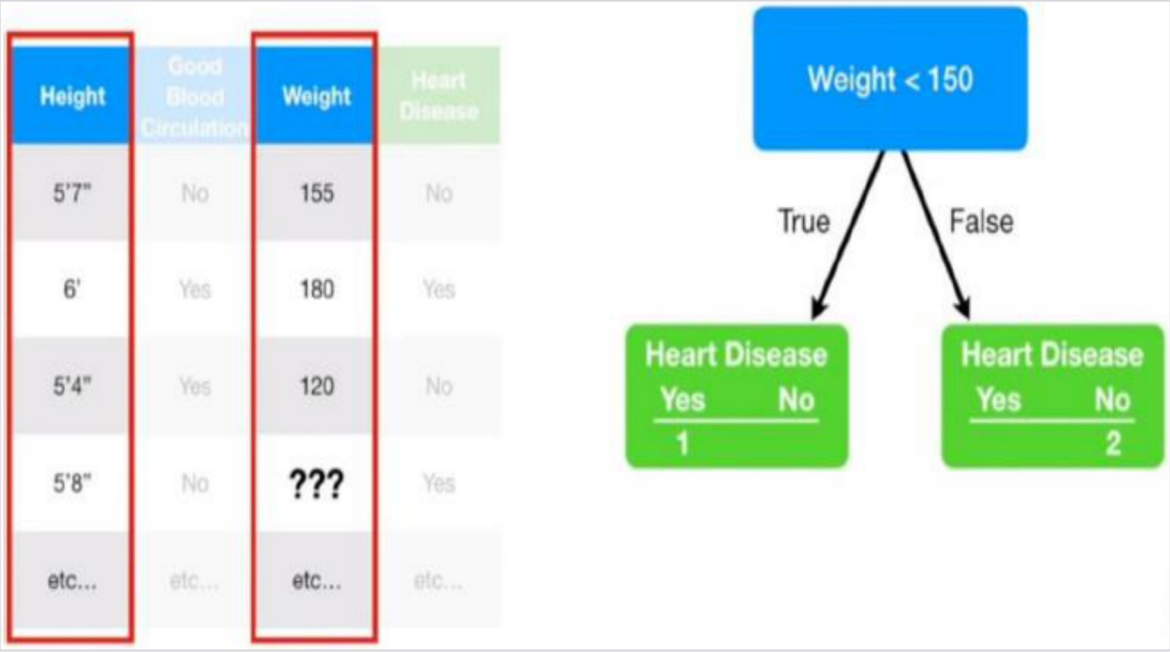
Impute the mean · median



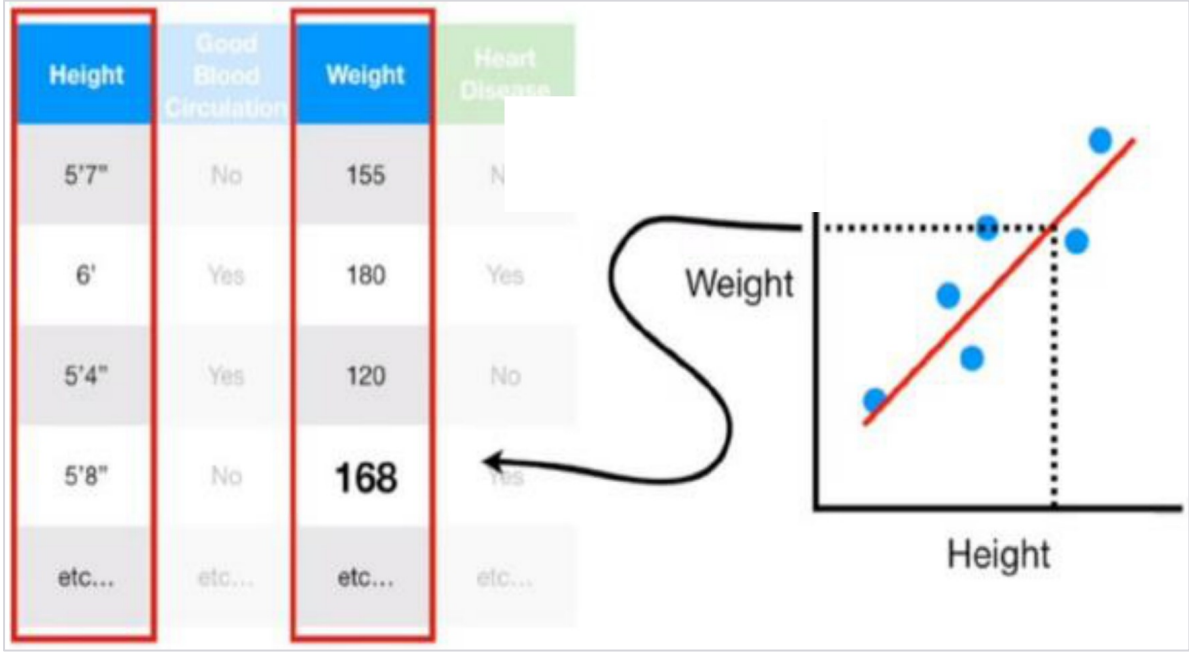
Numeric Missing ② Regression on a Correlated Column

Height and weight are correlated → regress weight on height to estimate the missing value

Look at height and weight together



Regression line estimates 168



Missing Values in Random Forests — Two Cases

Missing values in the original data · missing values in a new sample to classify

Two cases

- ① The data used to build the RF have missing values → patient 4's BA · weight
- ② The new sample to classify has missing values
- ① uses proximity-based iterative imputation; ② plugs in candidate values and votes
- Figures: from StatQuest lecture slides

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	???	???	No

Random Forests consider 2 types of missing data…

1) Missing data in the original dataset used to create the random forest.

2) Missing data in a new sample that you want to categorize.

New Sample

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	???	

① Original Data — First, a Rough Fill

Within the same class (heart disease No): mode for categorical, median for numeric

Initial guess

- BA: mode among patients with heart disease No = "No"
- Weight: median of the same patients = 167.5
- This is only a starting point — refined in the next steps



Step 1 — Build an RF on the Filled Data

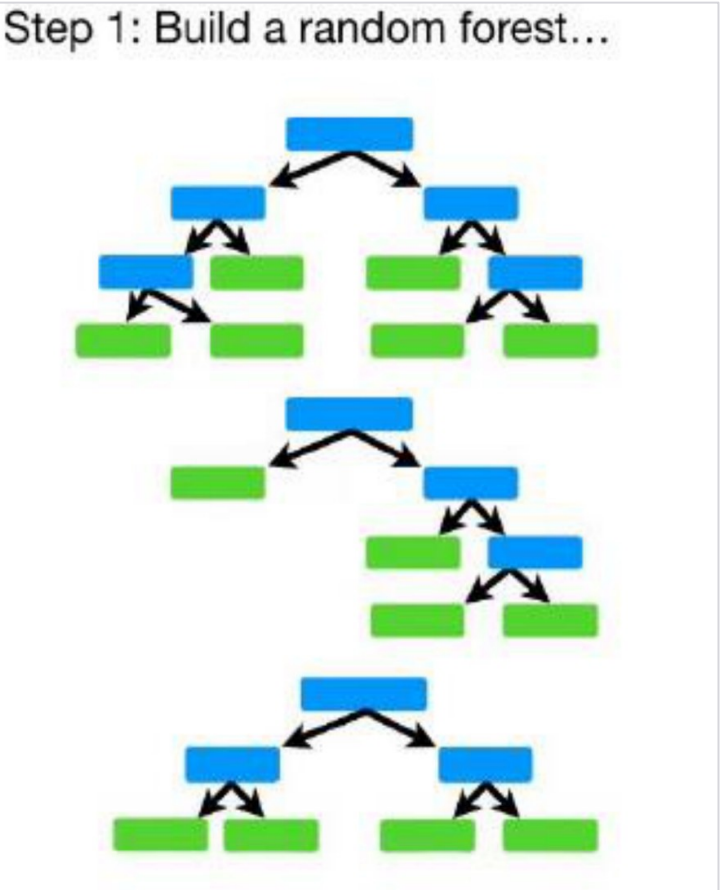
Train a random forest on the dataset with the initial guesses

Next

- The goal is not prediction but measuring "similarity between samples"
- Uses the fact that trees send similar samples to the same leaf

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



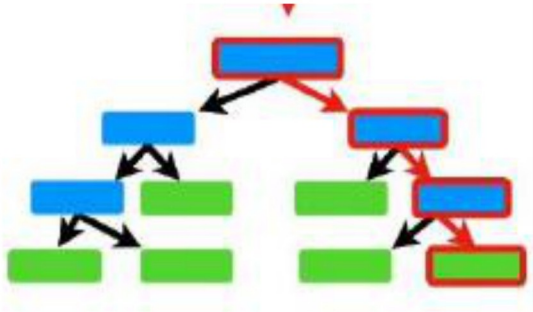
Step 2 — Run All Data Through All Trees

Record which leaf each sample lands in

Samples 1 · 2 into the first tree

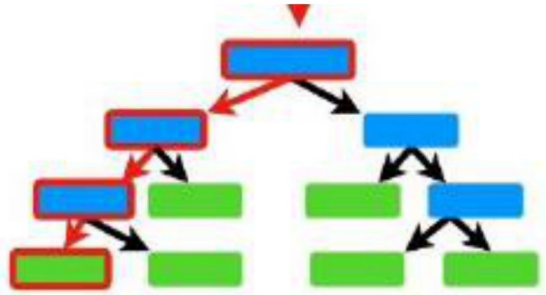
Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



Filled-in Missing Values

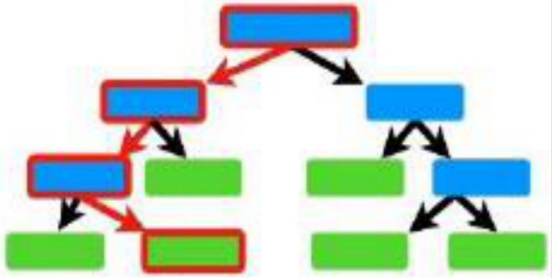
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



Samples 3 · 4 into the same tree

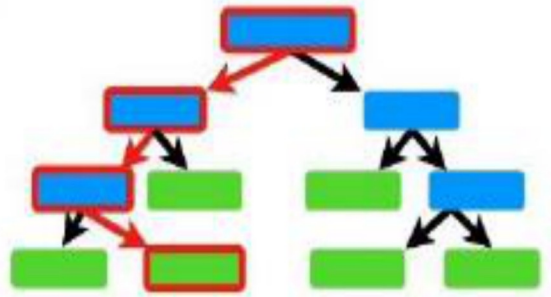
Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No



Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No

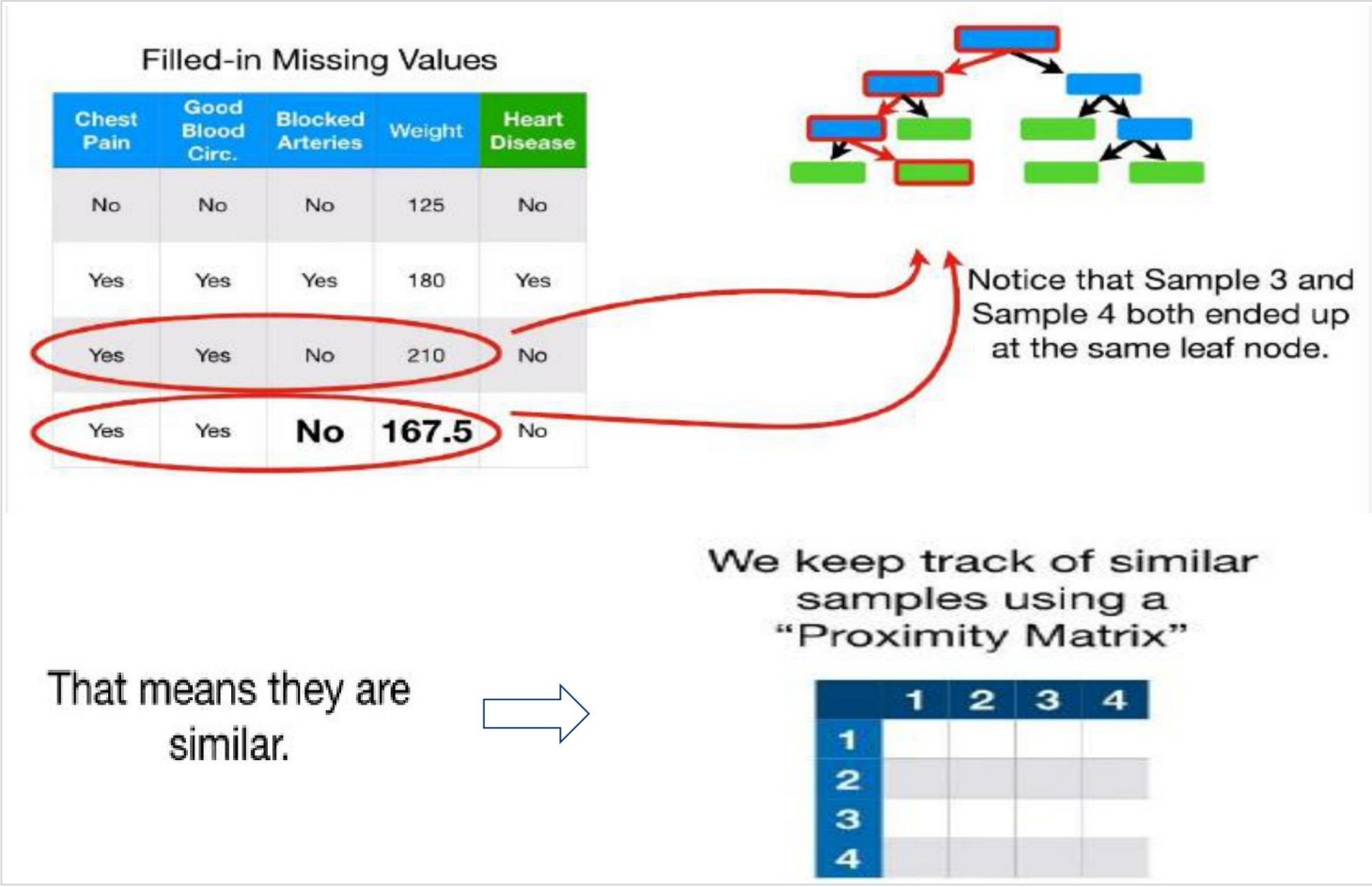


Samples in the Same Leaf = Similar Samples

Samples 3 and 4 land in the same leaf → record in the proximity matrix

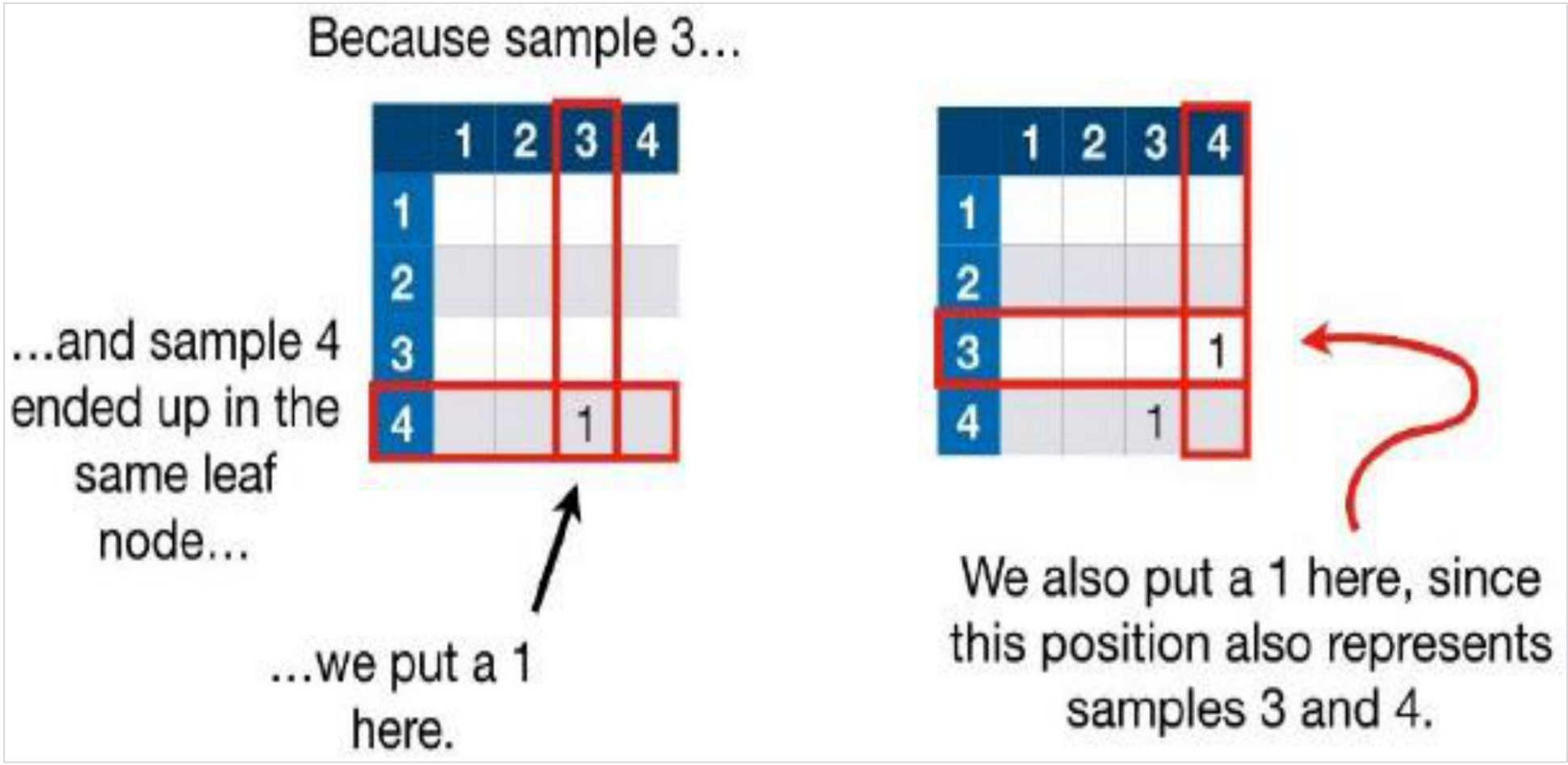
Proximity matrix

- Rows · columns = sample numbers
- Landing in the same leaf adds +1 to that cell
- Accumulates with every tree



Add 1 to the Proximity Matrix

Cells (3, 4) and (4, 3) both mean the same pair



Keep Accumulating in the Second and Third Trees

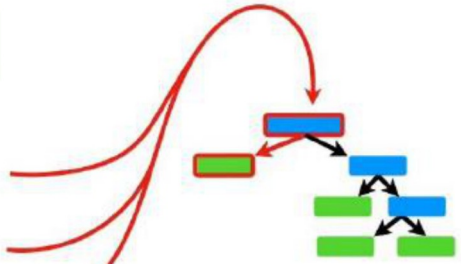
With each tree, add 1 for every pair that lands in the same leaf

After tree 2: samples 2 · 3 · 4

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**167.5**	No

NOTE: Samples 2, 3 and 4 all ended up in the same leaf node.



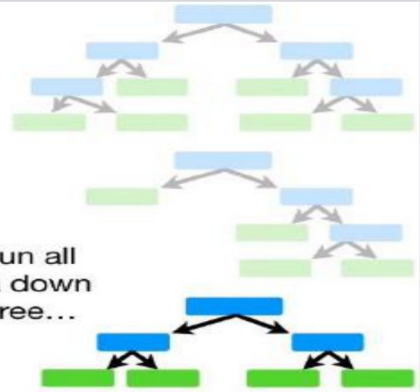
...after the second tree, we add 1 to any pair of samples that ended up in the same leaf node.

	1	2	3	4
1				
2			1	1
3		1		2
4		1	2	

After tree 3: (3, 4) = 3

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**No**	**180**	No



Now we run all of the data down the third tree...

...and here's the updated proximity matrix.

	1	2	3	4
1				
2			1	1
3		1		3
4		1	3	

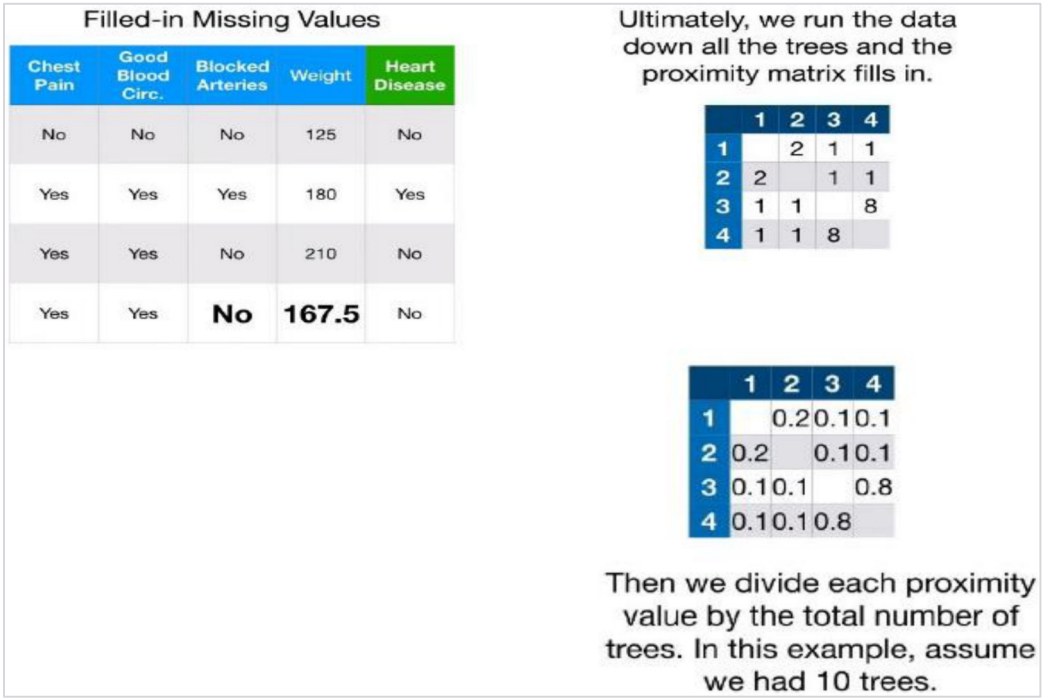
Only samples 3 and 4 ended up in the same leaf node.

After All Trees → Divide by the Number of Trees

e.g. 10 trees → (3, 4) = 8/10 = 0.8

Definition of proximity

- A value between 0 and 1
- The closer to 1, the more similar two samples look to the trees



$$\text{prox}(i,j) = \frac{1}{B}\sum_{b=1}^{B} \mathbf{1}\{\text{leaf}_b(i) = \text{leaf}_b(j)\}$$

Re-Estimating the Missing Values with Proximity

Row of sample 4: (1) 0.1 · (2) 0.1 · (3) 0.8

Idea

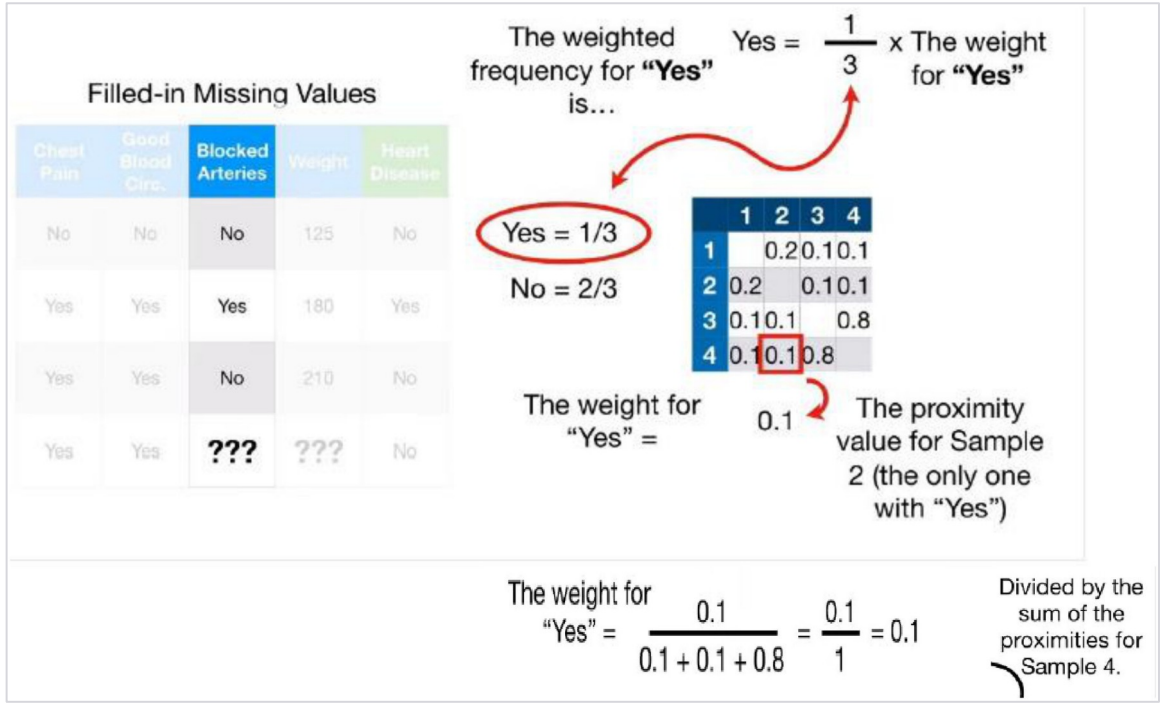
- Trust the values of samples close to sample 4, which has missing values
- = proximity-weighted vote (categorical) · weighted average (numeric)



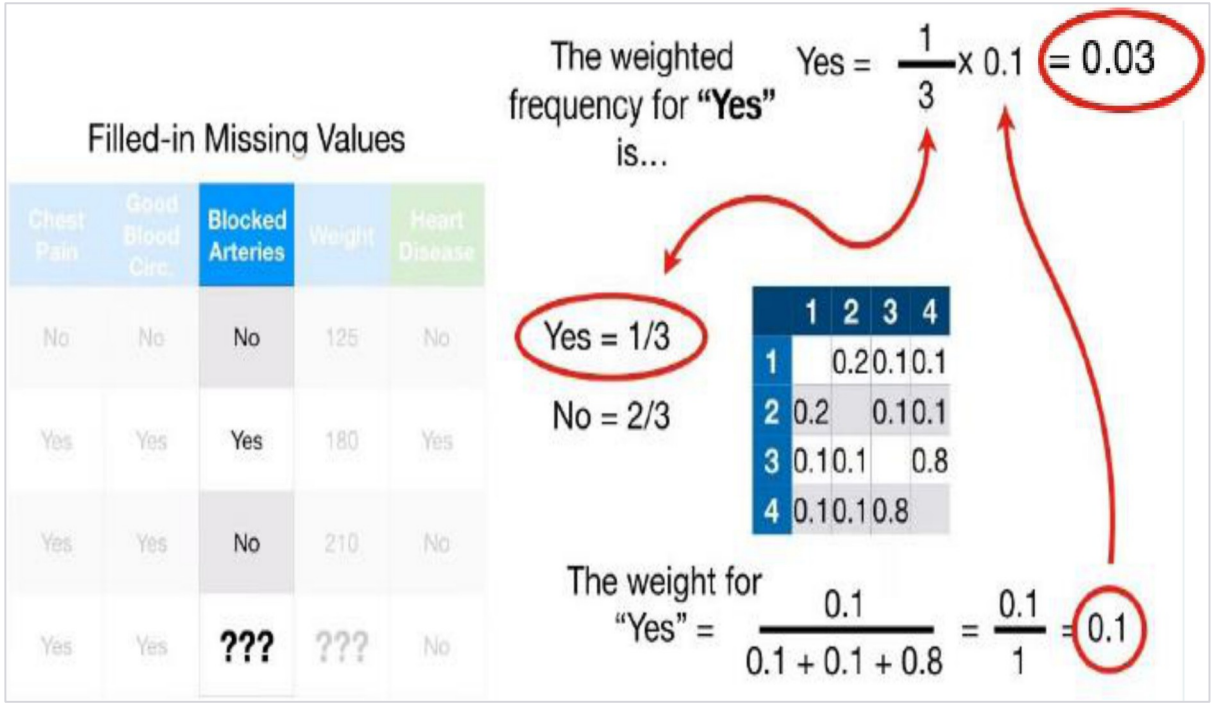
Categorical (BA): Proximity-Weighted Frequency

Weighted frequency of Yes = share of Yes 1/3 × weight for Yes 0.1

Weight for Yes = 0.1 / (0.1 + 0.1 + 0.8)

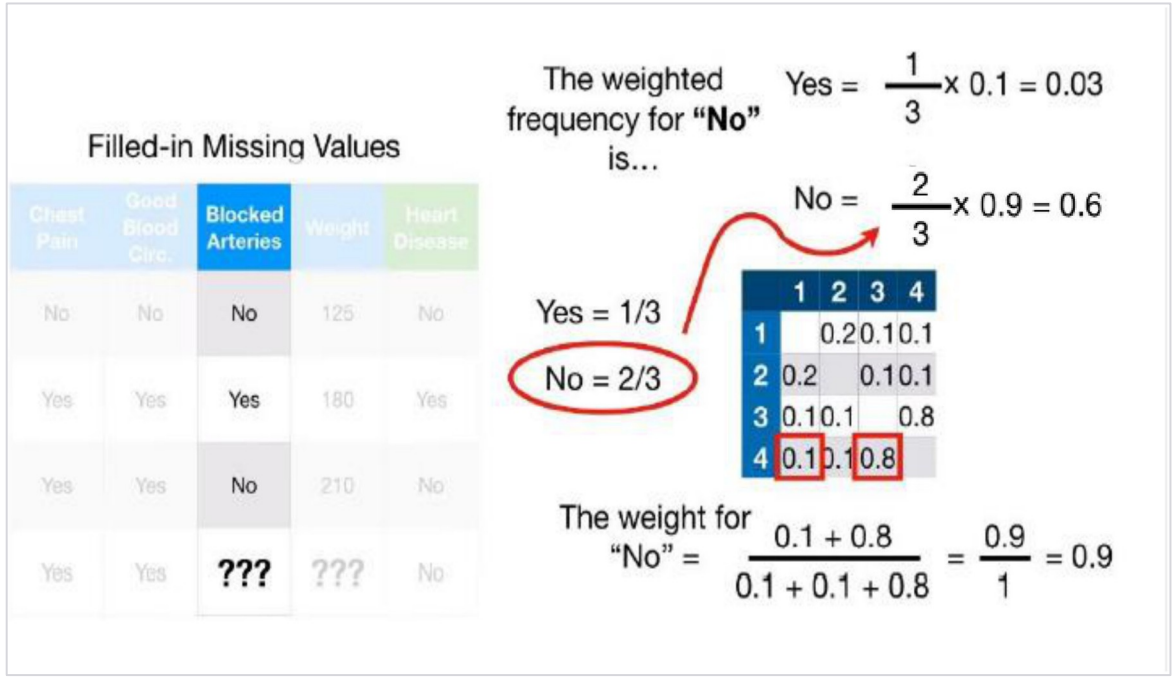


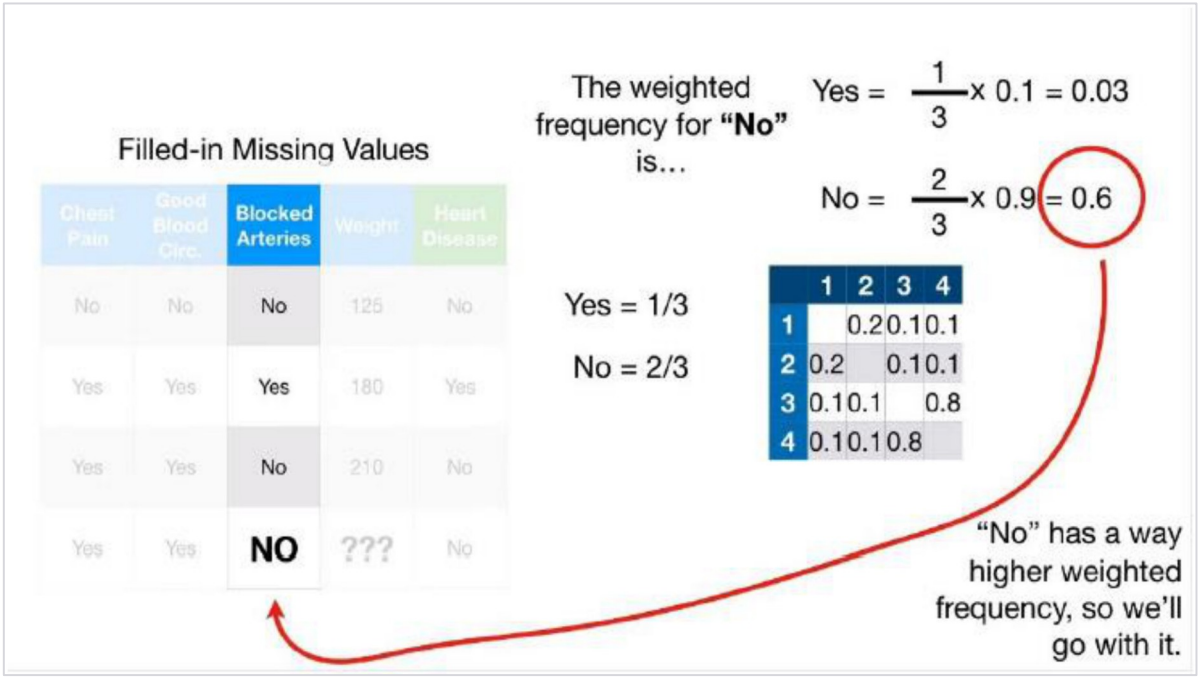
Weighted frequency of Yes = 0.03



The weighted frequency of No is much larger → "No"

Weighted frequency of No = 2/3 × 0.9 = 0.6





$$\text{Yes}: \ \frac{1}{3} \times \frac{0.1}{1.0} = 0.03 \qquad \text{No}: \ \frac{2}{3} \times \frac{0.1+0.8}{1.0} = 0.6$$

Supplement — Formulas for Proximity-Weighted Frequency · Average

Categorical: the value with the largest weighted frequency; numeric: the proximity-weighted average

$$\tilde{f}_i(c) = \Pr(c) \times \frac{\sum_{j:\, x_j = c} \operatorname{prox}(i, j)}{\sum_{j \neq i} \operatorname{prox}(i, j)}$$

$$\hat{x}_i = \sum_{j \neq i} x_j \frac{\operatorname{prox}(i, j)}{\sum_{k \neq i} \operatorname{prox}(i, k)}$$

Categorical (BA)

- Pr(c): share of value c in that column
- Fraction: sum of proximities to samples with value c / total proximity
- Fill with the largest c → "No"

Numeric (weight)

- Multiply other samples' values by their proximity share and add
- Sample 3 is closest at 0.8, so 210 gets most weight
- Result: 198.5 (next slide)

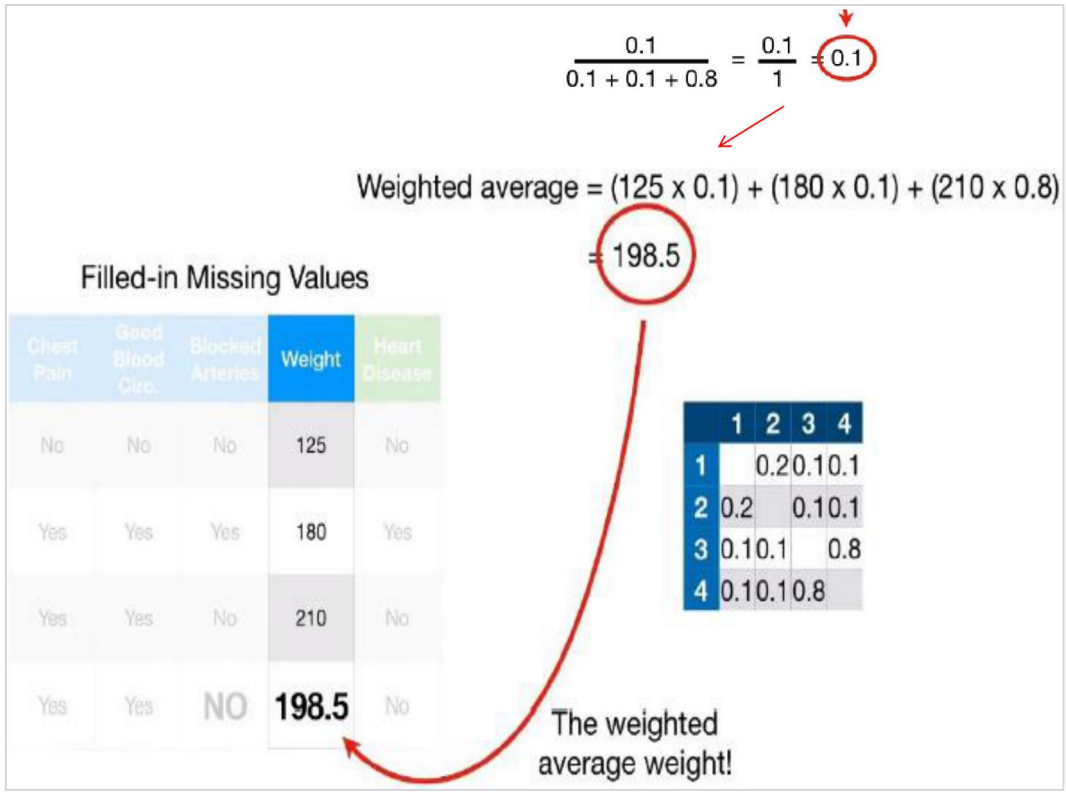
Numeric (Weight): Proximity-Weighted Average

125 × 0.1 + 180 × 0.1 + 210 × 0.8 = 198.5

Weighted average by proximity



Weights 0.1 / 0.1 / 0.8 → 198.5



Repeat This Process Until It Converges

Rebuild the RF → recompute proximities → re-estimate the missing values

Iteration

- Usually after 6–7 rounds the values stop changing
- Breiman's original RF implementation (R: rfImpute) works this way
- A similar idea: missForest (2012) — iterative imputation with an RF per column

Filled-in Missing Values

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	Yes	**NO**	**198.5**	No

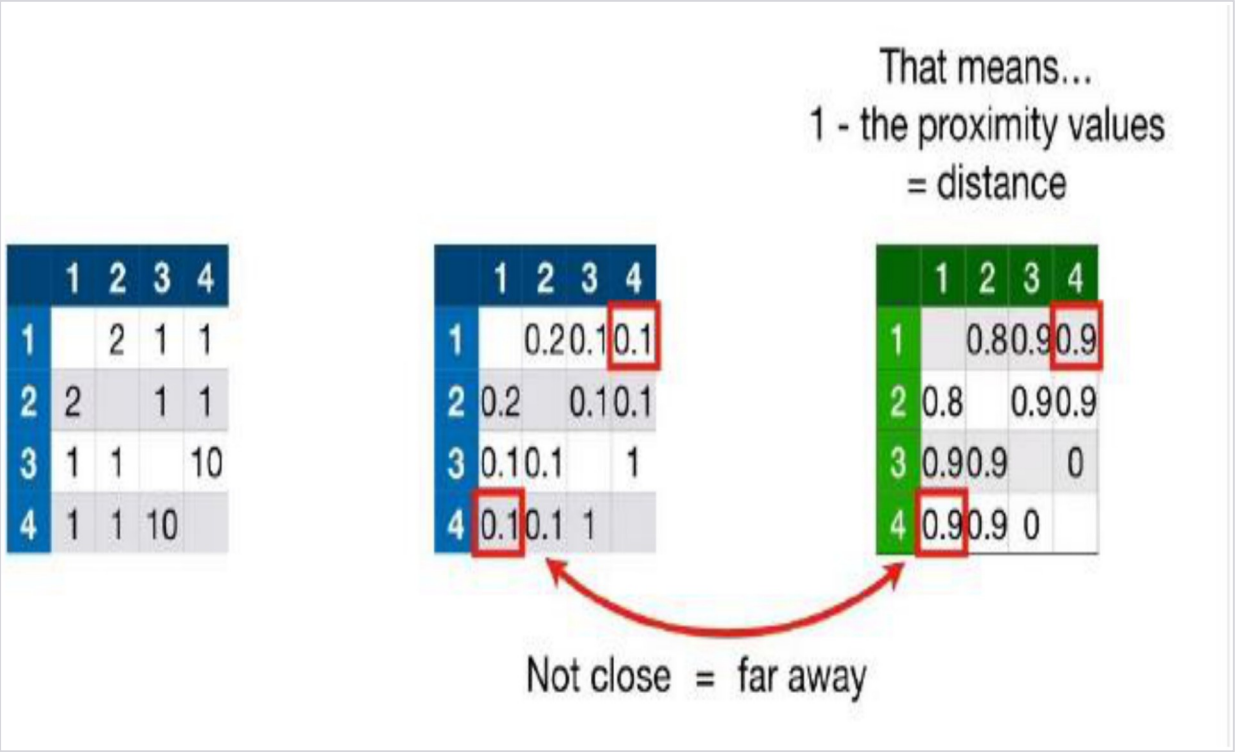
Now that we've revised our guesses a little bit, we do the whole thing over again...

We build a random forest, run the data through the trees, recalculate the proximities and recalculate the missing values.

We do this 6 or 7 times until the missing values converge (i.e. no longer change each time we recalculate).

Bonus — 1 − Proximity = Distance

Pairs with low proximity are far apart

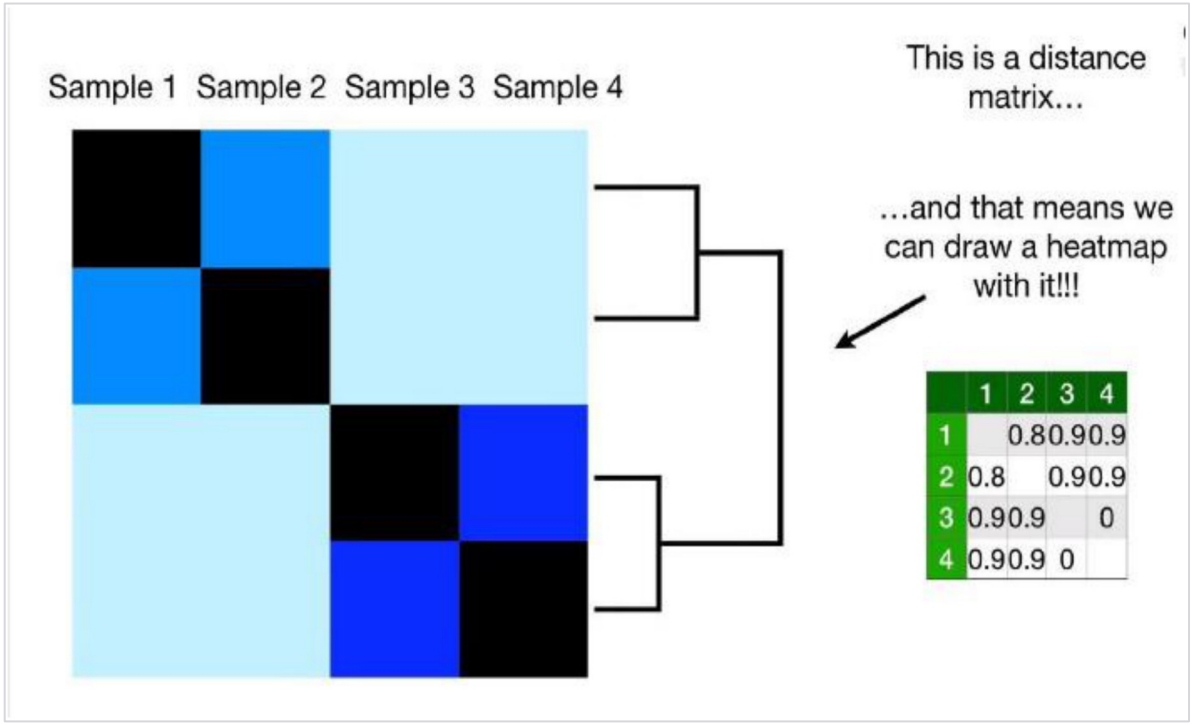


$$d(i,j) = 1 - \text{prox}(i,j)$$

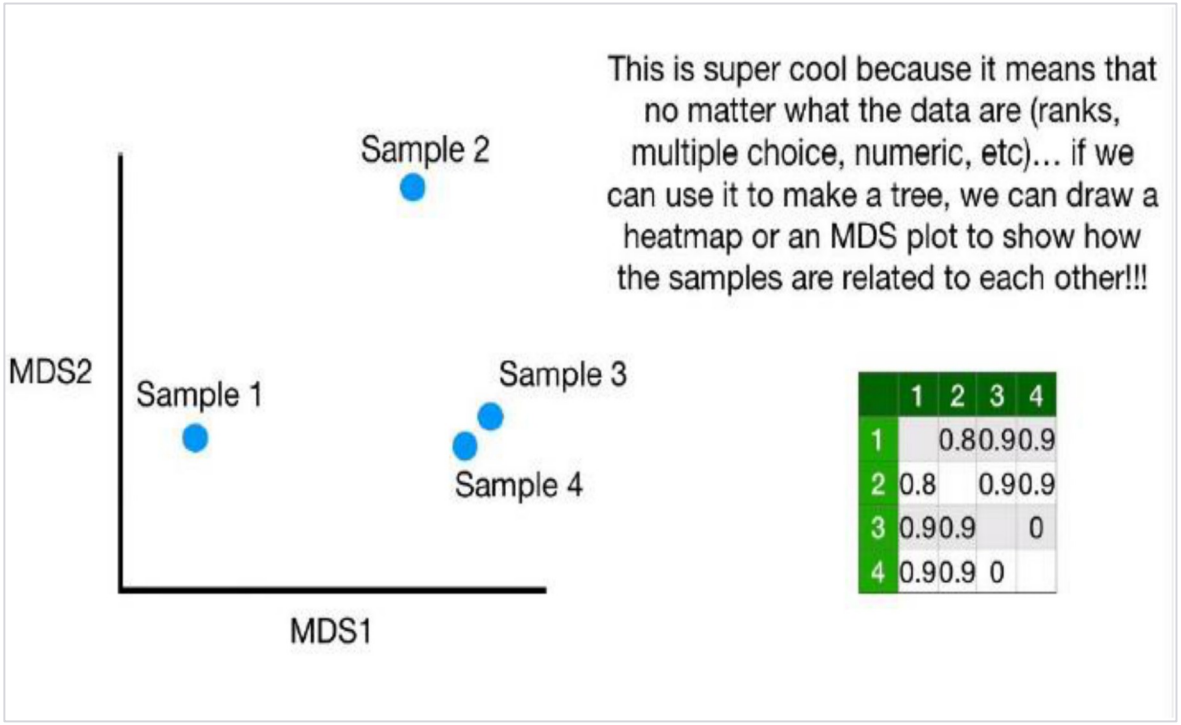
Draw heatmaps · MDS plots from the distance matrix

Whether the data are ranks, categories, or numbers, if you can build a tree you can visualize how samples relate

Distance-matrix heatmap + hierarchical clustering



MDS plane: samples 3 · 4 are close

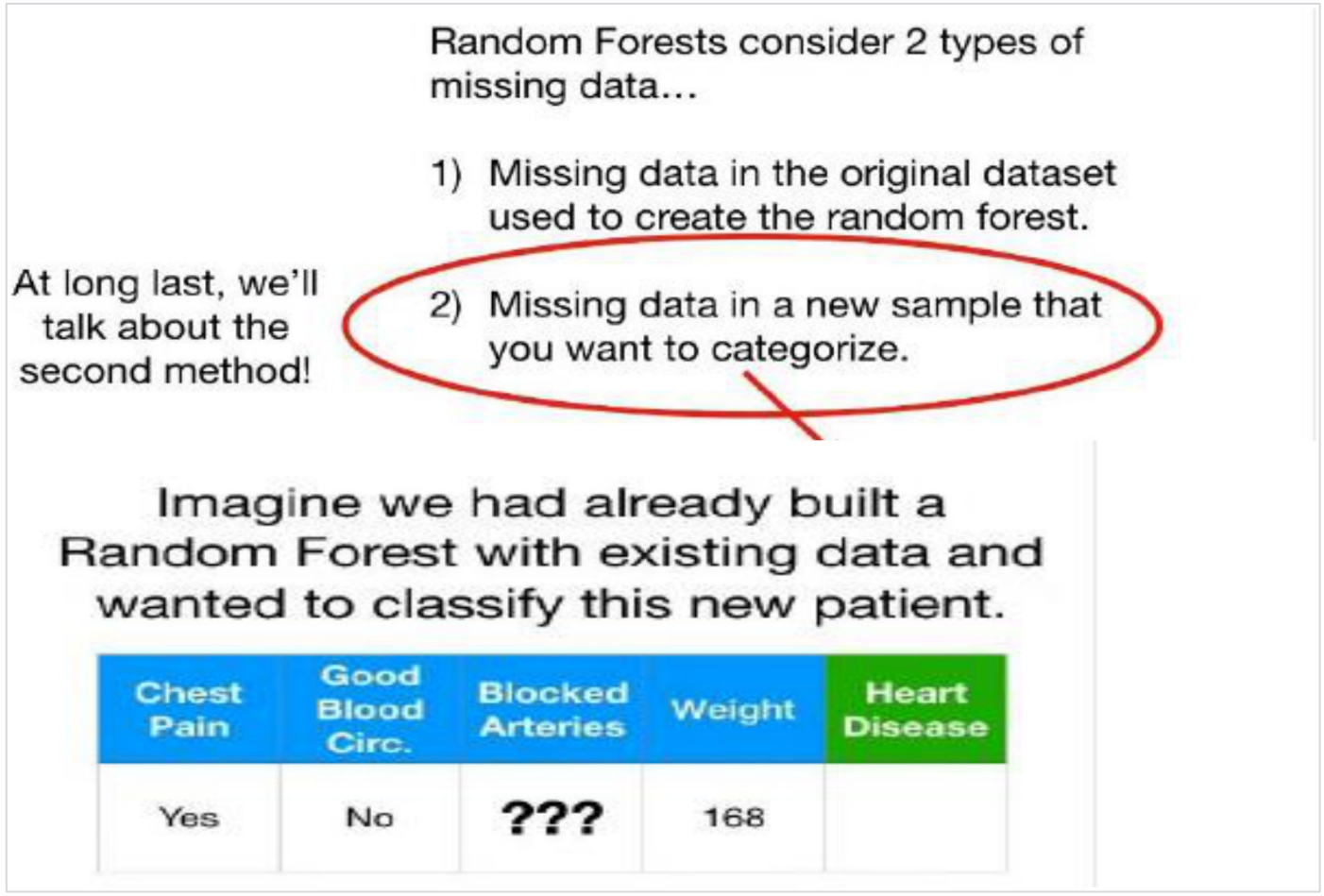


② When a New Sample Has Missing Values

We want to classify a new patient whose BA is "???" with an RF already built

Situation

- The RF has already been trained on existing data
- New patient: CP Yes · BC No · BA ??? · weight 168
- The target (heart disease) is unknown too

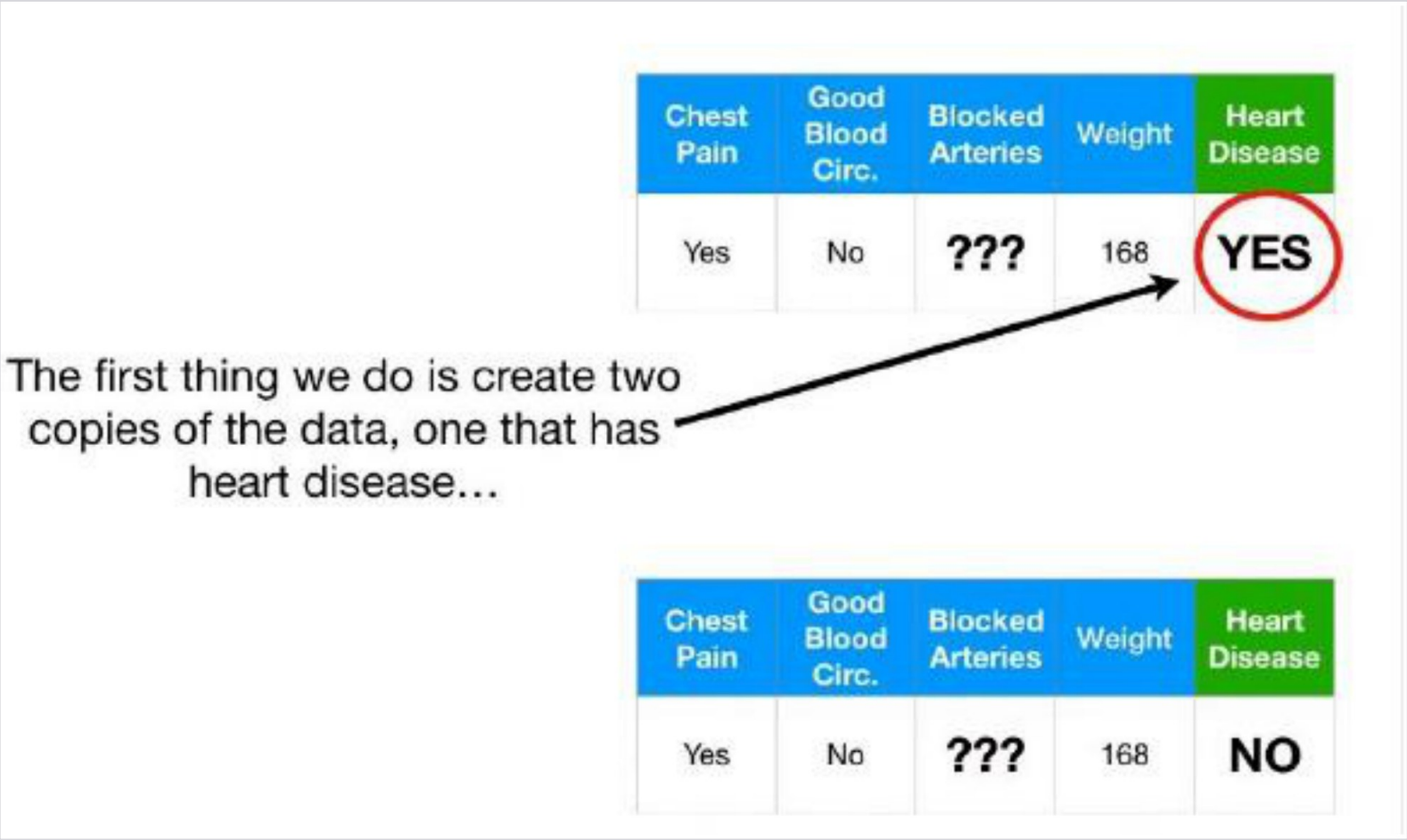


1. Make a Copy for Each Target Candidate

A copy with heart disease = Yes and a copy with heart disease = No

Procedure

- One copy per possible label
- Estimate the missing BA separately in each copy



2. Estimate the Missing Values in Each Copy

Estimate with the iterative method above — with heart disease Yes, BA is likely Yes too

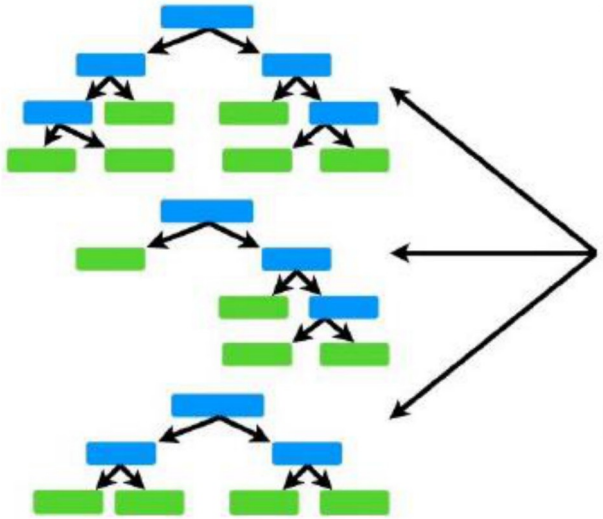
Estimate with the iterative method

Then we use the iterative method we just talked about to make a good guess about the missing values.

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	???	168	YES

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	???	168	NO

Yes copy → BA Yes, No copy → BA No



Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	YES	168	YES

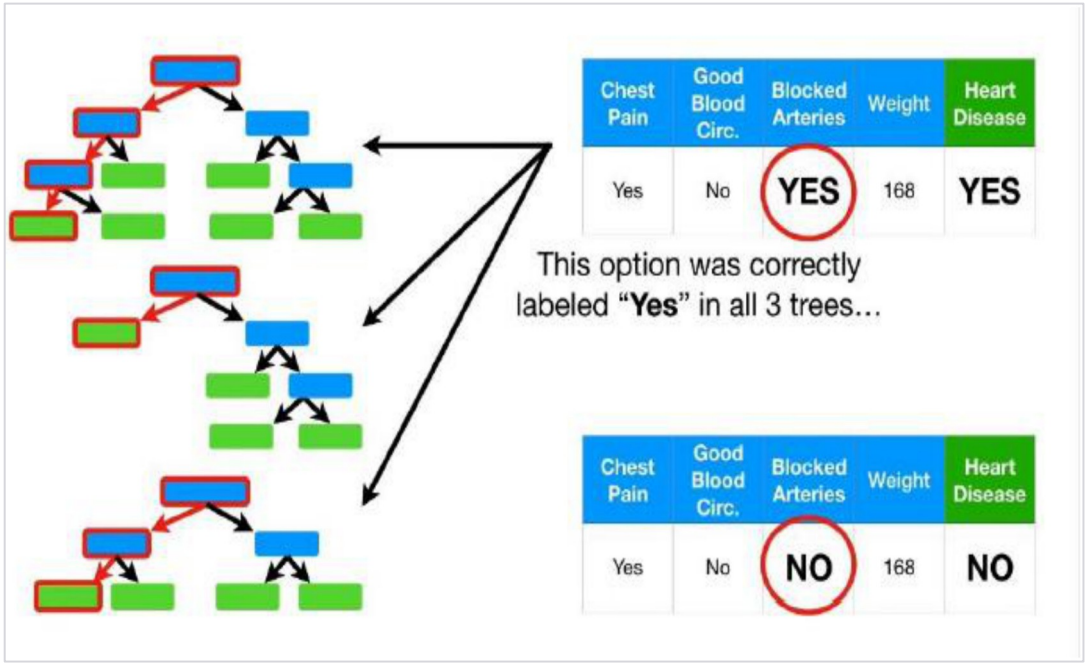
Then we run the two samples down the trees in the forest…

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	NO	168	NO

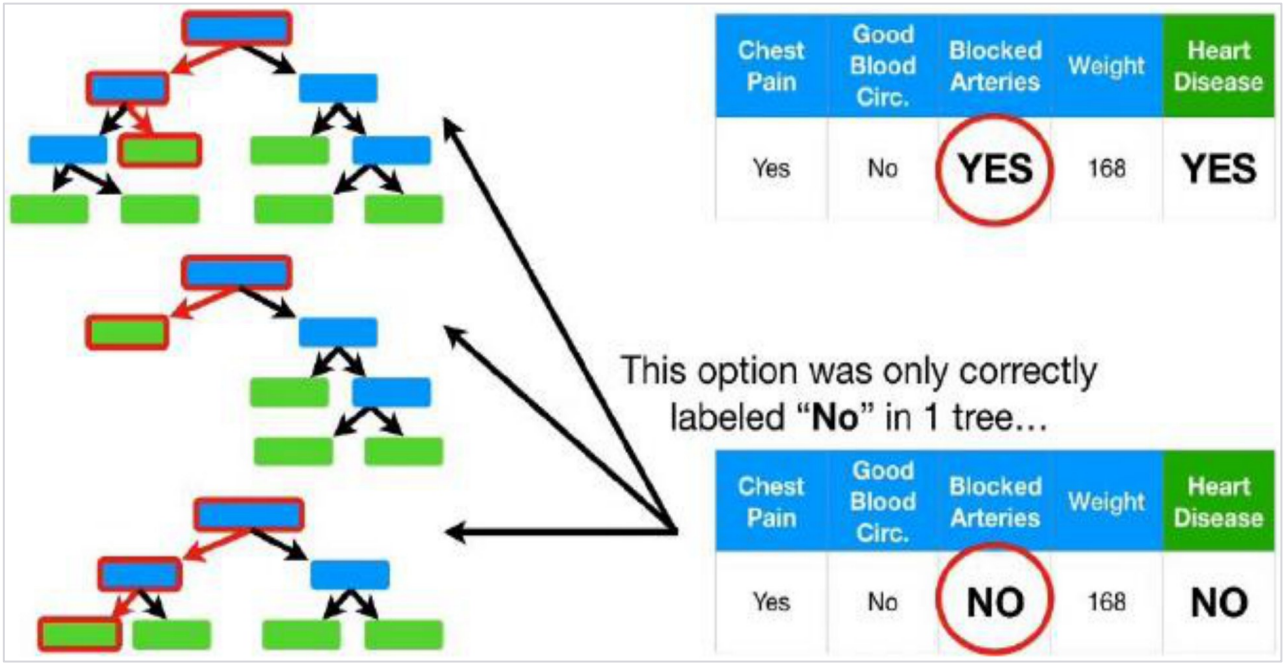
3. Run Both Copies Through All Trees and Compare

Which copy is more often classified "as its own label"?

Yes copy: Yes in all 3 trees



No copy: No in only 1 tree

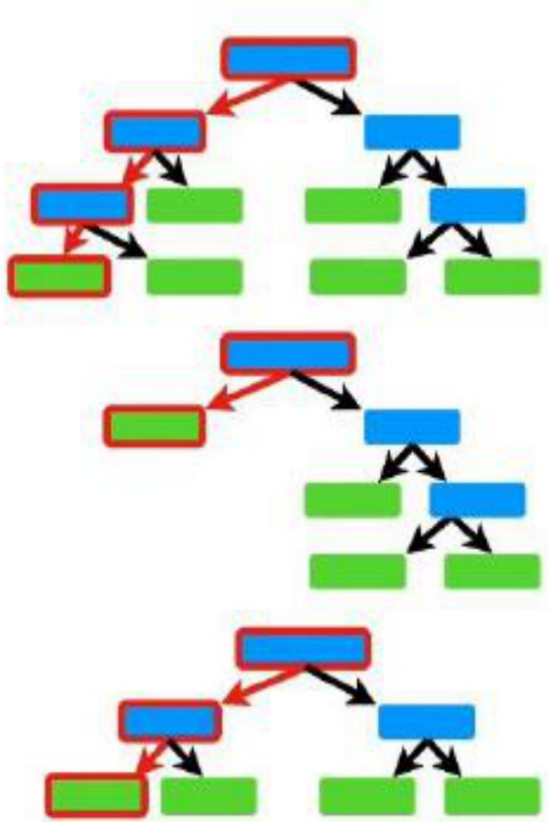


The Copy That Is Right More Often Wins

New patient: concluded heart disease Yes, BA Yes

Conclusion

- The Yes copy was classified correctly more often
- → adopt this option
- Filling the missing value and classifying are done at the same time



Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	YES	168	YES

This option wins because it was correctly labeled more than the other option.

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	No	NO	168	NO

Supplement — How Today's Libraries Handle Missing Values

scikit-learn tree models accept NaN and learn, for each split, which side (left/right) missing values go to

Model	Missing-value handling	Introduced
DecisionTree (scikit-learn)	Learns the NaN direction per split by gain	1.3
RandomForest (scikit-learn)	Same per tree; features with no missing values in training send NaN to the side with more samples	1.4
ExtraTrees (scikit-learn)	Added missing-value support	1.6
HistGradientBoosting	Native NaN support from the start	—
XGBoost · LightGBM	Learns a default direction — sparsity-aware split	—

Practical caveats

- Impute within cross-validation folds to avoid leakage
- Adding a missingness indicator is also common

Supplement — Injecting 20% Missing Values

Randomly set 20% of the breast cancer train/test features to NaN and compare (scikit-learn 1.8)

Python · scikit-learn

```
# X_train_m, X_test_m: 20% are NaN
rf  = RandomForestClassifier(random_state=0)
rf.fit(X_train_m, y_train)            # NaN as is

imp = make_pipeline(SimpleImputer(strategy="median"),
                    RandomForestClassifier(random_state=0))
imp.fit(X_train_m, y_train)

hgb = HistGradientBoostingClassifier(random_state=0)
hgb.fit(X_train_m, y_train)
```

Method	Test accuracy
RF (native NaN support)	0.951
Median imputation + RF	0.951
HistGB (native NaN support)	0.972

How to read it

- Under random missingness (MCAR), the methods differ little
- With systematic missingness, results can differ

04

Part 4 · AdaBoost

AdaBoost — Concentrating Weight on Misclassified Samples

Weak learners (stumps) · weighted error ε · amount of say α · weight update · final weighted vote

Background — Boosting and Weak Learners

Weak learner: a model only slightly better than random guessing. AdaBoost's default is a depth-1 tree (stump)

① Start with equal weights

Every sample has weight 1/N. Train the first stump.

② Raise weights of misclassified samples

Misclassified samples get higher weights; correctly classified ones get lower.

③ The next stump focuses on hard samples

It learns to get the high-weight samples right.

④ Weighted vote

The more accurate a stump, the larger its say α in the final decision.

RF trees are independent, each with an equal vote / AdaBoost stumps are sequential, with a say based on performance

Supplement — The AdaBoost Algorithm (Binary Classification, y ∈ {−1, +1})

Formulas for where the ε and α in the original figures come from

$$\varepsilon_t = \sum_{i=1}^{N} w_i^{(t)}\, \mathbf{1}\{h_t(x_i) \neq y_i\}$$

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$$

$$w_i^{(t+1)} = \frac{w_i^{(t)}\, \exp\!\big(-\alpha_t\, y_i\, h_t(x_i)\big)}{Z_t}$$

$$H(x) = \operatorname{sign}\!\Big(\sum_{t=1}^{T} \alpha_t\, h_t(x)\Big)$$

Weighted error → amount of say

- The smaller ε, the larger α (α = 0 when ε = 0.5)
- Misclassified weights ↑, correct ones ↓ → normalize so they sum to 1

Final classifier

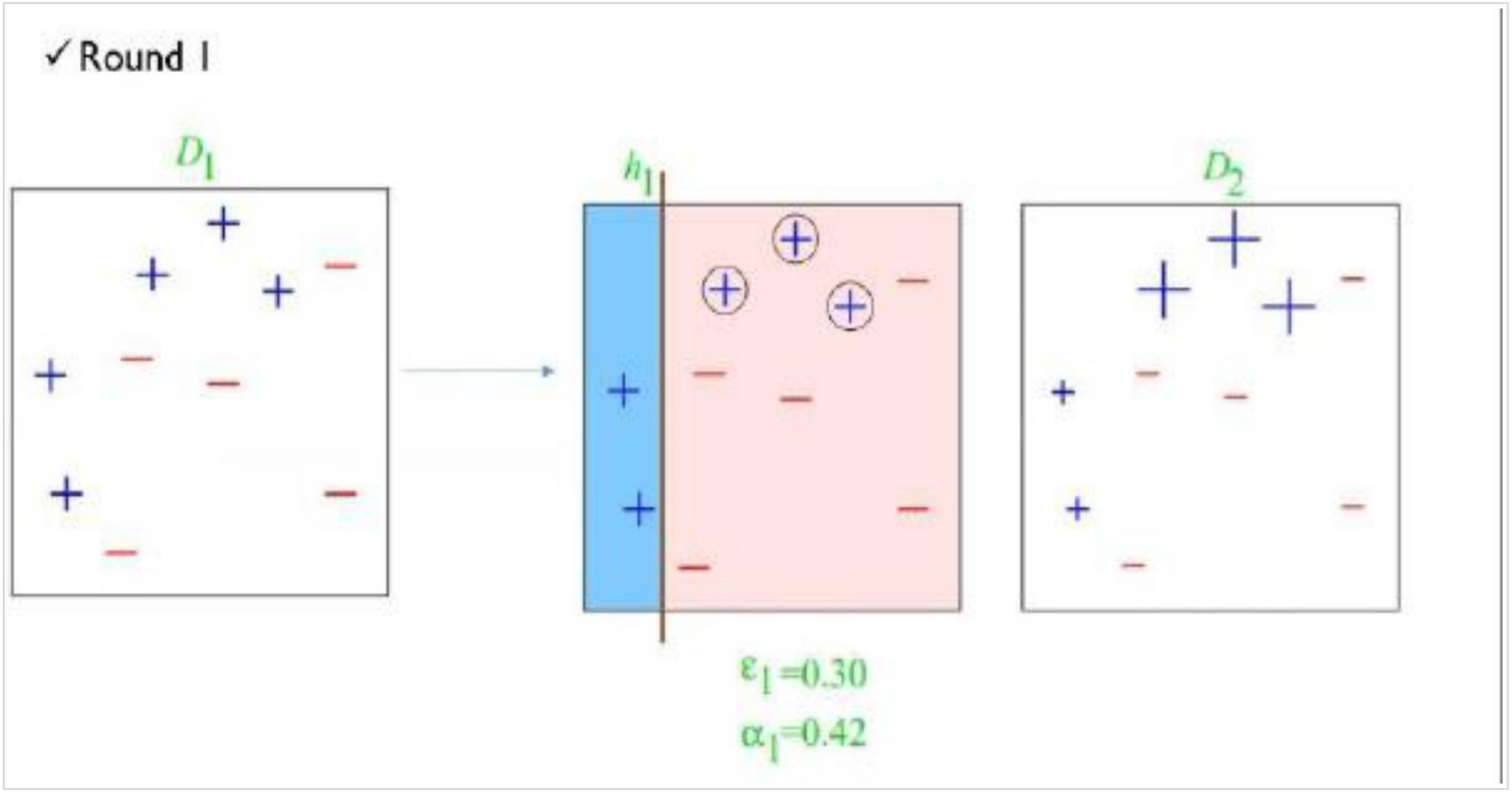
- Sum stump predictions (±1) × α and take the sign
- Freund · Schapire (1997), 2003 Gödel Prize

Example 1 — Round 1

D_1: all 10 samples have the same weight → the first stump h_1 is a single vertical line

How to read it

- h_1 misclassified three + (circled)
- $\varepsilon_1 = 0.30 \rightarrow \alpha_1 = 0.42$
- D_2: the three misclassified + are drawn large = higher weight

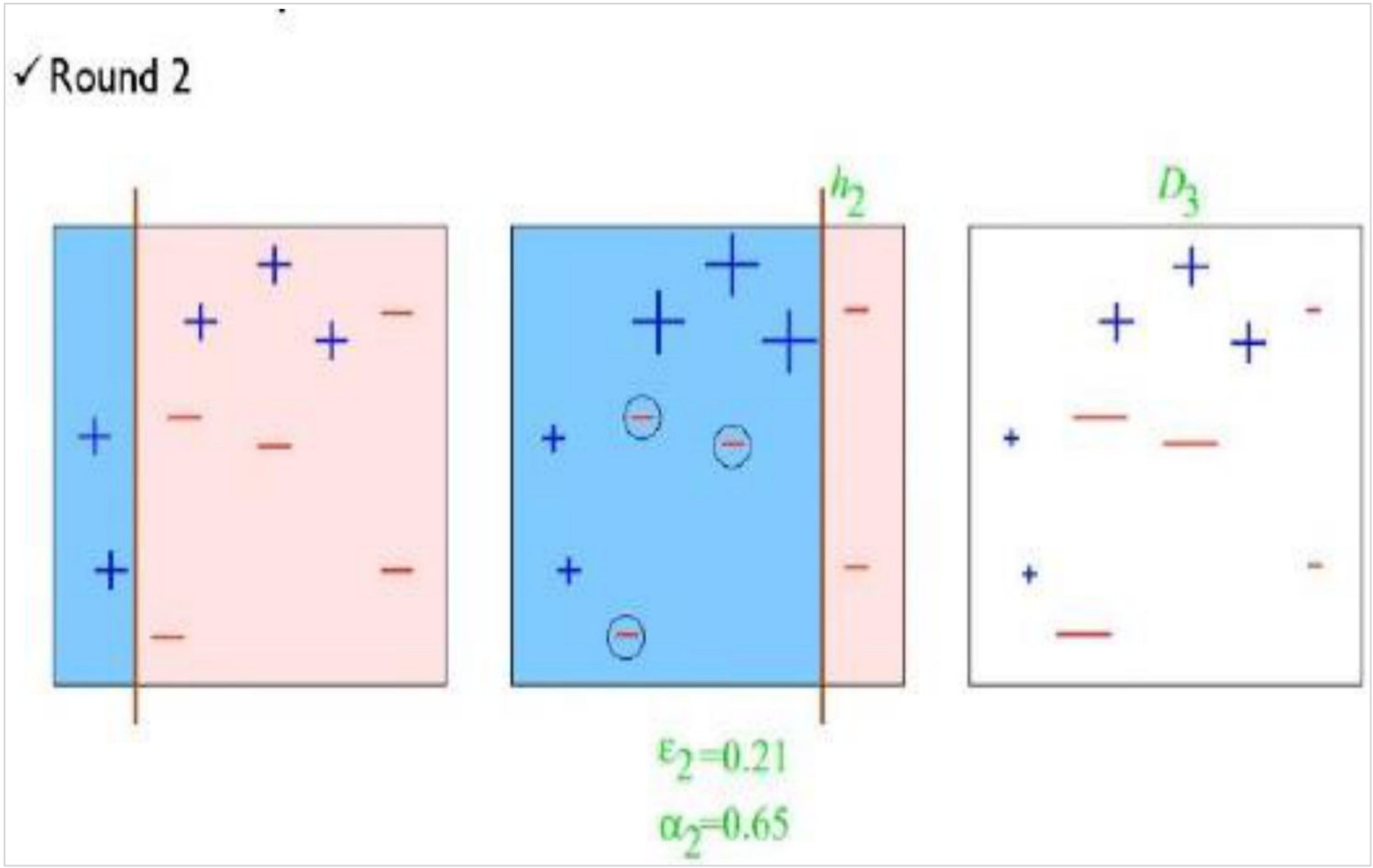


Example 1 — Round 2

To get the up-weighted + right, the second stump h_2 draws a vertical line on the right

How to read it

- h_2 misclassified three − (circled)
- $\varepsilon_2 = 0.21 \rightarrow \alpha_2 = 0.65$
- D_3: this time those three − get higher weight

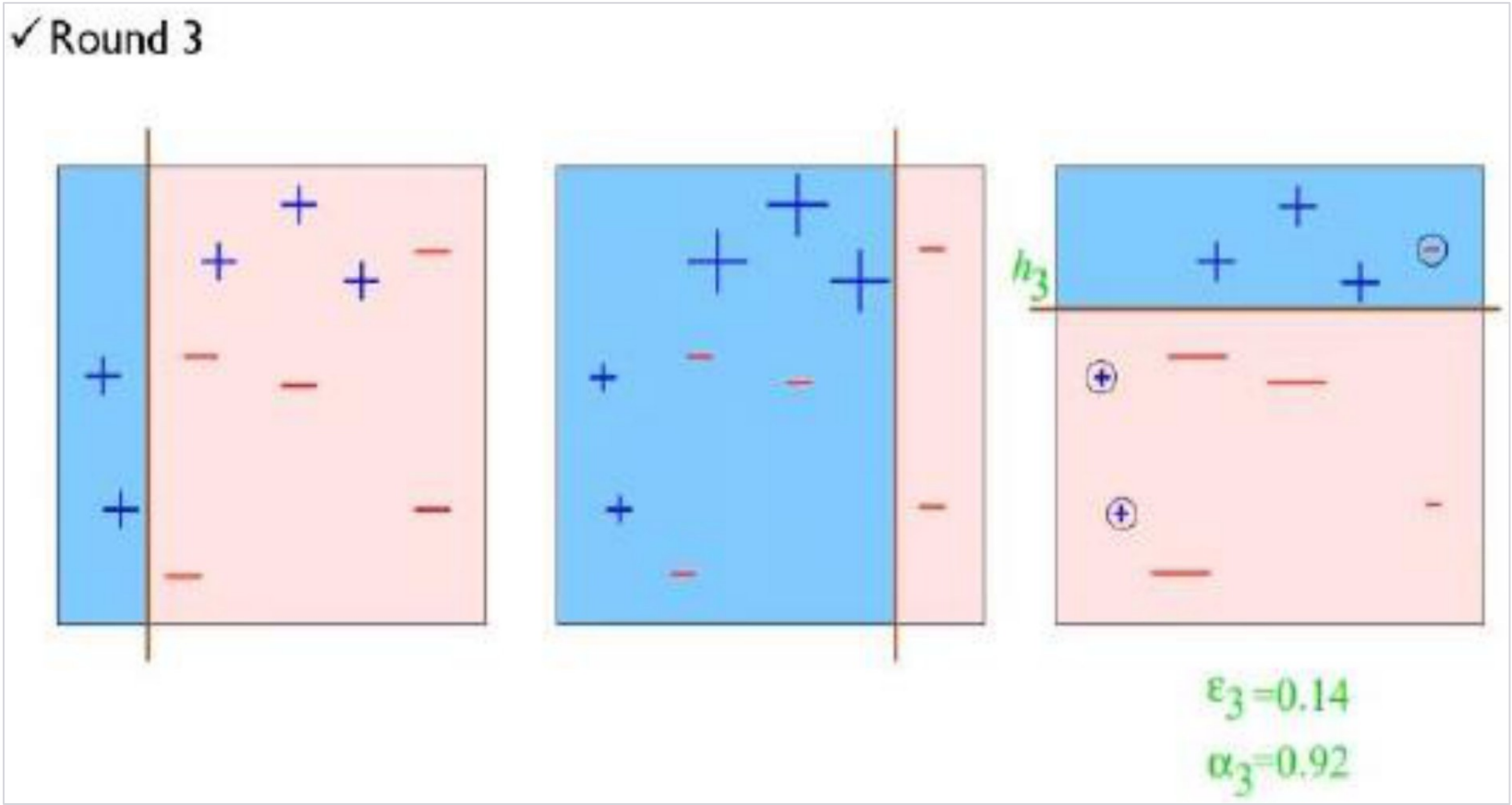


Example 1 — Round 3

The third stump h_3 is a horizontal line — aimed at the region the first two stumps missed

How to read it

- $\varepsilon_3 = 0.14 \rightarrow \alpha_3 = 0.92$
- Over the rounds, ε fell and α rose
- Each stump is weak on its own

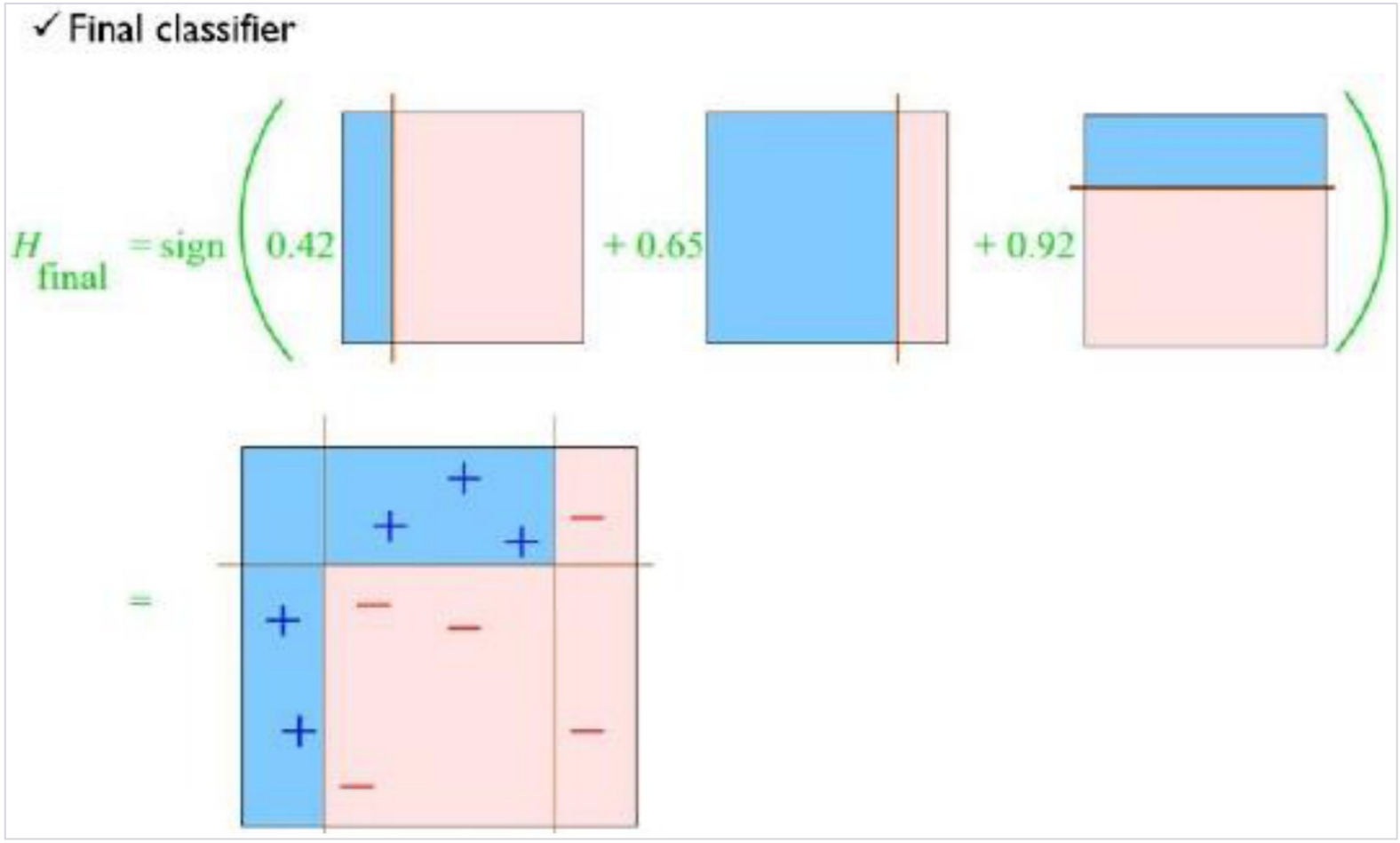


Example 1 — Final Classifier

$H_{final} = \text{sign}(0.42 \cdot h_1 + 0.65 \cdot h_2 + 0.92 \cdot h_3)$

Result

- The +/− that no single line could separate
- are all classified correctly by the staircase boundary of the three stumps' weighted sum
- Combining weak learners → a strong learner



Supplement — Verifying the α of Example 1

Tracking the weights exactly gives $\varepsilon_1 = 3/10$, $\varepsilon_2 = 3/14$, $\varepsilon_3 = 3/22$

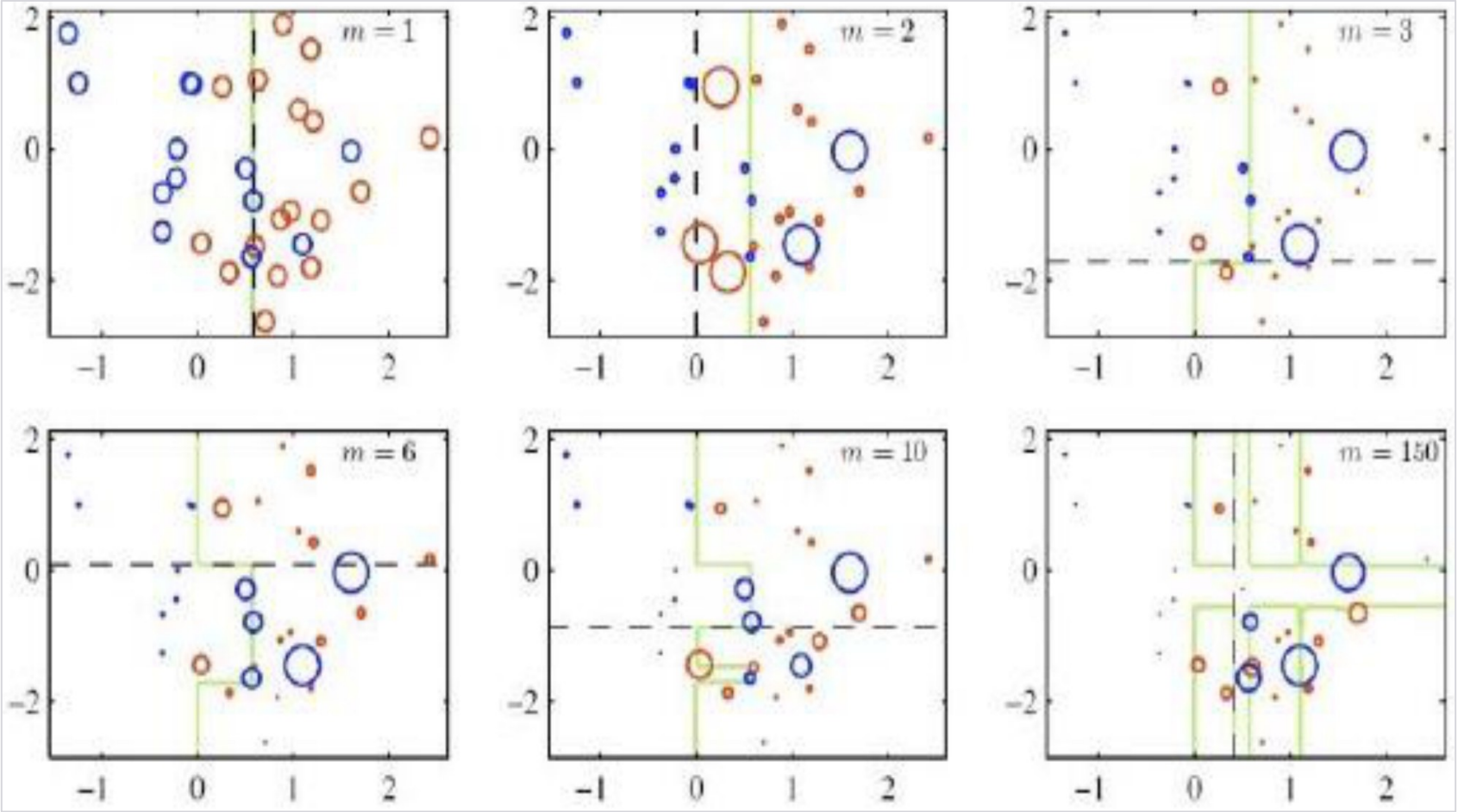
$$\alpha_1 = \tfrac{1}{2}\ln\tfrac{7/10}{3/10} = 0.42 \qquad \alpha_2 = \tfrac{1}{2}\ln\tfrac{11/14}{3/14} = 0.65 \qquad \alpha_3 = \tfrac{1}{2}\ln\tfrac{19/22}{3/22} = 0.92$$

Round	Misclassified samples	Sum of their weights ε	α = ½ ln((1−ε)/ε)
1	3 × + (1/10 each)	3/10 = 0.300	0.42
2	3 × − (1/14 each)	3/14 = 0.214	0.65
3	3 (1/22 each)	3/22 = 0.136	0.92

Normalization after round 1: the 3 misclassified sum to 1/2 (1/6 each), the 7 correct sum to 1/2 (1/14 each). The 0.21 · 0.14 in the original lecture are 3/14 · 3/22 rounded.

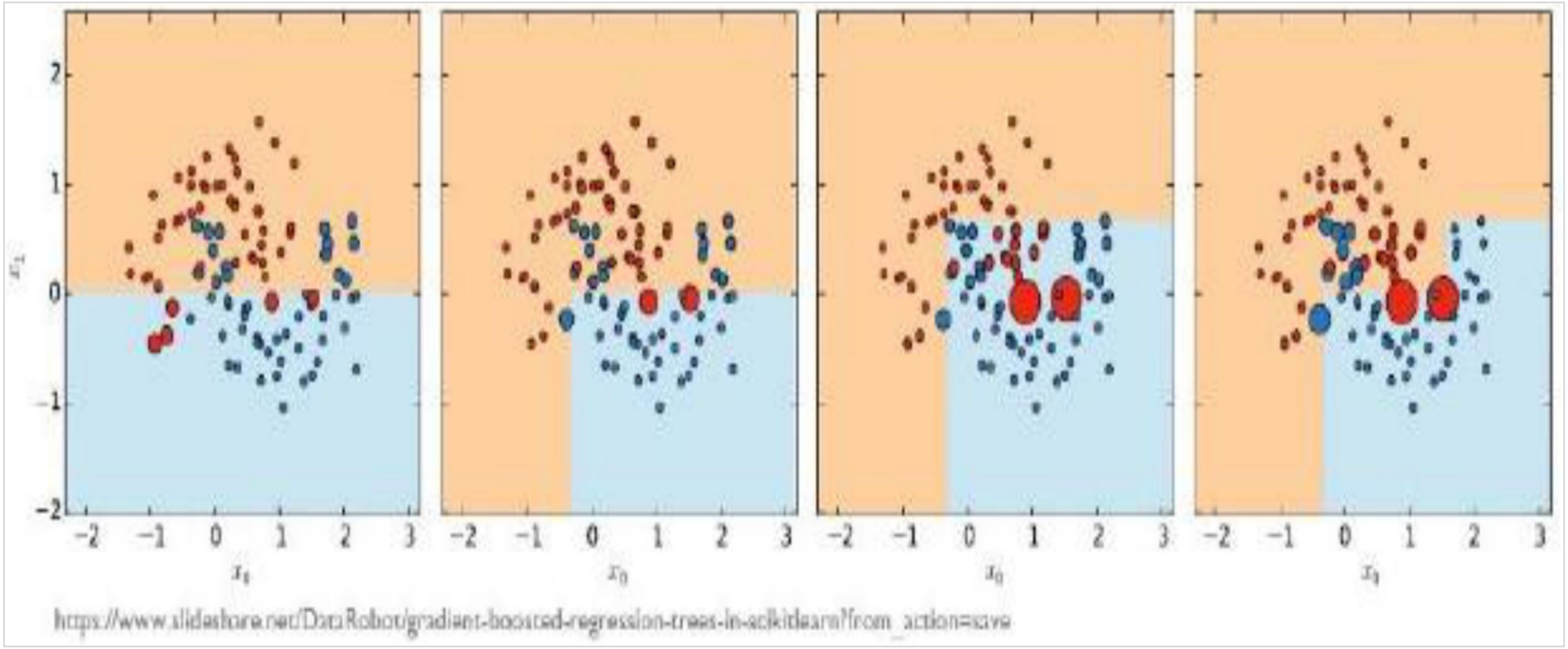
Example 2 — The Boundary Becomes Finer over the Rounds

m = 1, 2, 3, 6, 10, 150: circle size = sample weight, dashed line = current stump, green line = cumulative boundary



Example 3 — Misclassified Samples Grow Larger

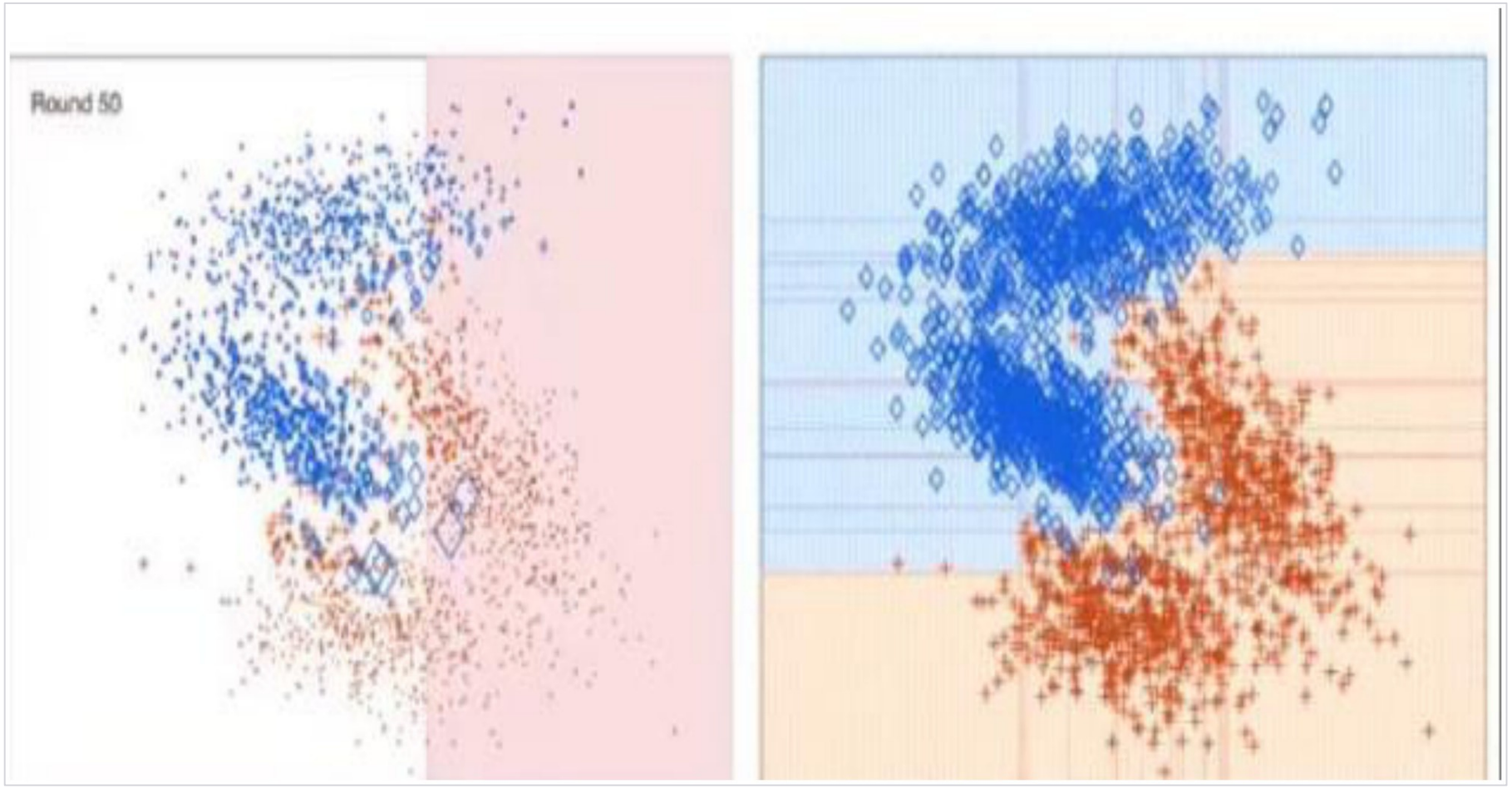
With each iteration, weight concentrates on misclassified samples near the boundary (large dots)



This is also why boosting is vulnerable to outliers and label noise — weight keeps piling onto noisy samples.

Example 4 — After 50 Rounds

Left: boundary of the 50th stump alone / Right: boundary of the cumulative ensemble

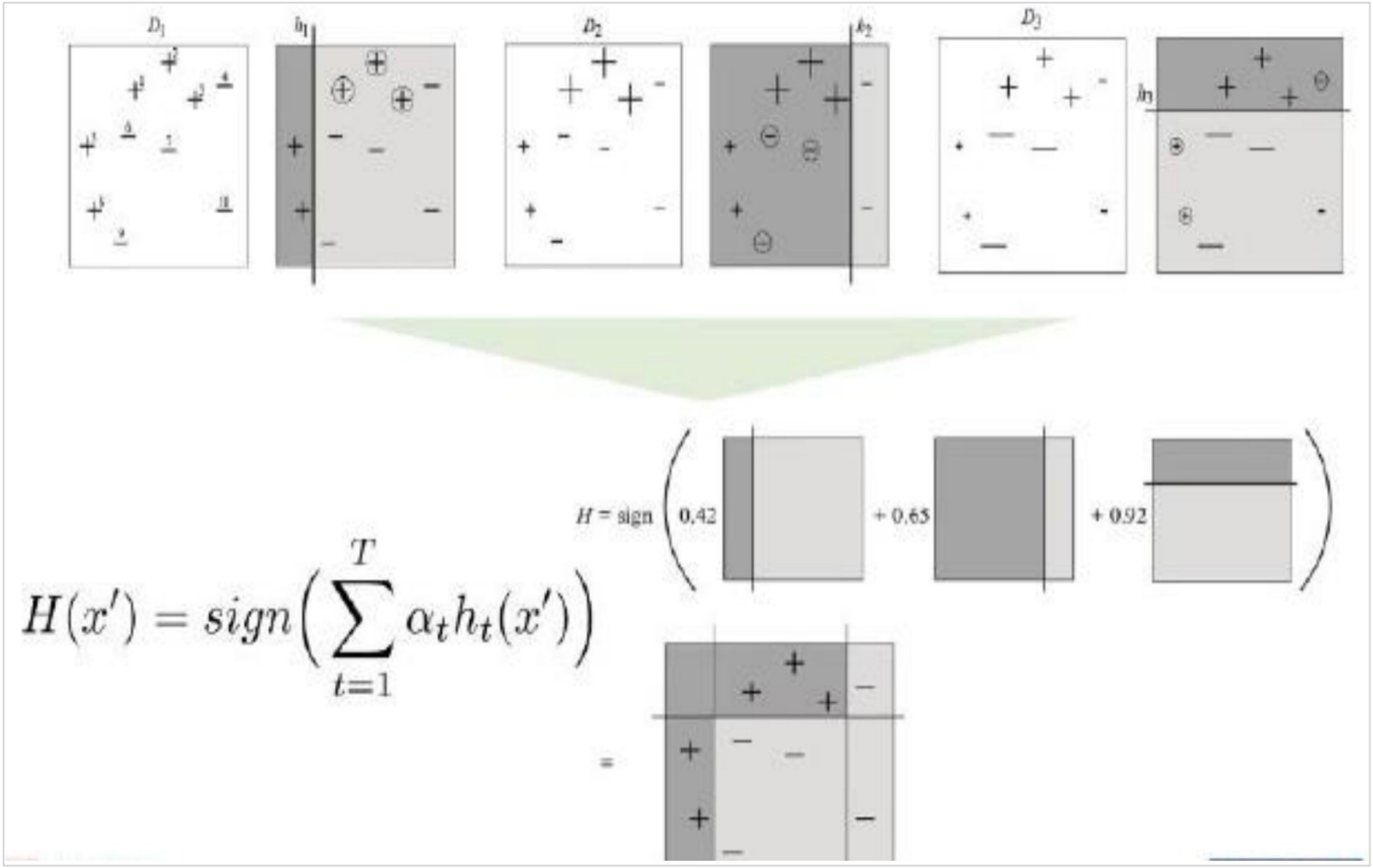


Example 5 — The Whole Process at a Glance

Weighted resampling ($D_1 \rightarrow D_2 \rightarrow D_3$) and the weighted vote on one slide

In one line

- Reweight every round
- Give each stump a say α
- Final: sign of the α-weighted vote



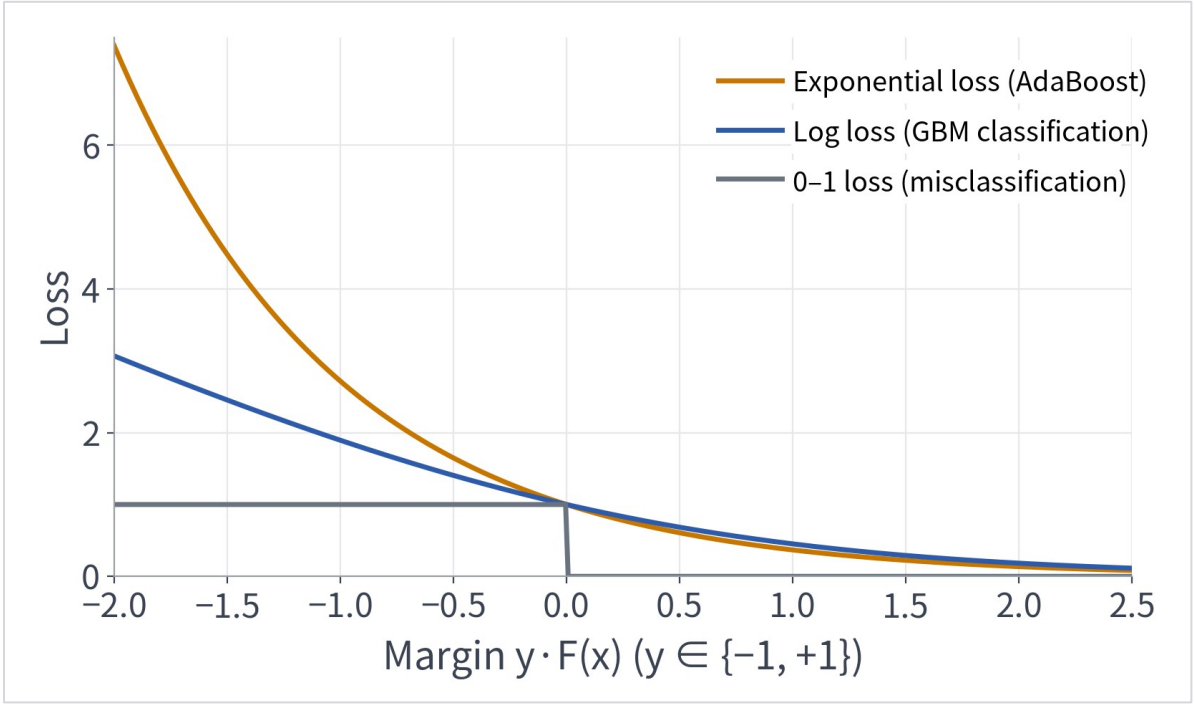
Supplement — AdaBoost Is an Additive Model That Reduces Exponential Loss

Friedman · Hastie · Tibshirani (2000): AdaBoost = forward stagewise additive model of exponential loss → the bridge to GBM

$$L\big(y, F(x)\big) = e^{-yF(x)}, \qquad F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$$

Why it matters

- AdaBoost's weights = magnitude of the exponential-loss gradient
- "What if we change the loss?" → Part 5 gradient boosting
- Exponential loss penalizes badly misclassified samples explosively → sensitive to noise



Supplement — scikit-learn AdaBoostClassifier

Breast cancer data, same split (scikit-learn 1.8 output)

Python · scikit-learn

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(n_estimators=100,
                         random_state=0)
ada.fit(X_train, y_train)
ada.score(X_test, y_test)    # 0.965
```

Defaults

- estimator: depth-1 tree (stump)
- n_estimators = 50
- learning_rate = 1.0 (shrinkage factor multiplied into α)

Version note

- The old algorithm="SAMME.R" option has been removed; only SAMME remains
- When porting old code, delete the algorithm argument

Episode — AdaBoost Finds Faces in Cameras

Viola · Jones (2001): a real-time face detector that chains simple features chosen by AdaBoost into a cascade

What they did

- AdaBoost picked useful ones from hundreds of thousands of simple rectangular (Haar-like) features
- Easy background is discarded early; only hard candidates pass to later stages
- Real-time detection on PCs of the time → the basis of face detection in digital cameras · phones

Link to this lecture

- Exactly today's structure: "many weak rules + weighted vote"
- Weak learner = a stump built on one rectangular feature
- A classic case of theory (1995) becoming an industry standard (2000s)

05

Part 5 · Gradient Boosting

Gradient Boosting — Fit the Next Tree to the Residuals

Residual learning · scikit-learn implementation · weight example step by step · learning rate · Friedman's algorithm

How Does It Differ from AdaBoost?

Instead of reweighting samples, it trains a new predictor on the residual errors left by the previous predictor

	AdaBoost	Gradient Boosting
What the next model sees	The same data with changed weights	The previous model's residuals (negative gradient)
Base learner	stump (depth 1)	Shallow regression tree (even for classification)
Combination	Weighted vote by α	Add after multiplying by the learning rate ν
Loss function	Fixed exponential loss	Any differentiable loss (squared · log · Huber)

Friedman (2001): gradient descent in function space — "A Gradient Boosting Machine"

Stacking Trees on Residuals in Sequence

Chaining three DecisionTreeRegressor(max_depth=2) by hand

Flow of the code

- tree_reg1: fit to the original y
- y2 = y − tree_reg1 prediction → fit tree_reg2
- y3 = y2 − tree_reg2 prediction → fit tree_reg3
- Prediction = sum of the three trees' predictions

```
from sklearn.tree import DecisionTreeRegressor

tree_reg1 = DecisionTreeRegressor(max_depth=2)
tree_reg1.fit(X, y)
```

- Train a second decision tree on the residual errors left by the first predictor.

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2)
tree_reg2.fit(X, y2)
```

- Train a third regression model on the residual errors left by the second predictor.

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2)
tree_reg3.fit(X, y3)
```

Individual Predictors vs the Ensemble Predictor

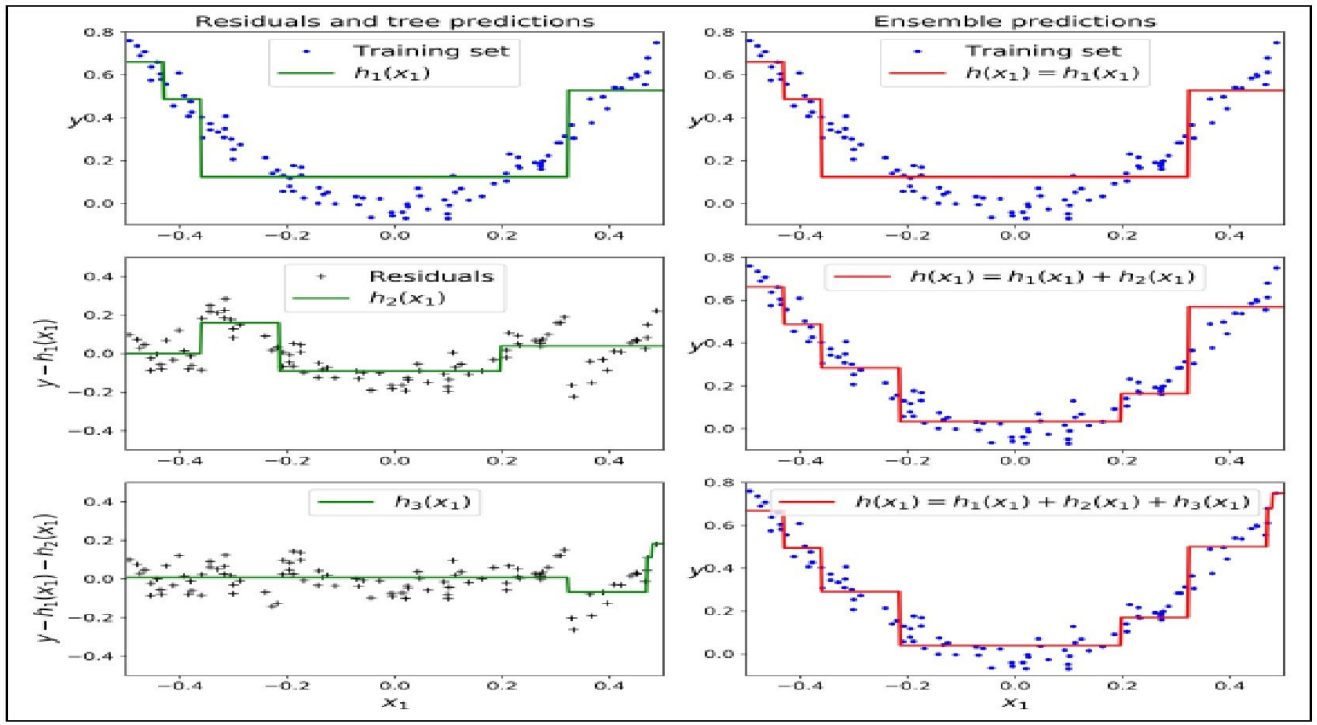
Left: residuals each tree fits / Right: cumulative ensemble $h_1 \rightarrow h_1+h_2 \rightarrow h_1+h_2+h_3$

How to read it

- y_pred = sum(tree.predict(X_new) for tree in trees)
- As trees are added, the ensemble moves closer to the data
- The remaining residuals (bottom left) flatten out like noise

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

- Comparing the individual predictors with the ensemble predictor



scikit-learn GradientBoostingRegressor

The manual process above = max_depth=2, n_estimators=3, learning_rate=1.0

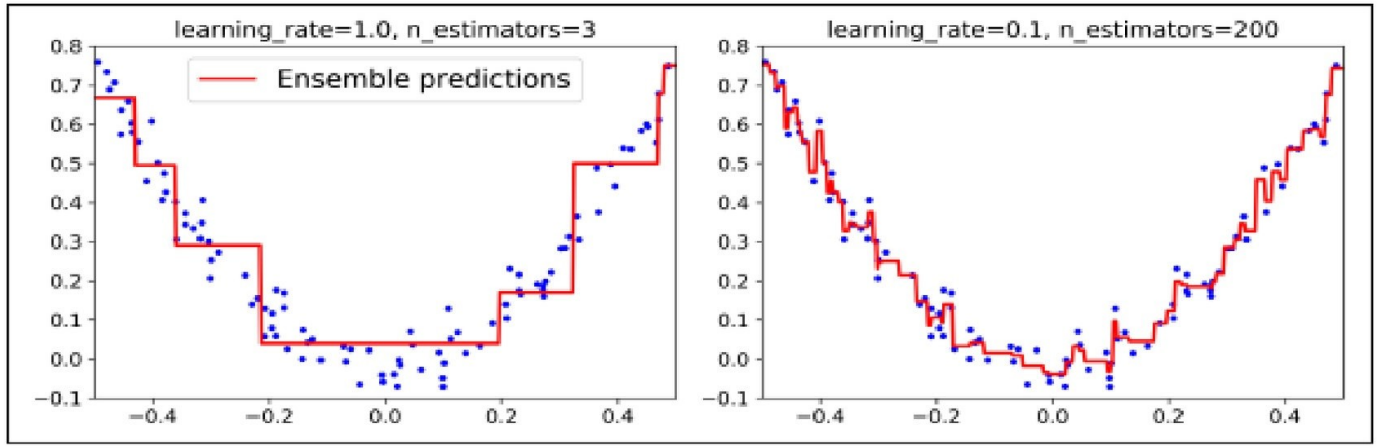
Learning Rate and Number of Trees

- Lowering learning_rate to, say, 0.1 requires many trees
- In exchange, predictive performance (generalization) improves — shrinkage
- Left: too few trees / Right: a smooth fit with 200 trees

```python
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0)
gbrt.fit(X, y)
```

- Setting the learning rate parameter low, e.g. 0.1, requires many trees for training, but predictive performance improves.

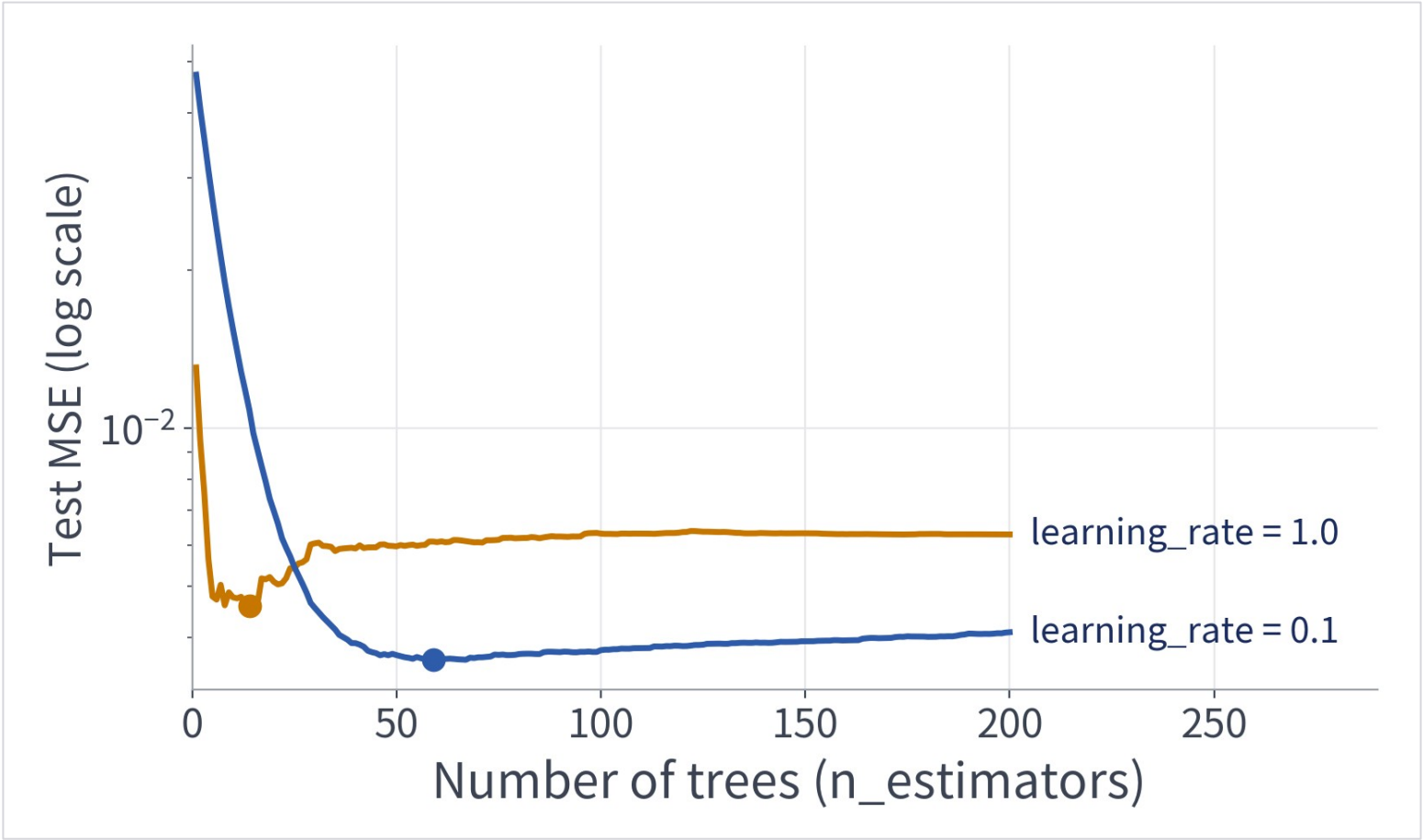


Supplement — Learning Rate 1.0 vs 0.1, Checked by Test Error

Same quadratic data ($y = 3x^2$ + noise), max_depth = 2 (reproduced in scikit-learn 1.8)

How to read it

- 1.0: lowest at 14 trees, then gets worse
- 0.1: lowest at 59 trees, and the minimum error is lower too
- Dot = minimum → find it with early stopping



GradientBoostingClassifier — Handling Overfitting

Breast cancer data: defaults → limit depth → lower learning rate

Result

- Default (n_estimators=100, max_depth=3): train 1.000 · test 0.958 → overfitting
- max_depth=1: 0.991 · 0.972
- learning_rate=0.01: 0.988 · 0.965

```
In [73]: from sklearn.ensemble import GradientBoostingClassifier

         X_train, X_test, y_train, y_test = train_test_split(
             cancer.data, cancer.target, random_state=0)

         gbrt = GradientBoostingClassifier(random_state=0)
         gbrt.fit(X_train, y_train)

         print("Train accuracy: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("Test accuracy: {:.3f}".format(gbrt.score(X_test, y_test)))

Train accuracy: 1.000
Test accuracy: 0.958

In [74]: gbrt = GradientBoostingClassifier(random_state=0, max_depth=1)
         gbrt.fit(X_train, y_train)

         print("Train accuracy: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("Test accuracy: {:.3f}".format(gbrt.score(X_test, y_test)))

Train accuracy: 0.991
Test accuracy: 0.972

In [75]: gbrt = GradientBoostingClassifier(random_state=0, learning_rate=0.01)
         gbrt.fit(X_train, y_train)

         print("Train accuracy: {:.3f}".format(gbrt.score(X_train, y_train)))
         print("Test accuracy: {:.3f}".format(gbrt.score(X_test, y_test)))

Train accuracy: 0.988
Test accuracy: 0.965
```

n_estimators=100, max_depth=3

Overfitting

Limit tree depth

Lower the learning rate

82

Weight Example — Predicting Weight from Height · Favorite Color · Gender

Data on 6 people (figures: from StatQuest lecture slides)

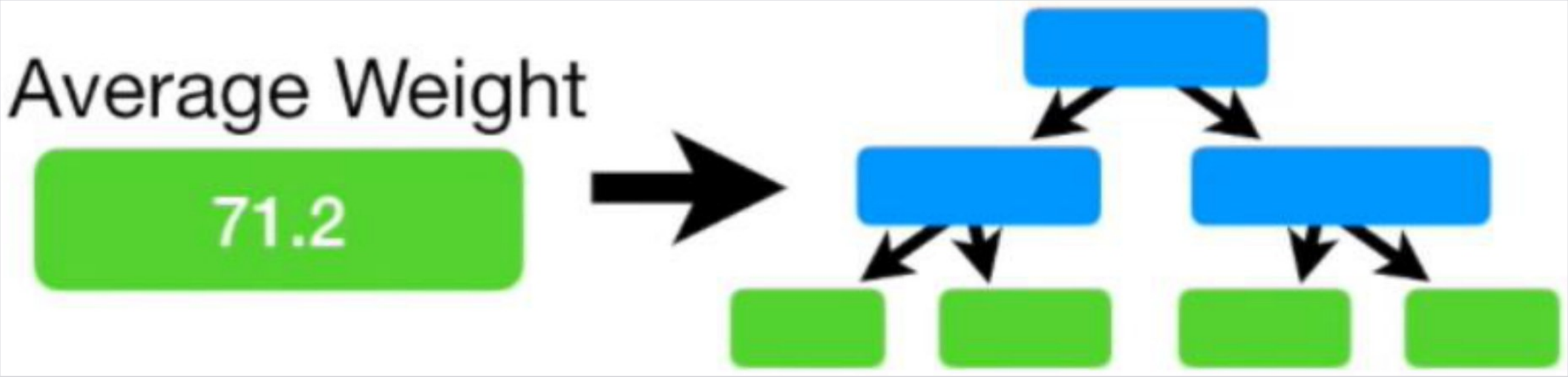
Task

- Target: weight (kg) — a regression problem
- Features: height (m), favorite color, gender
- Follow how GBM corrects the prediction one step at a time

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

Step 1 — Start with a Single Leaf (the Mean)

The first prediction is the average weight, 71.2 kg, for everyone



$$F_0 = \bar{y} = \frac{88 + 76 + 56 + 73 + 77 + 57}{6} = 71.2$$

Step 2 — Compute the Residuals

Residual = observed − predicted (71.2)

Residual

- 88 − 71.2 = 16.8
- 56 − 71.2 = −15.2
- The residual is exactly "what is not yet explained"

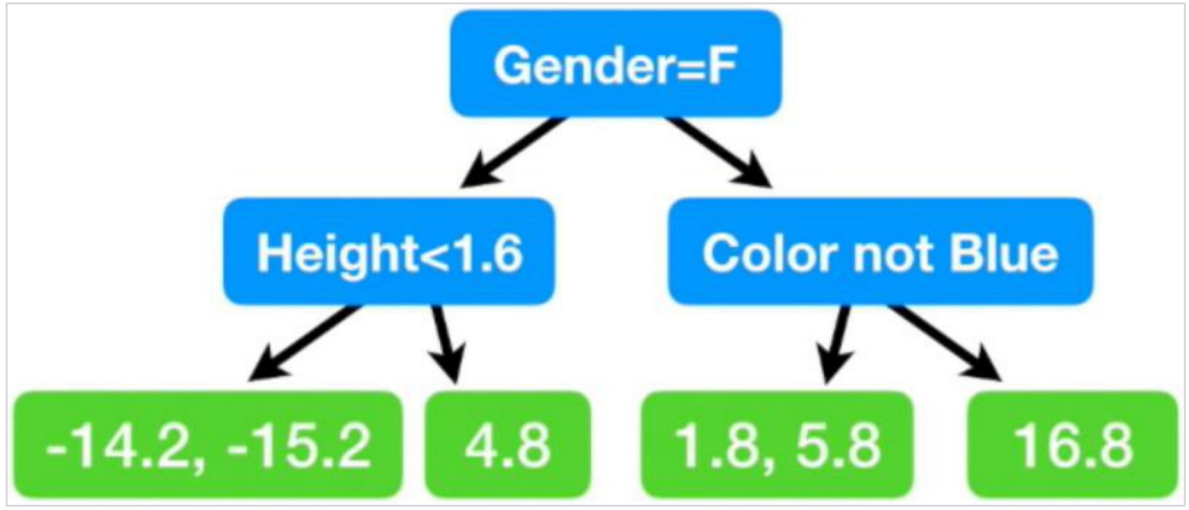
Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2

$$r_i = y_i - F_0(x_i)$$

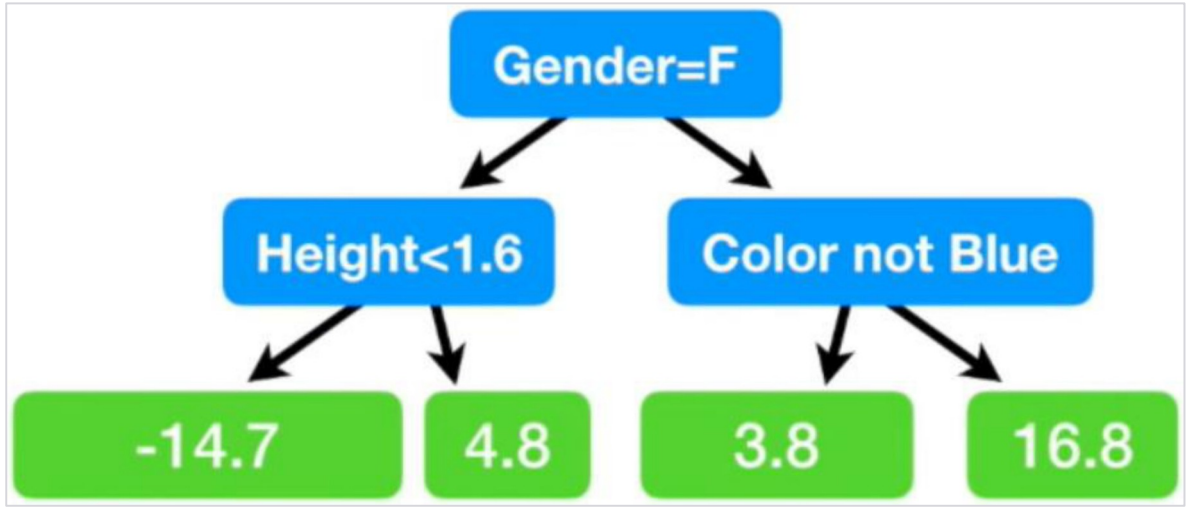
Step 3 — A Tree That Predicts the Residuals

Use height · color · gender to predict the residuals, not the weight. If a leaf has two values, store their mean

Assign residuals to leaves

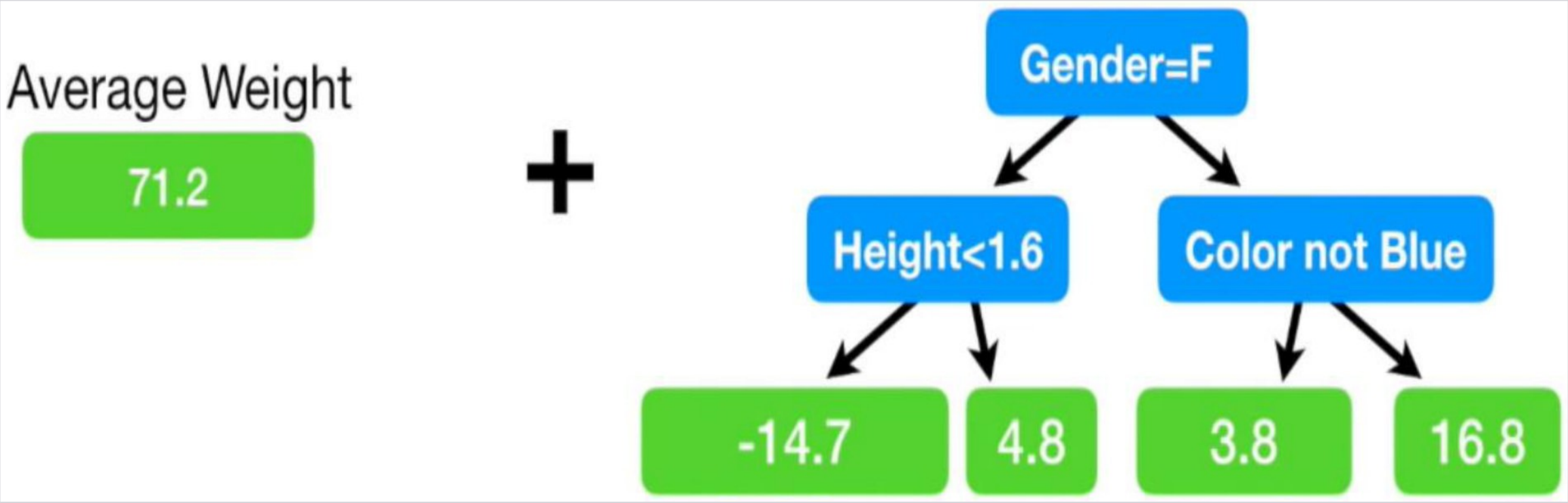


Two-value leaf → mean: (−14.2 − 15.2)/2 = −14.7



Step 4 — Combine the Initial Leaf and the Residual Tree

Prediction = average weight (71.2) + output of the residual tree



Predict weight by adding the initial tree (single leaf) and the second tree.

Step 4 — What If Male · Blue?

In the second tree, if gender is male and favorite color is blue, the residual is predicted as 16.8



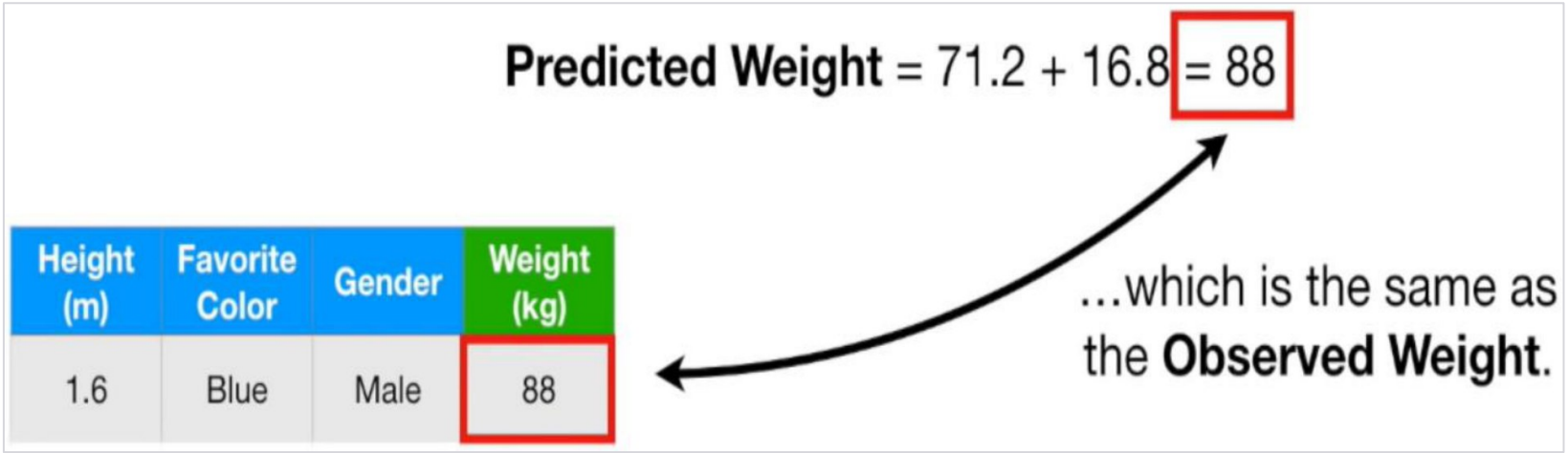
Predicted weight = 71.2 + 16.8 = 88 kg

Exactly Matches the Observed Value — Overfitting

Predicted 88 kg = actual 88 kg. No surprise

Why it is a problem

- We added the training residuals as they are, so of course it matches
- A model that overfits the training data
- It fits new data poorly (low bias, high variance)



The Fix — Multiply by a Learning Rate

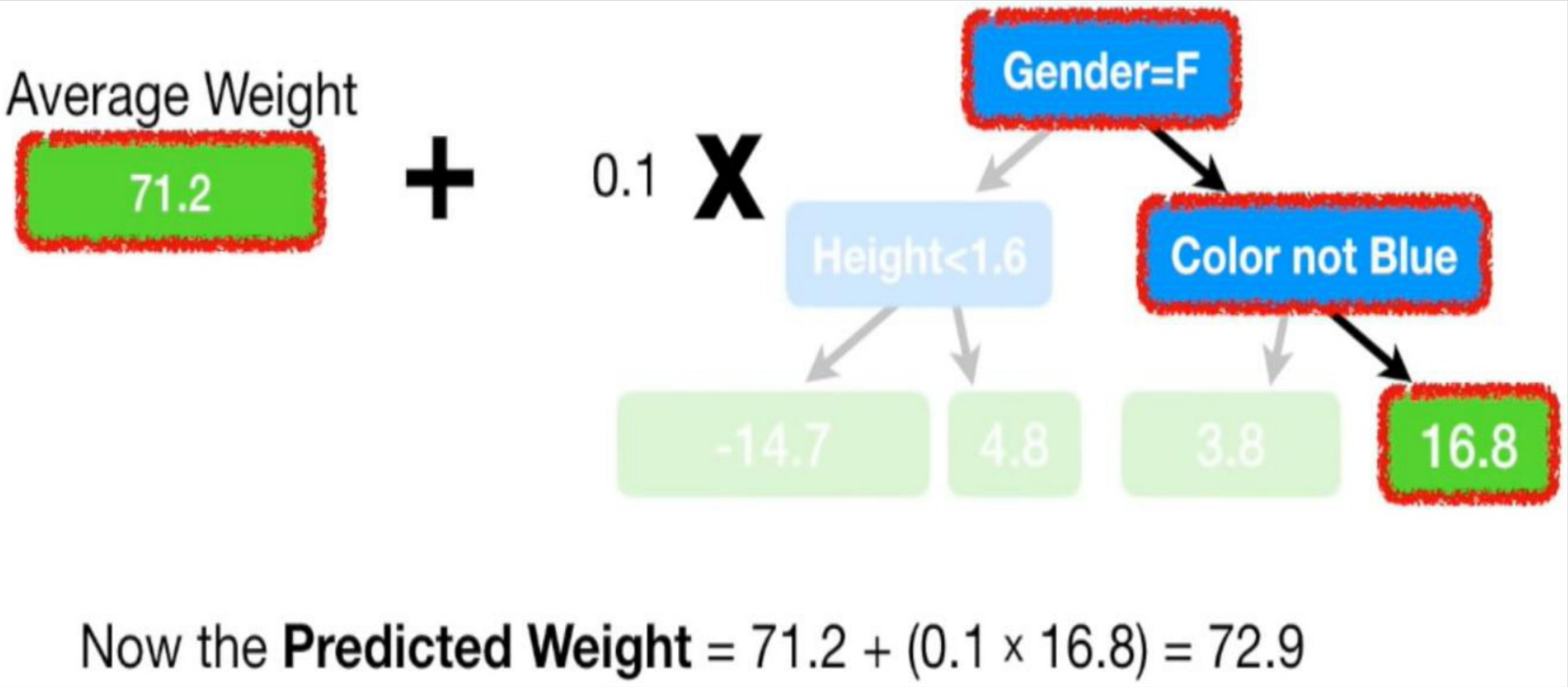
The learning rate is between 0 and 1. It is multiplied into the residual tree's output so each step adds only a little



$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad 0 < \nu \leq 1$$

Predicting Again with a Learning Rate of 0.1

Predicted weight = 71.2 + (0.1 × 16.8) = 72.9 kg



A little closer to the actual 88 kg than the 71.2 kg of the first single leaf.

Little by Little, in the Right Direction

72.9 kg is closer to the actual 88 kg than the first prediction of 71.2 kg

Principle

- Repeat small steps toward the actual value
- A model trained this way performs well on test data
- Friedman: a small learning rate (≤ 0.1) helps generalization

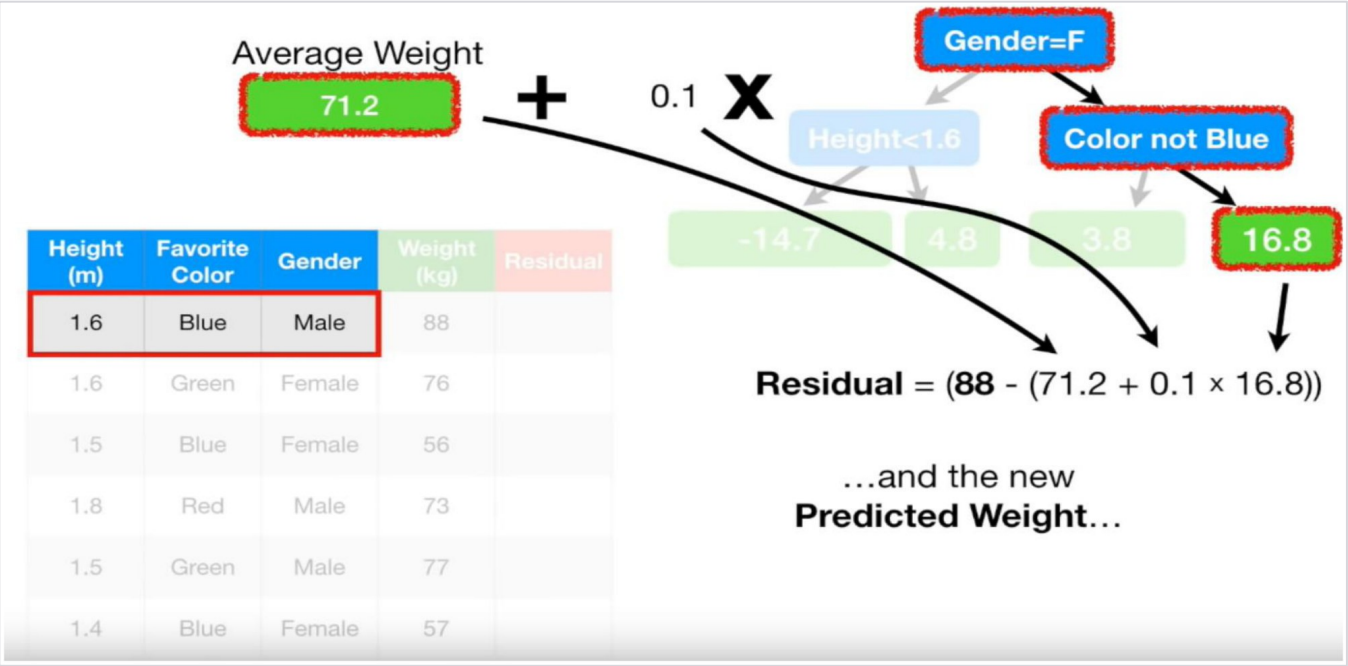
Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	
1.6	Green	Female	76	
1.5	Blue	Female	56	
1.8	Red	Male	73	
1.5	Green	Male	77	
1.4	Blue	Female	57	

← Residual = (Observed - Predicted)

Step 5 — Recompute the Residuals with the New Model

Residual = observed − (71.2 + 0.1 × tree output)

First person: 88 − (71.2 + 0.1 × 16.8)



Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	
1.6	Green	Female	76	
1.5	Blue	Female	56	
1.8	Red	Male	73	
1.5	Green	Male	77	
1.4	Blue	Female	57	

New residual column

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7

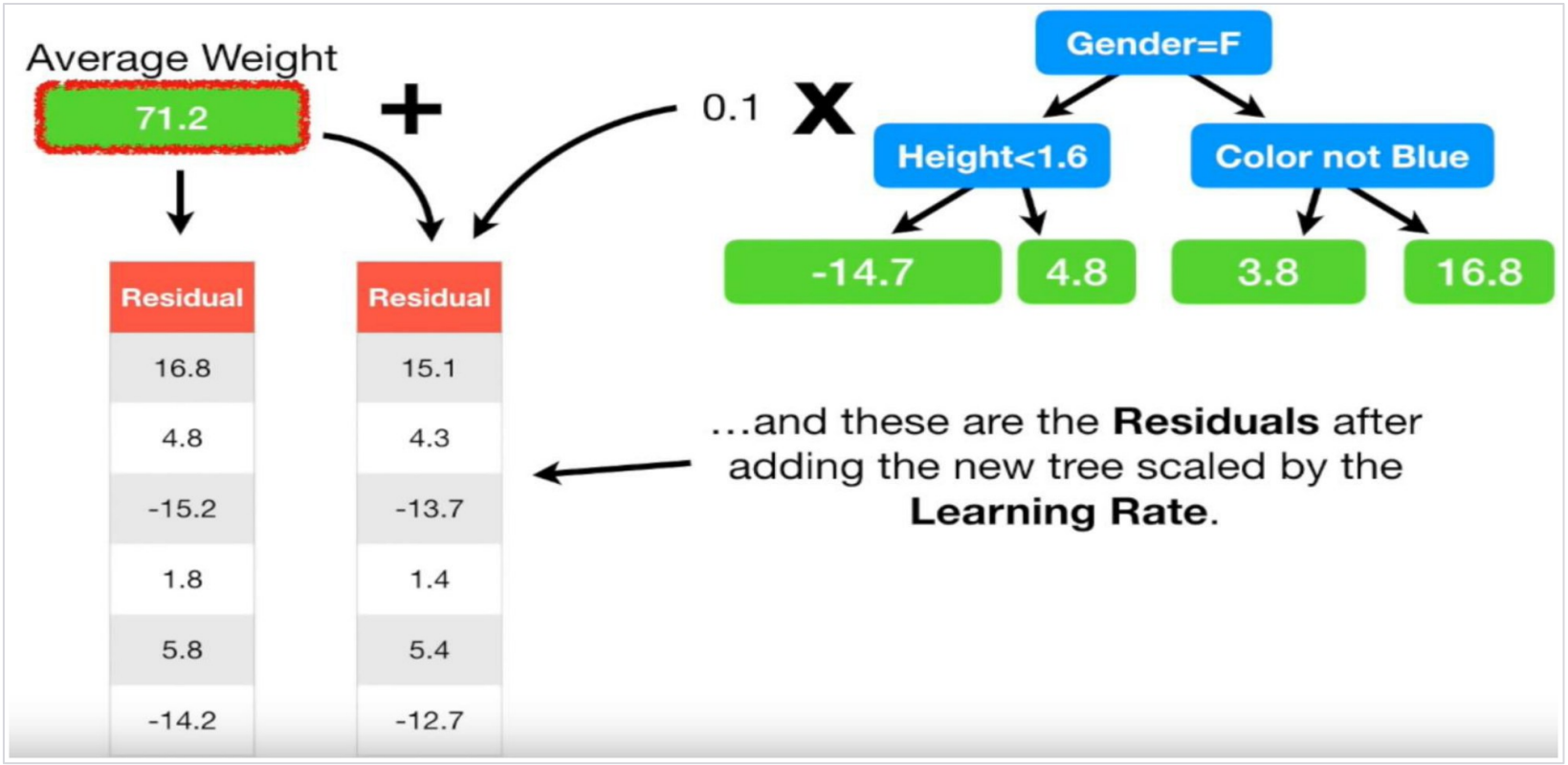
$$r = 88 - (71.2 + 0.1 \times 16.8) = 15.1$$

All the Residuals Shrank

Left: residuals using only the mean / Right: residuals after adding one tree

Observation

- 16.8 → 15.1, −15.2 → −13.7 …
- Every residual got smaller
- = the prediction is gradually approaching the actual values



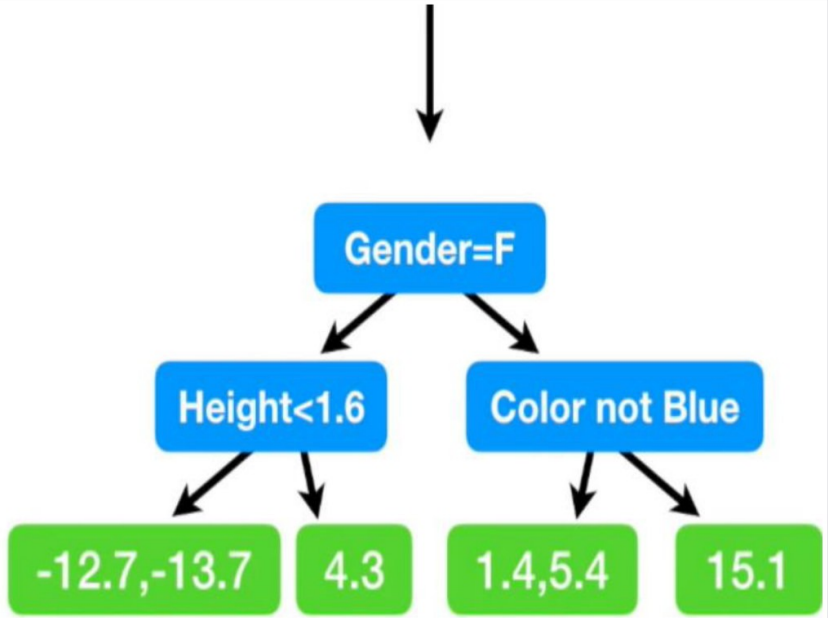
Step 6 — A Second Tree on the New Residuals

Leaves with two values take the mean: (−12.7, −13.7) → −13.2, (1.4, 5.4) → 3.4

Tree shape

- The example keeps using a 4-leaf tree
- In practice, the tree shape differs each iteration
- Typically trees with 8–32 leaves (StatQuest explanation)

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7

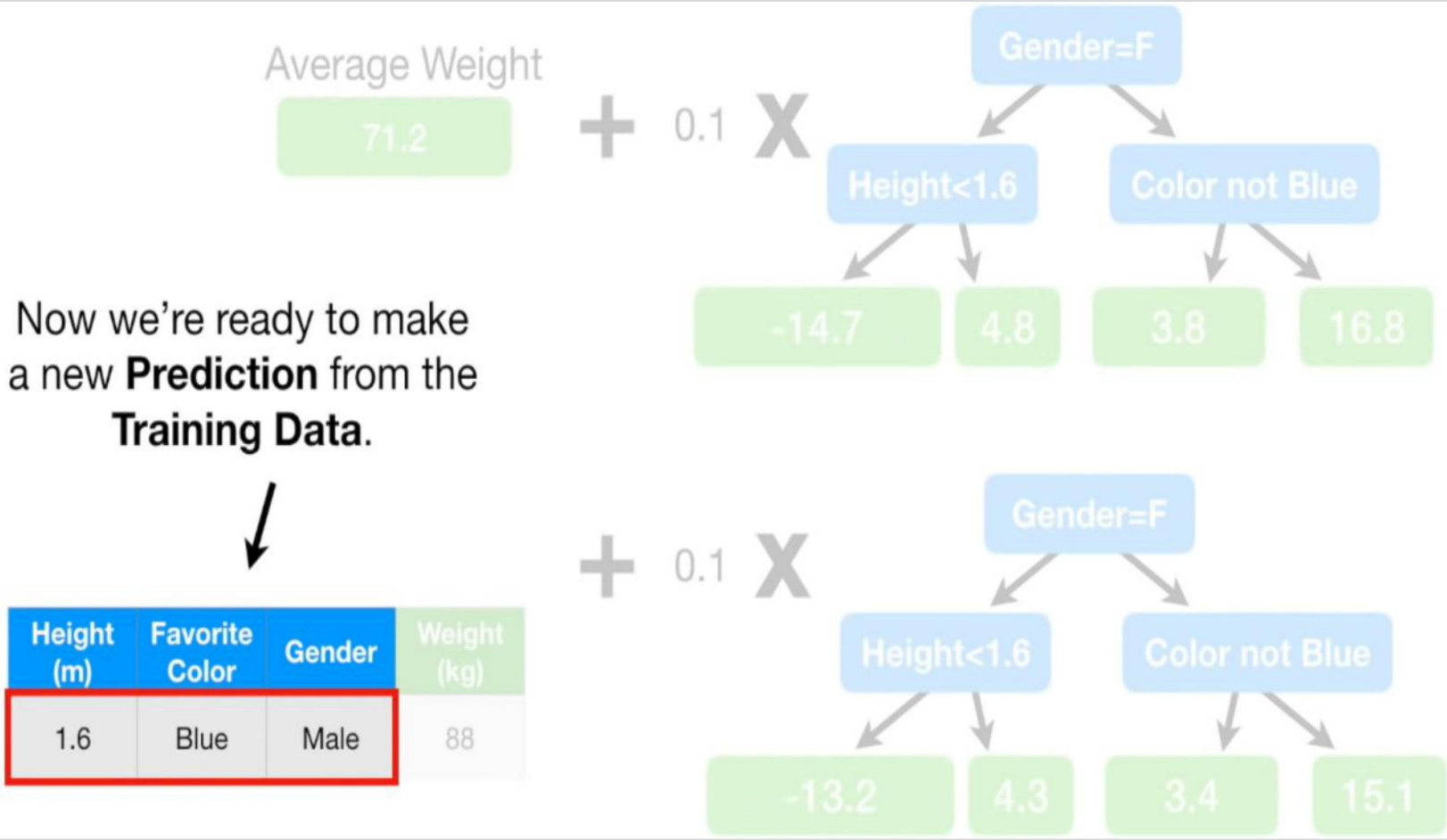


Step 7 — Add Up All the Trees So Far

Initial leaf + 0.1 × first residual tree + 0.1 × second residual tree

The model so far

- F_0 = 71.2 (single leaf)
- + 0.1 × first residual tree
- + 0.1 × second residual tree
- Predict the training data again with this model



Step 7 — Prediction for Height 1.6 m · Male · Blue

71.2 → 72.9 → 74.4 kg: gradually approaching the actual 88 kg



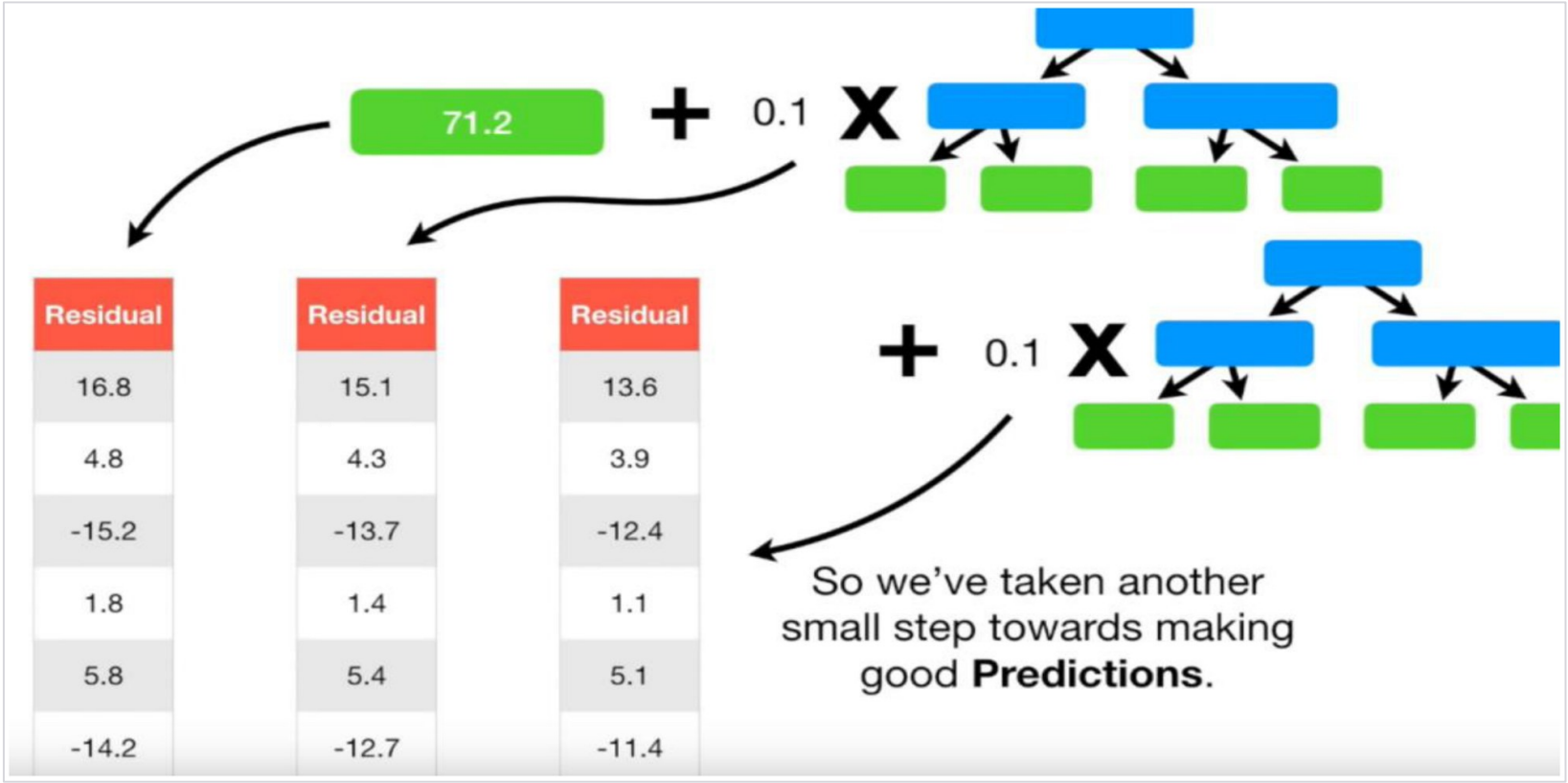
$$\hat{y} = 71.2 + 0.1 \times 16.8 + 0.1 \times 15.1 = 74.4$$

The Residuals Shrank One More Step

16.8 → 15.1 → 13.6: smaller with every iteration

Observation

- 71.2 → 72.9 → 74.4 → … → toward 88
- Smaller residuals = higher training accuracy
- Next slide: viewed through the sum of squared residuals

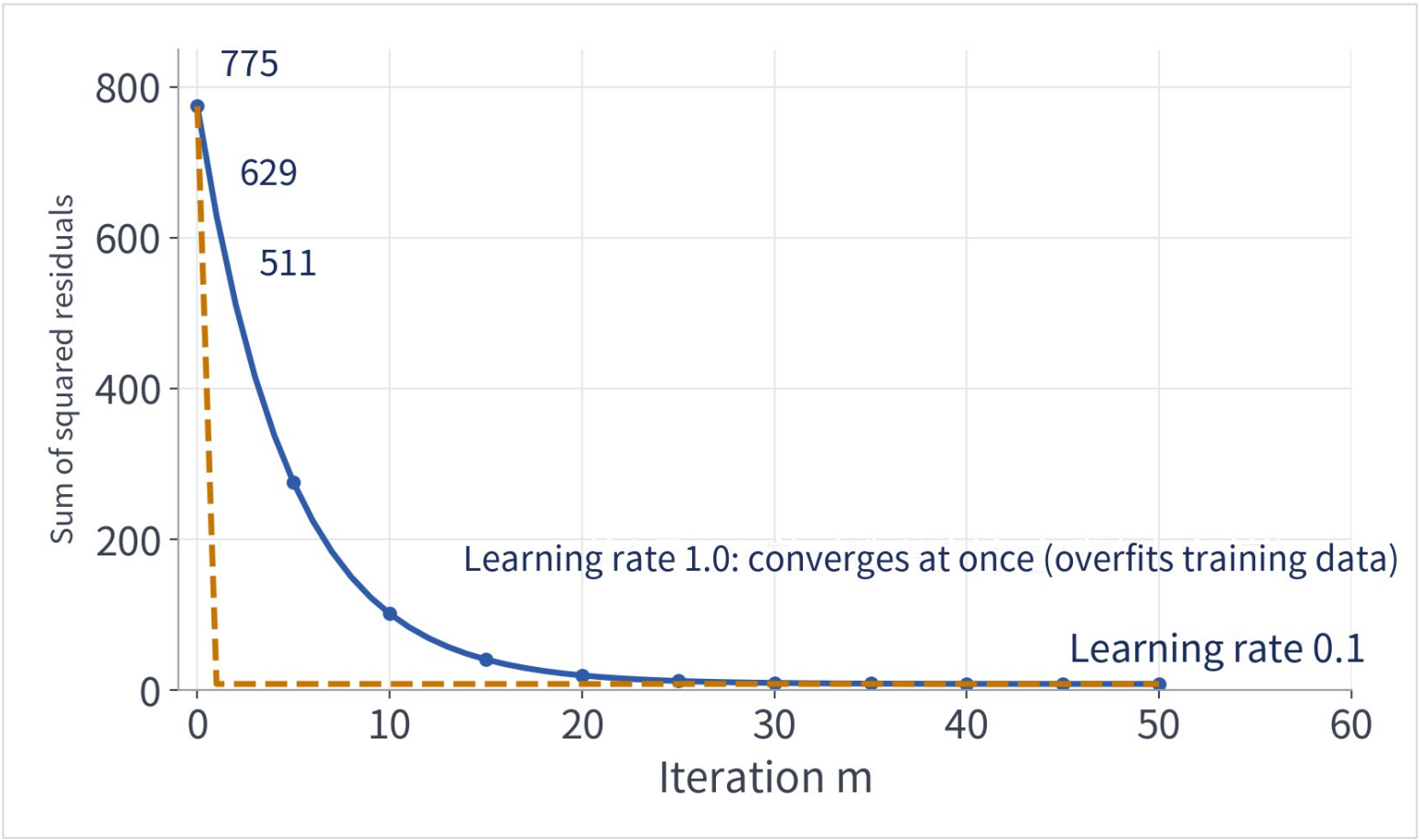


Supplement — Convergence Speed Seen Through the Sum of Squared Residuals

Keeping the original example's tree structure (4 leaves) fixed and continuing the iterations

How to read it

- Learning rate 0.1: 775 → 629 → 511 … decreases gradually
- Learning rate 1.0: hits bottom at once = memorizes the training data
- The floor (8.5) is the spread within the same leaf — a limit of the tree structure

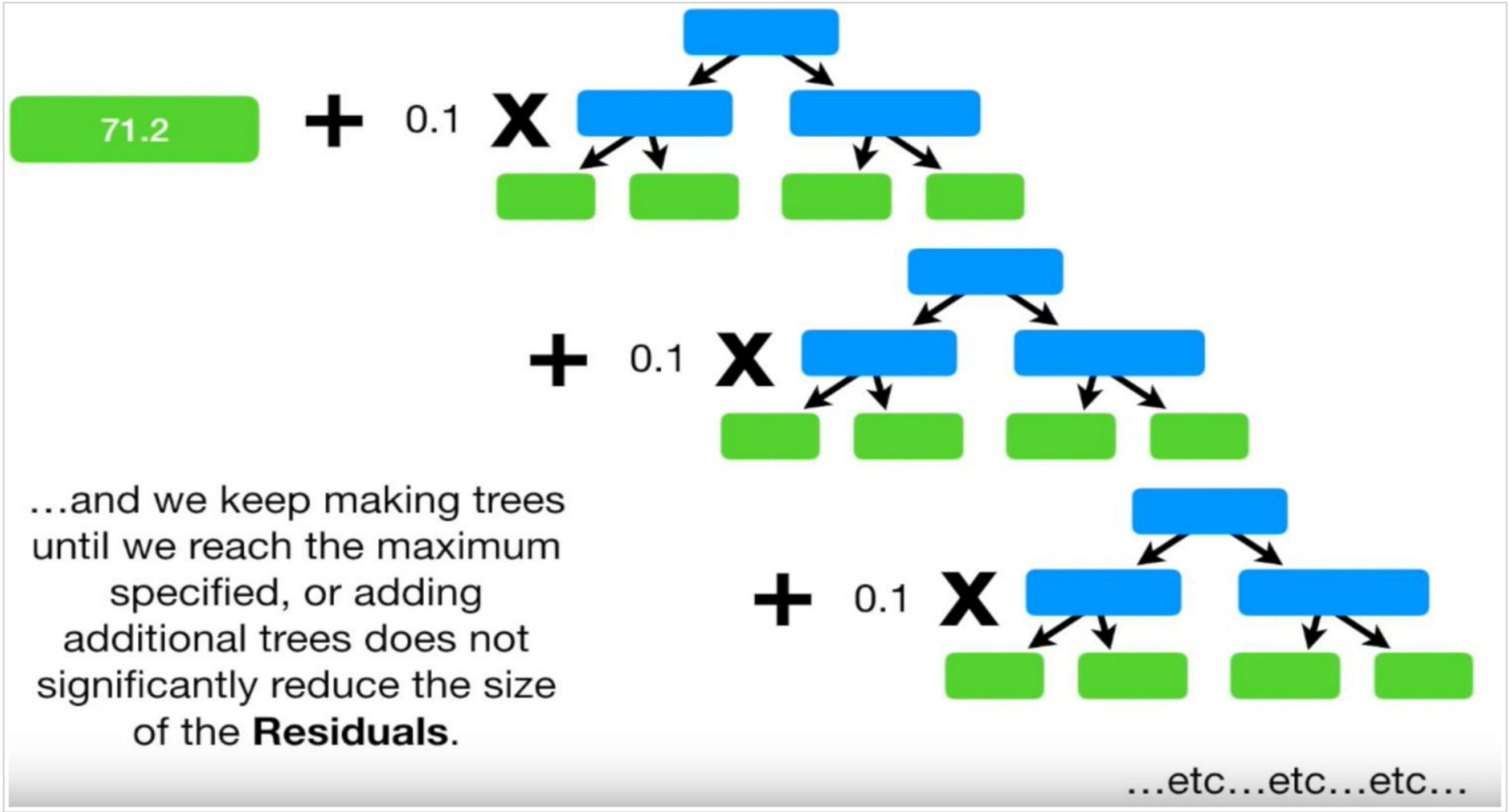


How Long to Iterate?

Until a preset number of iterations is reached, or adding trees no longer reduces the residuals much

Stopping conditions

- n_estimators (maximum number of trees)
- Early stopping when validation performance stops improving (n_iter_no_change)
- Tune the learning rate and number of trees together

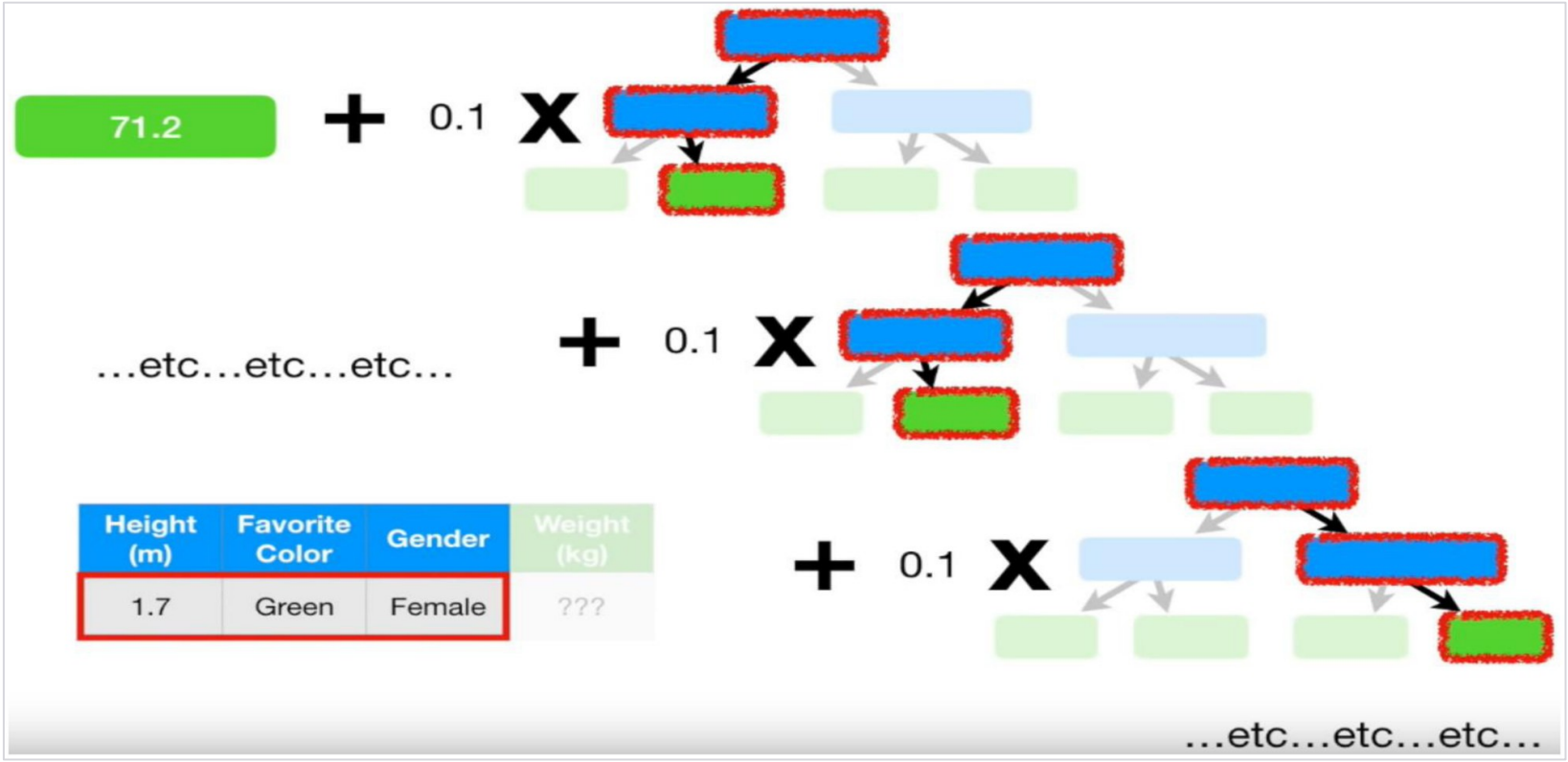


Predicting a New Person with the Finished Model

Height 1.7 m · green · female → initial value 71.2 + (learning rate × residual prediction) of every tree

Result

- For each tree, find the matching leaf value, multiply by 0.1, and add
- Result in the original lecture example: about 70 kg
- This Is How GBM Learns



Supplement — Why "Gradient" Boosting?

Under squared loss, the negative gradient of the loss is exactly the residual

$$L = \tfrac{1}{2}(y - F)^2 \quad \Longrightarrow \quad -\frac{\partial L}{\partial F} = y - F \;=\; \text{residual}$$

Gradient descent in function space

- Ordinary gradient descent: move parameters θ in the −∇L direction
- GBM: move the predictions $F(x_i)$ themselves in the −∂L/∂F direction
- Approximate that direction with a tree and step by the learning rate ν

What this buys us

- Change only the loss and the same framework does classification · quantile · robust regression
- The "residual" in the weight example is a special case (squared loss)
- AdaBoost = the special case with exponential loss

Supplement — Friedman's (2001) General Algorithm

Once the loss L is chosen, these four lines are all. Repeat for m = 1, ..., M

① Initial value

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

A single constant (the mean 71.2 under squared loss)

② Pseudo-residuals

$$r_{im} = -\left[\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

Negative gradient of the loss

③ Leaf values

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L\big(y_i,\ F_{m-1}(x_i) + \gamma\big)$$

Fit a tree to the pseudo-residuals, then the optimal constant per leaf

④ Update

$$F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm}\, \mathbf{1}(x \in R_{jm})$$

Multiply by the learning rate ν and add

Supplement — A Different Loss Means Different Pseudo-Residuals

What GBM's "next tree fits" is the negative gradient of the loss

Loss	$L(y, F)$	$-\partial L/\partial F$
Squared	$\frac{1}{2}(y-F)^2$	$y-F$
Absolute	$\lvert y-F\rvert$	$\operatorname{sign}(y-F)$
Huber	quadratic / linear	clipped residual
Log-loss	$-[y\log p+(1-y)\log(1-p)]$	$y-p$
Exponential	e^{-yF}	$y\,e^{-yF}$

Practical choice

- Squared: default regression (predicts the mean)
- Absolute · Huber: robust to outliers — fat-tailed returns
- Log loss: binary classification (default · churn)
- Quantile loss: quantile forecasts such as VaR

Supplement — GBM for Classification Stacks Log-Odds

Regression trees are used for classification too. What the trees add is not probability but log-odds

$$F_0 = \log \frac{\bar{p}}{1-\bar{p}}, \qquad p_i = \frac{1}{1+e^{-F(x_i)}}, \qquad r_{im} = y_i - p_i$$

Procedure

- Initial value = log-odds of the overall positive rate
- Pseudo-residual = observed (0/1) – predicted probability
- Fit tree → compute leaf values → scale by ν and add to the log-odds

Leaf value (log loss, one Newton step)

$$\gamma_{jm} = \frac{\sum_{x_i \in R_{jm}} r_{im}}{\sum_{x_i \in R_{jm}} p_i(1-p_i)}$$

XGBoost generalized this "gradient / second derivative" structure to any loss

Supplement — Five Knobs That Curb GBM Overfitting

Hyperparameter (scikit-learn)	Role	Tips
learning_rate (ν)	Shrinks each tree's contribution	0.01–0.1; lower means more trees
n_estimators	Maximum number of trees	Early stopping with n_iter_no_change
max_depth · max_leaf_nodes	Complexity of one tree = interaction order	Around 3–8 (default depth 3)
subsample	Uses only a subset of rows per tree	0.5~0.8 → Stochastic GB (Friedman 2002)
min_samples_leaf	Minimum samples per leaf	Prevents noisy leaves

With subsample < 1 you can also see the OOB improvement (oob_improvement_) — the OOB idea of Part 2 also serves boosting

Models That Advanced GBM — XGBoost · LightGBM · CatBoost

Original lecture: "think of them as faster, better-performing models" → what differs, on one slide

Model	Key idea	Strength
XGBoost (2014, paper 2016)	Second-order Taylor approximation + regularized objective · learns a default direction for missing values	Accuracy · parallelism; the Kaggle standard
LightGBM (2017)	Histogram splits · leaf-wise growth · GOSS · EFB	Speed on large data
CatBoost (2017)	Ordered boosting · categorical target statistics · symmetric trees	Data with many categoricals; strong default performance
HistGradientBoosting	scikit-learn implementation of the LightGBM approach · native NaN · categorical support	Much faster than GBM on large data

The next lecture (advanced boosting) covers the three models' objectives · splits · regularization in detail

Case — Predicting Credit Card Delinquency with Boosted Trees

Khandani · Kim · Lo (2010), Journal of Banking & Finance

2005.1–2009.4

Transaction and credit data on customers of a large commercial bank

85%

R^2 of actual on predicted delinquency rates

6~25%

Estimated cost savings (relative to total losses)

What they did

- A nonlinear model applying boosting to CART-type trees
- Combines card transaction histories with credit bureau data
- Out-of-sample prediction of the surge in delinquencies just before and during the financial crisis

Link to this lecture

- Trees: capture interactions among variables automatically
- Boosting: stack weak trees to reduce bias
- Credit-line cut decisions → lead to loss savings

06

Part 6 · Synthesis

Synthesis — Four Models in One Table, Then into Practice

Comparison table · asset pricing case · recent trends · practical checklist · discussion

The Four Models at a Glance

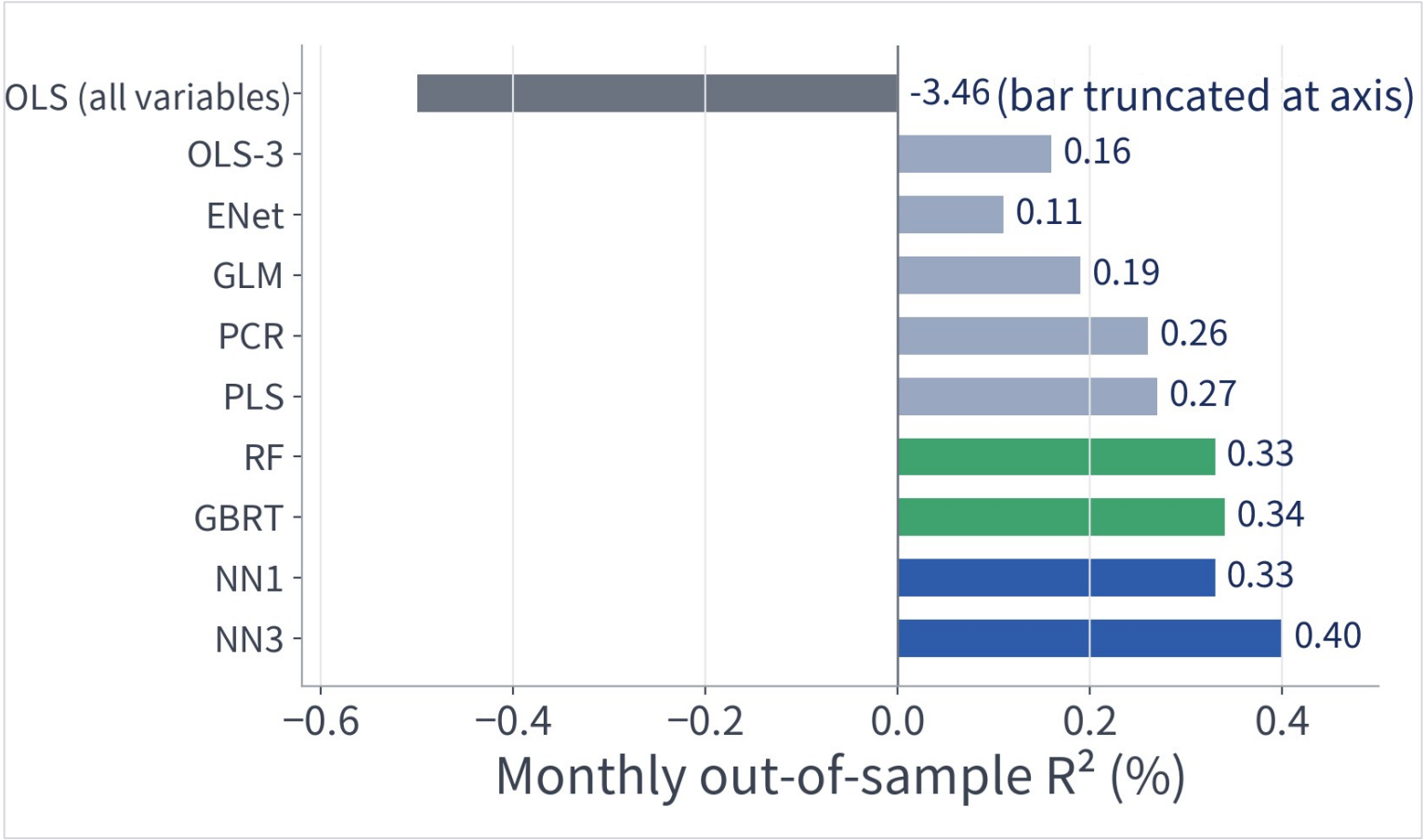
	Decision tree	Random forest	AdaBoost	Gradient boosting
Learning scheme	Single tree	Parallel · independent	Sequential · weights	Sequential · residuals (gradient)
Error mainly reduced	—	Variance	Bias	Bias
Base learner	Deep trees	Deep trees	stump	Shallow regression trees
Combination	—	Average · majority vote	α-weighted vote	ν × sum
Key knobs	max_depth	n_estimators · max_features	n_estimators · learning_rate	learning_rate · n_estimators · depth
Weaknesses	Overfitting · instability	Hard to interpret · no extrapolation	Sensitive to noise · outliers	Needs tuning · slow because sequential

Case — Predicting Stock Returns with Tree Ensembles

Gu · Kelly · Xiu (2020, RFS): about 30,000 US stocks · 1957–2016 · 94 firm characteristics + macro · industry variables

Key findings

- Trees (RF · GBRT) and neural networks beat linear models
- Reason: interactions among predictors
- Main signals: momentum · liquidity · volatility
- A monthly R^2 of 0.3–0.4% still makes a big difference for portfolios



Recent Trends — Where Tree Ensembles Stand on Tabular Data

Deep learning dominates images · text, but on tabular data GBDT has long been the baseline

Why they remain strong

- Grinsztajn et al. (NeurIPS 2022): tree models beat neural networks on 45 medium-sized tabular datasets
- Robust to irregular functions · uninformative features
- Almost no preprocessing (scaling) needed; native missing-value · categorical support

New challengers

- TabPFN v2 (Nature, 2025): a pretrained transformer that rivals tuned GBDT on small (≤ ~10,000 rows) tabular data
- Competing for rank on benchmarks such as TabArena
- Practical conclusion: keep GBDT as the baseline and compare new models against it

Checklist for Applying Them to Financial Data

Respect time order

Use time-series splits instead of random K-fold · prevent leakage with purging and embargo.

Make the target stationary

Trees cannot extrapolate. Predict returns · rates of change · ranks instead of price levels.

Treat missingness as information

Missingness patterns are systematic. Impute within folds, and consider missingness indicators.

Interpret importance with care

MDI is biased toward continuous · correlated features. Cross-check with permutation · SHAP.

Discussion Questions

Q1	The Gini gap between root candidates was only 0.004 (0.364 vs 0.360). How does this relate to the instability of a single tree?	**Part 1 · Variance**
Q2	In RF, if max_features is cut to 1, how do the correlation between trees and the accuracy of each tree change?	**Part 2 · Correlation**
Q3	When missingness is MNAR, like "poor performers delaying disclosure," is proximity imputation alone enough?	**Part 3 · Missing values**
Q4	Between AdaBoost and GBM (learning rate 0.1), which is more robust on default data with heavy label noise? Why?	**Parts 4–5 · Loss**

Summary

1 Trees: greedily pick the question that reduces impurity the most

Gini · entropy · weighted average · stopping rules. Deep trees overfit → control with depth · pruning.

2 RF: make trees different with bootstrap + random features, then average

Averaging still leaves variance from inter-tree correlation. Validate with OOB; fill missing values with proximity.

3 Boosting: keep fixing the previous model's mistakes

AdaBoost via weights, GBM via negative gradients (residuals). The balance of learning rate × number of trees is key.

Next: XGBoost · LightGBM · CatBoost — the same framework, with finer objectives and splits

References · Figure Sources

- Breiman, Friedman, Olshen, Stone (1984). Classification and Regression Trees.
- Breiman (1996). Bagging predictors. Machine Learning 24. / Breiman (2001). Random forests. Machine Learning 45.
- Freund & Schapire (1997). A decision-theoretic generalization of on-line learning… JCSS 55.
- Friedman, Hastie, Tibshirani (2000). Additive logistic regression. Annals of Statistics 28.
- Friedman (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics 29. / (2002) Stochastic gradient boosting.
- Viola & Jones (2001). Rapid object detection using a boosted cascade of simple features. CVPR.
- Khandani, Kim, Lo (2010). Consumer credit-risk models via machine-learning algorithms. JBF 34. / Gu, Kelly, Xiu (2020). Empirical asset pricing via machine learning. RFS 33.
- Bryzgalova, Lerner, Lettau, Pelger (2025). Missing financial data. RFS 38. / Grinsztajn et al. (2022). NeurIPS. / Hollmann et al. (2025). TabPFN. Nature.
- scikit-learn docs (1.3–1.9 release notes: missing-value support in tree models) · Figures: StatQuest (Josh Starmer) lectures, Müller & Guido, Introduction to Machine Learning with Python, Géron, Hands-On Machine Learning examples